

Maestro Scripting Language (MSL)

KI-optimierte technische Referenz · Basis: SCM-Dokumentation Rev. 17 vom 17.05.2022

Zweck. Verdichtete, textbasierte Referenz für das Erzeugen, Analysieren und Debuggen von XCS/MSL. Funktionsnamen, Parameterangaben, Einschränkungen und Beispiele stammen aus der Quelldokumentation. Wiederholende Erläuterungen, dekorative Seiten und redundante Vollbeispiele wurden reduziert.

Wichtige Quellenregel. Die Originaldokumentation enthält einzelne Tippfehler und Inkonsistenzen in Tabellen/Beispielen. Diese Fassung erfindet dafür keine stillen Korrekturen. Bei produktivem Einsatz gegen Maestro/XConverter validieren.

Ausführungsmodell. MSL basiert auf C#. Anweisungen werden sequenziell interpretiert. Neu erzeugte Workpieces, Planes, Geometries, Machining Operations und Workplans werden jeweils zum aktiven Objekt; nachfolgende Befehle können implizit auf diesen aktiven Kontext wirken. Namen müssen innerhalb eines Scripts eindeutig sein; leere Namen/Beschreibungen erlauben Maestro, Standardnamen zu vergeben.

Schnellnavigation

Project & Machining Parameters · Parametrics · WorkPiece · WorkPlan · Planes · 2D Geometries · Machining Operations · Lead-in/Lead-out · Strategies · Machine Functions · Attributes · Correction/Compensation · Explicit Toolpath · Technologies · Feedrate Profiles · Macro · NC Functions · Workpiece Blocking · Mathematical/Additional Functions · Tools · Edge Banding · Labeling · Nesting · XConverter · Appendix

Maestro scripting language (MSL) is a programming language that, through the Maestro platform primitives, is used to create scripting in textual format to create:

1. Machining-macros
2. Machining programs

MSL is based on Microsoft's C# programming language; therefore all its instructions and constructs can be used. For the language syntax see the C# programming language reference manuals (an extensive literature on the subject can be found in the internet).

Programming via scripting provides an extremely powerful textual programming language and provides instructions that allow access to all the operational functions that can be performed interactively with Xilog-Maestro: definition of the shape and position of the workpieces, machining assignment, entering machine functions Furthermore it allows the use of parametrics, flow control (conditional instructions, cycles, jumps,...) and algebraic expressions.

Maestro scripting language (MSL) is aimed at:

1. Process technologists, advanced users and SCM branches/agents, need a flexible and powerful instrument to produce machining-macros or parametric machining programs autonomously.
2. External software houses (operational software, CAD/CAM, etc...) that must produce post-processors for machining programs in PGMX format for Maestro. MSL in fact can be used to construct a complete project, with a description of the workpieces to machine, the geometries and machining and the phases required for the machining sequence. In short it is the Xilog-Maestro input point for external applications.

In particular, the second case applies to situations where the woodworking workshop already has software to manage the production of door and window frames, stairs, furniture or other products, and it needs to transfer the information to Maestro.

This manual describes how to transfer the geometric data and the data that describes the entire machining process for a specific workpiece to Maestro.

Various instruments or methods available in the Maestro environment will be used to transfer the machining process.

2 Basic Concept

The main basic objects that can be managed with MSL and that can be defined and created in a generic script are:

1. Machining parameters
2. Workpiece
3. Plane
4. Geometry
5. Machining operation
6. Workplan

The instructions of a generic scripting file are interpreted and executed in sequence.

During the scripting execution phase, when a new object part of one of the basic types defined in the list is created, the object created automatically becomes the active object for the specific type.

For every type of object listed, during the scripting execution an updated state is maintained with the active object, the last one created during the interpretation.

The active object is used by the system to automatically associate the information necessary to create new objects.

See below for some application examples:

1. Geometries: when a geometry is created, the plane that it is associated to is the active plane (current plane).
2. Machining operation: when a machining operation is created it is associated with the geometry and the active workpiece. Furthermore, the operation is associated to the current work phase or, if no work phase has been defined, it is associated to the default phase.

For the complete list of the instructions available to create scripting files in .XCS format, see the XilogMaestroScripting.chm manual in the Languages folder found in the folder that Xilog Maestro is installed in.

Many of the instructions require the Name and Description string type of parameters, in order to associate a name and description to the instruction. The system checks that a script does not contain more than one instruction with the same name. It is therefore important to use different names for the Name parameter. The user can also choose not to specify any value either for the Name or the Description, by entering an empty string as the parameter value (e.g. ""). In this case the system will associate a correct default name to the object created.

3 Project Project refers to an object that can contain the definition of one or more workpieces with 2D geometries and associated machining.

The project contains all the information needed to define a machining process. The project must have one or more workpiece, where could be defined reference geometries to create machinings.

3.1 Project management

The project is managed in MSL by writing .xcs files.

3.1.1 Creating programs (.xcs)

The programs (XCS) are used to define a complete part program to machine one or more workpieces in textual mode. They can be imported interactively in the Maestro

editor (see script import function) or they can be converted in pgmx programs, in

batch mode, with the XConverter tool that is distributed with the Maestro software.

The definition of machining programs with the scripting language is usually used so that from an external environment to Maestro or with a generic post-processor, a complete workpiece can be defined without using the interactive environment.

To create a program in .xcs format there is no specific help function. The program must be created on a text file with .xcs extension and it must contain only instructions recognized by Maestro, described in this manual and the basic instructions of the language C#.

For the complete list of the instructions available to create scripting files in .XCS format, see the XilogMaestroScripting.chm manual in the Languages folder found in the folder that Xilog Maestro is installed in.

The program as well as containing recognized instructions and with the correct syntax, it must also have a set of instructions compatible with each other and have the definition of the finished workpiece.

See below for an example of a workpiece program defined with the scripting language.

```
// definitions for the program header
SetMachiningParameters("AB", 1, 21, 1114112, false);
// definition of the finished workpiece
CreateFinishedWorkpieceBox("Frame_Right",937.464,80,68);
// definition of the raw workpiece
CreateRawWorkpiece("Frame_Right",2,2,2,2,0,0);
// definition of the workpiece position
SetWorkpieceSetupPosition(4.25,423,0, 0);
//definition of the external blocking profile
CreateBlockingProfile("Frame_Right_2",939.46,82,-15);
AddSegmentToBlockingProfile(35.46,82, "");
// definition of blade cut
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(true, true, 1.5);
ResetDustpan();
CreateSegment("Blade Cut at 0 mm_4", 937,464, 0, 937,464, 80);
SetRetractStrategy(true, true, 1.5, 0);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 0 mm_5",0,"Blade Cut at 0 mm", TypeOfProcess.GeneralRouting ,"E025","3",2,-1,-1.5,0);
// definition of blade cut
ResetApproachStrategy();
ResetRetractStrategy();
ResetDustpan();
CreateSegment("Blade Cut at 5 mm_8", 30.29, 79,789, -8,667, -3,398);
SetPneumaticHoodPosition(1);
ResetAuxiliaryHood();
CreateSlot("Blade Cut at 5 mm_9",0,"Blade Cut at 5 mm", TypeOfProcess.GeneralRouting ,"E025","3",2,-1,-1.5,0);
// head routing definition
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetDustpanOffset(10, 0, false);
CreatePolyline("Frame External Routing_10",37,464,80);
AddArc2PointRadiusToPolyline(0,0, 850,false,false);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(3);
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing1",0,"Frame External Routing", TypeOfProcess.GeneralRouting
,"E023","3",2,-1,-1.5,0);

// Definition of the Swap phase
CreateWorkplan("Swap");
// definition of the workpiece position
SetWorkpieceSetupPosition(4.25,423,0, 0);
//definition of the internal blocking profile
CreateBlockingProfile("Frame_Right_3",0,0, -42);
AddSegmentToBlockingProfile(937.464280,0, "");
// definition of routing with the grain
ResetApproachStrategy();
ResetRetractStrategy();
SetApproachStrategy(false, true, 1.5);
SetDustpanOffset(10, 0, false);
CreatePolyline("Frame External Routing_6",937,464,80);
AddSegmentToPolyline(37,464.80);
SetRetractStrategy(false, true, 1.5, 0);
SetPneumaticHoodPosition(3);
ResetAuxiliaryHood();
CreateRoughFinish("Frame External Routing2",0,"Frame External Routing", TypeOfProcess.GeneralRouting
,"E023","3",2,-1,-1.5,0);
```

3.2 Machining parameters

Machining parameters are a set of parameters associated with the project that let to load and execute it on machine. Machining parameters definition is better to do as first instructions of the project, before to create all necessary objects to create the project.

3.2.1 Reference parameters

Defined with instruction SetMachiningParameters.

This instruction is used to configure the machine parameters used to execute the program. Generally this function must be the first instruction in the script, because it configures the main parameters for the correct execution of the program. This instruction is not necessary for programs for PowerFlex machines. The table below shows the input parameters for the

instruction.

Name Type Description executionFields String Work area repetitions Int32 Number of repetitions tableOptions Int64 Type of blocking configuration mechanicalOptions Int64 Mechanical options configuration continuousCycle Bool Enables the continuous cycle execution

The example below shows an extract code where the SetMachiningParameters instruction is applied on an operation in AB area machine that requires only one execution repetition:

```
SetMachiningParameters("AB", 1, 0, 1114112, false);  
CreateFinishedWorkpieceBox("Frame_Left", 1200, 70, 63);  
SetWorkpieceSetupPosition(10, 435, 0, 0);
```

Instruction returns an object of type MachineParameters that gives access to project machining parameters. Same object could be extracted using the instruction GetMachineParameters.

3.2.2 Y position of workpiece referred to opposite side stop

Defined with instruction SetRelatedToOppositeSideStop. The instruction is used to set Y value of the mobile side stop to load the workpiece.

Nome Tipo Descrizione isRelatedToOppositeSideStop Bool true if workpiece has to be referred to opposite side stop, otherwise false. SetRelatedToOppositeSideStop parameters

Esempio :

```
SetRelatedToOppositeSideStop(true);
```

3.2.3 Configuration default blocking options

Defined with instruction SetRelatedToOppositeSideStop. The instruction is used to use the default blocking type set in the machine panel configuration When set with the true value, this function actually overrides the setting of the "tableOptions" input parameter of the SetMachiningParameters instruction.

Name Type Description useDefaultForTableOptions Bool true for configuration blocking options, otherwise false.

SetDefaultForTableOptions parameters Example:

```
SetDefaultForTableOptions(true);
```

3.2.4 Technological compatibility on mirror working fields

Defined with instruction SetTechnologicalMirror. This instruction is used to maintain tool technology along toolpath, climbing or conventional, also on mirrored working fields. Machining is modified in the way the tool works with the same technology also on mirrored working fields.

Name Type Description isTechnologicalMirror Bool true to maintain tool technology on X or Y specular working fields, otherwise false. SetTechnologicalMirror parameters Example:

```
SetTechnologicalMirror(true);
```

3.2.5 Automatic table set up on machine program loading

Defined with instruction SetAutoSetup. This instruction is used to set up the machine table automatically on program loading on the machine. In this way is not necessary to set up the machine table entering "Set up" in Maestro. Set up is executed on machine program loading.

Name Type Description autoSetup Bool true to enable automatic setup during Post Processing, otherwise false.

SetAutoSetup parameters Example:

```
SetAutoSetup(true);
```

3.2.6 Workpiece side presser exclusion at program

beginning

Defined with instruction SetRelatedToOppositeSideStop. Instruction is used to exclude the workpiece side presser at program beginning. Valid for CX100 machine. For other machines it takes no effect.

Name Type Description isWorkPieceSidePresserDisabledOnStart Bool true to disable side presser at part program start, otherwise false. SetWorkPieceSidePresserDisabledOnStart parameters

Example:

```
SetWorkPieceSidePresserDisabledOnStart(true);
```

3.2.7 Maintaining the process sequence

Defined with instruction SetRelatedToOppositeSideStop. Instruction is used to maintain the original sequence of program operations even after optimization.

Name Type Description preserveWorkingSequence Bool true to maintain part program executable sequence on optimized part program, otherwise false. SetPreserveWorkingSequence parameters Example:

```
SetPreserveWorkingSequence(true);
```

3.2.8 Process sequence modification due to multiple

drillings

Defined with instruction SetRelatedToOppositeSideStop. The instruction is used, even if maintaining the machining sequence is enabled, to modify the optimised program to perform multiple drills in one operation.

Name Type Description multipleDrillingsCanChangeWorkingSequence Bool true to allow multiple drilling to modify program executable sequence, otherwise false. SetMultipleDrillingsCanChangeWorkingSequence parameters Example:

```
SetMultipleDrillingsCanChangeWorkingSequence(false);
```

3.2.9 Workpiece fall prevention

Defined with instruction SetWorkPieceFallPrevention. The instruction is used to enable the workpiece fall prevention function in case of long workpiece. This avoids opening clamps at program ending. Valid for CX machine. For other machines it takes no effect.

Name Type Description useWorkPieceFallPrevention Bool Enable anti-fall function SetWorkPieceFallPrevention parameters Example:

```
SetWorkPieceFallPrevention(true);
```

3.2.10 ToeKick scrap unload

Defined with instruction SetToeKickWait . The instruction is used to enable the activation of scrap unload in the presence of ToeKick.

Name Type Description waitForLeftToeKick Bool true to enable left ToeKick scrap unload. waitForRightToeKick Bool true to enable right ToeKick scrap unload. SetToeKickWait parameters Example:

```
SetToeKickWait (true,true);
```

3.2.11 Program optimization enable

Defined with instruction SetOptimization. The instruction is used to enable the activation of program optimization on machine loading.

Name Type Description enableOptimization Bool true to enable automatic optimization during Post Processing, otherwise false. SetOptimization parameters

Example:

```
SetOptimization(true);
```

3.2.12 Comment definition

Defined with instruction SetComment. The instruction is used to create a comment to the project.

Name Type Description comment string comment to the project.

SetComment parameters

Example:

```
SetComment("Project comment");
```

3.2.13 Description definition

Defined with instruction SetDescription. The instruction is used to create a description to the project.

Name Type Description description string description to the project.

SetDescription parameters

Example:

```
SetDescription("Project description");
```

3.2.14 Dynamic working fields enable

Defined with instruction SetEnableDynamicAreas. The instruction is used to enable dynamic working fields handling if it is enabled on machine configuration. Dynamic working fields handling lets to increase or decrease the working field length on a semi part of the machine table, in the way a project can use a different working field respect to configuration working field definition.

Name Type Description enableDynamicAreas bool true to enable dynamic areas use, otherwise false.

SetEnableDynamicAreas parameters

Example:

```
SetEnableDynamicAreas(true);
```

3.2.15 Edge banding optimization

Defined with instruction SetEdgeBandingOptimization.. The instruction is used to enable Edge Banding process optimization. These options are used at program loading time to optimize the edge banding process.

Name Type Description isRotationOptimizationEnabled bool true to enable edgebanding head rotation, otherwise false.

isAutostartOptimizationEnable bool true to enable Autostart machine function, d otherwise false.

isClosingOptimizationEnabled bool true to enable closing optimization, otherwise false. isLampOptimizationEnabled bool

true to enable lamp optimization, otherwise false. isApproachOptimizationEnable bool true to enable lead in optimization, d otherwise false. isRetractOptimizationEnabled bool true to enable lead out optimization, otherwise false.

isRollStopOptimizationEnabled bool true to enable speed optimization, otherwise false. isSpeedOptimizationEnabled bool

true to enable speed optimization, otherwise false. isTrimmingOptimizationEnable bool true to enable trimming rotation d optimization, otherwise false. SetEdgeBandingOptimization parameters

Example:

```
SetEdgeBandingOptimization(true, true, true, true, true, true, true, true, true,
true);
```

3.2.16 Workpiece machinings shift

Defined with instruction `SetWorkPieceShift`. The instruction enable the workpiece machinings shift after probing. Shift could be automatic, probing values along X and/or Y are applied to move machinings, or manual, X and/or Y shifts applied are the value passed as parameters.

Name Type Description `isAutomaticWorkPieceShiftEna` bool `<c>true</c>` to enable automatic probing bled shift, `<c>>false</c>` manual probing shift.

`workPieceShiftX` double Manual shift along X. Valid only if `isAutomaticWorkPieceShiftEnabled` is false. `workPieceShiftY` double Manual shift along Y. Valid only if `isAutomaticWorkPieceShiftEnabled` is false.

`SetWorkPieceShift` parameters

The example below set a manual shift of 2 along X and 3 along Y to machinings after probing.

```
SetWorkPieceShift(false, 2, 3);
```

The example below sets an automatic shift to machinings after probing.

```
SetWorkPieceShift(true, 0, 0);
```

3.2.17 Intermediate unloading

Defined with instruction `SetIntermediateUnloadingType`. The instruction sets the intermediate unloading for clamps machine with intermediate unloading available.

Name Type Description `intermediateUnloadingType` bool Intermediate unloading type 0 - Not Specified. 1 - Right unload. 2 - Left unload. 3 - Right unload with clamp repositioning. 4 - Left unload with clamp repositioning. 5 - Right unload with clamp repositioning and probing. 6 - Left unload with clamp repositioning and probing.

`SetIntermediateUnloadingType` parameters

The example below set the intermediate right unload

```
SetIntermediateUnloadingType(1);
```

3.3 Parametrics

3.3.1 Parameter creation

Defined with instruction `AddVariable`.

The instruction creates a parameter that can be used to create parametric objects or to create objects that can be parametric calculated.

Nome Tipo Descrizione name string Name of parameter value Object Value of parameter (Double,Int32,String,Bool) type int Type of parameter : 0 - Decimal 1 - Integer 2 - Boolean. unitType int Physical unit of parameter : 0 - Adimensional 1 - Length (mm) 2 - Speed (m/min) description string Description of parameter isReadOnly bool true to define a read only variable, otherwise false to define a modifiable variable. isExportable bool true to define a variable visible and modifiable from Machine Panel, false to define a variable not visible and not modifiable from machine panel. `AddVariable` parameter

The example below creates different types of parameters, length type, speed type, angle type, conditional type and number type.

```
AddVariable("Length", 100, 0,1, "Length",false,true);
AddVariable("Speed", 8, 0,2, "Speed", true,true);
AddVariable("Angle", 45, 0,0, "Angle", true,true);
AddVariable("Condition",true, 2,0,"Enable",false,true);
AddVariable("Number",true, 1,0,"Number",false,true);
```

3.3.2 Getting parameter value

Defined with instruction `GetVariableTypedValue<T>`. The instruction gets the value of a parameter previously defined from its name. If the type T must be the same as the parameter (integer, decimal, boolean).

This allows during machine panel xcs loading (available only with Active), to specify a different value than default value for parameters, and gives the possibility to create objects with different values than default.

Nome Tipo Descrizione name string Name of parameter T type of parameter `GetVariableTypedValue` parameters

The example below shows how to get a parameter value in order to use it to create other objects. Two parameters are created with name X and Y and using the instruction `GetVariableTypedValue` the value is assigned to the local variable of double type. Those 2 local variables could be used as input parameters in instruction methods.

```
AddVariable("X", 100, 0,1, "Posizione X",false,true);
AddVariable("Y", 200, 0,1, "Posizione Y",false,true);
double x = GetVariableTypedValue<double>("X");
double y = GetVariableTypedValue<double>("Y");
CreatPoint("Point", x, y);
```

In this case, statically, x and y values are equal to default values 100 e 200 and created point will have this coordinates, but if xcs file will be loaded in machine panel (available only with Active), it is possible to pass different values to X and Y (example 300, 400) and instruction returned values will be new passed values, so the point will be created with coordinates 300, 400. In this way, if the 2 parameters are used to create another object (example a point), this object will be created with passed values instead of default values.

3.4 WorkPiece

The workpiece must be defined as either “finished workpiece” or “raw workpiece”. More than one workpiece can be entered in a project.

3.4.1 WorkPiece creation

3.4.1.1 Finished WorkPiece

The dimensions and the shape of the finished workpiece can be specified with various instructions.

3.4.1.1.1 Box WorkPiece

Defined with instruction CreateFinishedWorkpieceBox. The WorkPiece is created as Box from length, width and depth.

Name Type Description name string Workpiece name dx double Workpiece length dy double Workpiece width dz double Workpiece thickness

In the example below a workpiece called “Example workpiece” with dimensions 1800x800x18 is created:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
```

3.4.1.1.2 Extrusion WorkPiece

Defined with the instruction CreateFinishedWorkpieceFromExtrusion. The instruction creates a workpiece from extrusion along axis Z starting from a closed geometry defined on the XY plane. The reference geometry for the workpiece “boundary” is the active geometry defined in the script before the call at the instruction CreateFinishedWorkpieceFromExtrusion. The instruction let also to create workpiece holes, simply passing geometry names as input parameter. Hole geometries must be created previously to call the instruction.

Name Type Description Name string Workpiece name Dz double Workpiece thickness internalProfiles string[] (opt) List of internal profiles geometries names CreateFinishedWorkpieceFromExtrusion parameters

See the example below for the definition of a finished and raw of an arch workpiece

```
// Create finished workpiece
CreatePolyline("",70,600);
AddArc2PointRadiusToPolyline(1130,600,530,true,false);
AddSegmentToPolyline(1200,600);
AddArc2PointRadiusToPolyline(0,600,600,false,false);
AddSegmentToPolyline(70,600);
CreateFinishedWorkpieceFromExtrusion("Telaio_Up",63);
```

3.4.1.2 Raw WorkPiece

The dimensions and the shape of the raw workpiece can be specified with various primitives.

3.4.1.2.1 Over materials WorkPiece

Defined with the instruction CreateRawWorkpiece. Raw WorkPiece is created as a box from the finished workpiece box, increasing it by overmaterials.Overmaterials are applied to the standard plane of finished workpiece.

Name Type Description Name string Workpiece name

bx1 double Machining allowance in x- bx2 double Machining allowance in x+ by1 double Machining allowance in y- by2 double Machining allowance in y+ bz1 double Machining allowance in z- bz2 double Machining allowance in z+

In the example below a box finished workpiece called “Example workpiece” is created with dimensions 1600x800x18 and associated to a raw with overmaterials

```
left (x-) and right (x+) equal to 10mm, front (y-) 6mm, back (y+) 2 mm, bottom (z-)
```

and top (0+) equal to 0 mm.

```
CreateFinishedWorkpieceBox("Example workpiece",1106 ,81 ,68);
CreateRawWorkpiece("Example workpiece",10 ,10 ,6 ,2 ,0 ,0);
```

3.4.1.2.2 Extrusion WorkPiece

Defined with the instruction CreateRawWorkpieceFromExtrusion. The instruction creates a workpiece from extrusion along axis Z starting from a closed geometry defined on the XY plane. The reference geometry for the workpiece “boundary” is the active geometry defined in the script before the call at the instruction CreateFinishedWorkpieceFromExtrusion. The instruction let also to create workpiece holes, simply passing geometry names as input parameter. Hole geometries must be created previously to call the instruction.

Name Type Description Name string Workpiece name Dz double Workpiece thickness Bx double (opt def. 0) Distance in x of the raw geometry from the origin of the finished workpiece By double (opt def. 0) Distance in y of the raw geometry from the origin of the finished workpiece Bz double (opt def. 0) Distance in z of the raw geometry from the origin of the finished workpiece internalProfiles string[] (opt) List of internal profiles geometries names CreateRawWorkpieceFromExtrusion Parameters

See the example below for the definition of a finished and raw of an arch workpiece

```
// Create finished workpiece
CreatePolyline("",70,600);
AddArc2PointRadiusToPolyline(1130,600,530,true,false);
AddSegmentToPolyline(1200,600);
AddArc2PointRadiusToPolyline(0,600,600,false,false);
```

```
AddSegmentToPolyline(70,600);
CreateFinishedWorkpieceFromExtrusion("Telaio_Up",63);
// Define raw workpiece
CreatePolyline("",74,593);
AddSegmentToPolyline(339,1052);
AddSegmentToPolyline(861,1052);
AddSegmentToPolyline(1126,593);
AddSegmentToPolyline(1310.7,593);
AddSegmentToPolyline(953.3,1211);
AddSegmentToPolyline(246.6,1211);
AddSegmentToPolyline(-110.7,593);
AddSegmentToPolyline(74,593);
CreateRawWorkpieceFromExtrusion("Telaio_Up",63);
```

3.4.2 WorkPiece positioning

Defined with instruction `SetWorkpieceSetupPosition`. Instruction is used to place the current workpiece in the machine table in the current workplan.

Name Type Description x double X position of the workpiece. y double Y position of the workpiece z double Z position of the workpiece zRot double Rotation with respect to the Z axis of the workpiece. `SetWorkpieceSetupPosition` parameters

Current workpiece position could be extracted by the instruction `GetWorkPieceSetupPosition`.

3.4.2.1 Workpieces position on canonical areas

Workpieces position on canonical areas (see appendix 9.1 Canonical working fields) is always done referring to the workpiece corner closest to working area origin. Workpiece positioning is made by the instruction described in the paragraph 3.4.2 WorkPiece positioning.

3.4.2.1.1 Lower left working field

Workpiece position is defined by its lower left corner with respect to working field origin.

The example below positions 3 workpieces (200, 100), (300,200), (400, 200) at

```
positions (200,100), (600,100), (1000,100) respect to working field origin.
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece1",200,200,18);
SetWorkpieceSetupPosition(200,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece2",300,200,18);
SetWorkpieceSetupPosition(600,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece3",400,200,18);
SetWorkpieceSetupPosition(1000,100,0,0);
```

3.4.2.1.2 Upper left working field

Workpiece position is defined by its upper left corner with respect to working field origin.

The example below positions 3 workpieces (200, 100), (300,200), (400, 200) at

```
positions (200,100), (600,100), (1000,100) respect to working field origin.
SetMachiningParameters("E", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece1",200,200,18);
SetWorkpieceSetupPosition(200,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece2",300,200,18);
SetWorkpieceSetupPosition(600,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece3",400,200,18);
SetWorkpieceSetupPosition(1000,100,0,0);
```

3.4.2.1.3 Lower right working field

Workpiece position is defined by its lower right corner with respect to working field origin.

The example below positions 3 workpieces (200, 100), (300,200), (400, 200) at

```
positions (200,100), (600,100), (1000,100) respect to working field origin.
SetMachiningParameters("D", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece1",200,200,18);
SetWorkpieceSetupPosition(200,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece2",300,200,18);
SetWorkpieceSetupPosition(600,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece3",400,200,18);
SetWorkpieceSetupPosition(1000,100,0,0);
```

3.4.2.1.4 Upper right working field

Workpiece position is defined by its upper right corner with respect to working field origin.

The example below positions 3 workpieces (200, 100), (300,200), (400, 200) at

```
positions (200,100), (600,100), (1000,100) respect to working field origin.
SetMachiningParameters("H", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece1",200,200,18);
SetWorkpieceSetupPosition(200,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece2",300,200,18);
SetWorkpieceSetupPosition(600,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece3",400,200,18);
SetWorkpieceSetupPosition(1000,100,0,0);
```

3.4.2.2 Workpieces position on non canonical working fields

Workpieces position on non canonical areas (see appendix 9.2 Non canonical working fields) could be done in standard mode or in Layout mode. Workpiece positioning is made by the instruction described in the paragraph 3.4.2 WorkPiece positioning.

3.4.2.2.1 Workpieces position in standard mode

Workpieces position on non canonical working fields in standard mode is made referring to the origin working field of the closest corner of the rectangle that contains all workpieces. Workpiece position depends on all other workpieces positions and must be remind that the reference point of the workpiece and the reference point of the rectangle that contains all workpieces change with reference to working field origin.

Referring to the below image

where the brown rectangles identify workpieces, blu rectangle identifies the workpieces container and the black rectangle identifies the working field, it is defined

BoundaryLength : Length of workpieces boundary (along X). BoundaryWidth : Width of workpieces boundary (along Y). BoundaryXOrigin : X position of boundary corner referred to working field origin. BoundaryYOrigin : Y position of boundary corner referred to working field origin. WorkpieceLength : Length of workpiece to position (along X). WorkpieceWidth : Width of workpiece to position (along Y). WorkpieceXPosition : X position desired value of workpiece. WorkpieceYPosition : Y position desired value of workpiece.

WorkpieceXPositionInstruction : X position value of the workpiece to put in the positioning instruction.

WorkpieceYPositionInstruction : Y position value of the workpiece to put in the positioning instruction.

3.4.2.2.1.1 Lower left working field

Workpieces positioning is made with reference to the lower left corner of the rectangle that contains all workpieces.

X and Y position value to insert in the position instruction have to be calculated by this formulas:

WorkpieceXPositionInstruction = BoundaryLength - WorkpieceXPosition

-WorkpieceLength + BoundaryXOrigin

WorkpieceYPositionInstruction = BoundaryWidth - WorkpieceYPosition

-WorkpieceWidth + BoundaryYOrigin

3.4.2.2.1.2 Upper left working field

Workpieces positioning is made with reference to the upper left corner of the rectangle that contains all workpieces.

X and Y position value to insert in the position instruction have to be calculated by this formulas:

WorkpieceXPositionInstruction = BoundaryLength - WorkpieceXPosition

-WorkpieceLength + BoundaryXOrigin

WorkpieceYPositionInstruction = BoundaryWidth - WorkpieceYPosition

-WorkpieceWidth + BoundaryYOrigin

3.4.2.2.1.3 Lower right working field

Workpieces positioning is made with reference to the lower right corner of the rectangle that contains all workpieces.

X and Y position value to insert in the position instruction have to be calculated by this formulas:

WorkpieceXPositionInstruction = BoundaryLength - WorkpieceXPosition

-WorkpieceLength + BoundaryXOrigin

WorkpieceYPositionInstruction = BoundaryWidth - WorkpieceYPosition

-WorkpieceWidth + BoundaryYOrigin

3.4.2.2.1.4 Upper right working field

Workpieces positioning is made with reference to the upper right corner of the rectangle that contains all workpieces.

X and Y position value to insert in the position instruction have to be calculated by this formulas:

WorkpieceXPositionInstruction = BoundaryLength - WorkpieceXPosition

-WorkpieceLength + BoundaryXOrigin

WorkpieceYPositionInstruction = BoundaryWidth - WorkpieceYPosition

-WorkpieceWidth + BoundaryYOrigin

3.4.2.2.2 Workpieces position in Layout mode

Workpieces position on non canonical working fields in Layout mode could be done by putting on positioning instruction the desired workpiece position values like in Layout. It is applied in the same way in all kinds of working fields.

Defined with the instruction ConfigWorkpiecePositioningMode. Input value equal to 0 means standard position, input value equal to 1 means X and Y positions are referred to the working field origin.

Name Type Description mode int Workpiece setup reference mode 0: standard 1: user coordinates referred to area origin

ConfigWorkpiecePositioningMode parameters

In the example below workpieces are positioned at (200,100), (600, 100), (1000, 100) referred to working field origin.

```
SetMachiningParameters("H", 1, 0, 0, false);
//Layout mode
ConfigWorkpiecePositioningMode(1);
CreateFinishedWorkpieceBox("WorkPiece1",200,200,18);
SetWorkpieceSetupPosition(200,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece2",300,200,18);
SetWorkpieceSetupPosition(600,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece3",400,200,18);
SetWorkpieceSetupPosition(1000,100,0,0);
```

3.4.3 WorkPiece selection

Defined with instruction SelectWorkpiece. The instruction lets to select the specified workpiece as active workpiece. The workpiece can be selected with numeric index

Name Type Description index int WorkPiece index to select SelectWorkpiece parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1106 ,81 ,68);
SelectWorkpiece(0);
```

or by name

Name Type Description name string WorkPiece name to select SelectWorkpiece parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1106 ,81 ,68);
SelectWorkpiece("WorkPiece");
```

3.4.4 WorkPiece deletion

Defined with instruction DeleteWorkpiece. The instruction deletes the workpiece from the project.

WorkPiece can be deleted by an index that represents the position in the workpiece list. The first sequence position is 0.

Name Type Description index int Index of workpiece to delete.

DeleteWorkpiece parameters

In the example below the workpiece in position 1 ("Piece2") will be deleted.

```
CreateFinishedWorkpieceBox("Piece1", 1600, 1200, 18);
CreateFinishedWorkpieceBox("Piece2", 1600, 1200, 18);
DeleteWorkpiece(1);
```

WorkPiece can be deleted by name

Name Type Description name string Name of workpiece to delete.

DeleteWorkpiece parameters

In the example below the workpiece named "Piece2" will be deleted.

```
CreateFinishedWorkpieceBox("Piece1", 1600, 1200, 18);
CreateFinishedWorkpieceBox("Piece2", 1600, 1200, 18);
DeleteWorkpiece("Piece2");
```

3.4.5 WorkPiece X position detection

Defined with the instruction SetWorkpieceNeedXPositionDetection. The instruction specifies that the active workpiece needs X position detection on the workplane.

Name Type Description needXPositionD bool Specifies that the active workpiece needs X position etection detection on the workplane.

Parametri SetWorkpieceNeedXPositionDetection

The example below enables the X position detection for workpiece 2.

```
SetMachiningParameters("H", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece1",200,200,18);
SetWorkpieceSetupPosition(200,100,0,0);
CreateFinishedWorkpieceBox("WorkPiece2",300,200,18);
SetWorkpieceSetupPosition(600,100,0,0);
SetWorkpieceNeedXPositionDetection(true);
CreateFinishedWorkpieceBox("WorkPiece3",400,200,18);
SetWorkpieceSetupPosition(1000,100,0,0);
```

3.5 WorkPlan

A WorkPlan defines the sequence of the executable to execute on the machine. When a new project is generated a default workplan, called "Setup", is created automatically. This is also the case when executing a scripting project

3.5.1 WorkPlan creation

Defined with instruction CreateWorkplan. The instruction creates a Workplan in the project.

Name Type Description name string Name of Workplan

CreateWorkplan parameters

The example below creates a workplan named "WorkPlan" and activates it

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);
CreateWorkplan("WorkPlan");
```

3.5.2 WorkPlan selection

Defined with instruction SelectWorkplan. The instruction selects the specified workplan as active workpiece.

The workplan can be selected with numeric index

Name Type Description index int WorkPlan index to select SelectWorkplan parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1106 ,81 ,68);  
CreateWorkplan("WorkPlan1");  
CreateWorkplan("WorkPlan2");  
SelectWorkplan(0);
```

or by name

Name Type Description name string WorkPlan name to select SelectWorkplan parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1106 ,81 ,68);  
CreateWorkplan("WorkPlan1");  
CreateWorkplan("WorkPlan2");  
SelectWorkplan("WorkPlan1");
```

3.5.3 WorkPlan deletion

Defined with instruction DeleteWorkplan. The instruction deletes the workplan from the project.

WorkPlan can be deleted by an index that represents the position in the workplan list. The first sequence position is 0.

Name Type Description index int Index of workplan to delete.

DeleteWorkplan parameters

In the example below the workplan in position 1 ("WorkPlan2") will be deleted.

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);  
CreateWorkplan("WorkPlan1");  
CreateWorkplan("WorkPlan2");  
DeleteWorkplan(1);
```

WorkPlan can be deleted by name

Name Type Description name string Name of workpiece to delete.

DeleteWorkplan parameters

In the example below the workplan named "WorkPlan2" will be deleted.

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);  
CreateWorkplan("WorkPlan1");  
CreateWorkplan("WorkPlan2");  
DeleteWorkplan("WorkPlan2");
```

3.5.4 WorkPlane rename

Defined with instruction RenameWorkplan. The instruction renames the active workplan.

Nome Tipo Descrizione name string Name to use to rename the workplan.

Parametri RenameWorkplan

In the example below workplan named "WorkPlan1" is renamed as "WorkPlanNew1"

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);  
CreateWorkplan("WorkPlan1");  
RenameWorkplan("WorkPlanNew1");
```

3.6 Planes

Plane is associated to a finished workpiece and let to manage 2d objects creation. Planes are of 2 types, standar and user. Standard plane are always created by workpiece creation, the instruction to create a workpiece automatically creates the box planes. User planes can be created and associated to a workpiece.

3.6.1 Standard planes

Generating a finished workpiece requires the automatic creation of standard workplanes. In particular, for a parallelepipedal shaped workpiece six planes will be automatically generated : - "Top" = upper side - "Bottom" = bottom side - "Right" = right side - "Left" = left side - "Front" = front side - "Back" = back side)

In case of extrusion workpiece, standard planes are relative to the box that contains the extrusion.

3.6.2 User plane

User planes are defined by a placement and by Z and X absolute rotation.

3.6.2.1 Parallel planes

Defined with instruction CreateWorkplane. Creates a plane parallel to the active plane. Plane is created as a new plane equal to active plane translating to Z axis direction.

Name Type Description name String Workplane name offsetZ Double Plane translation value along Z CreateWorkplane parameters

In the example below a plane called “Workplane” is created, parallel to the upper side and a plane “Workplane2” parallel to the right side:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreateWorkplane("Workplane", -10);
SelectWorkplane("Right");
CreateWorkplane("Workplane2", -800);
```

3.6.2.2 Slanted plane

Slanted planes are user planes created by placement and rotations.

Defined with instruction CreateWorkplane. This instruction has more than one prototype to create planes.

3.6.2.2.1 Plane by three points

Used to create a plane starting from the identification of three points on the plane.

Name Type Description Name String Workplane name p1X Double Coordinate X of point 1 p1Y Double Coordinate Y of point 1 p1Z Double Coordinate Z of point 1 p2X Double Coordinate X of point 2 p2Y Double Coordinate Y of point 2 P2Z Double Coordinate Z of point 2 p3X Double Coordinate X of point 3 p3Y Double Coordinate Y of point 3 P3Z Double Coordinate Z of point 3 CreateWorkplane parameters

In the example below a plane called “Workplane” is created, starting from points (30, 40, 0), (100, 200, 5), (1000, 40, 0):

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreateWorkplane("Workplane", 30, 40, 0, 100, 200, 5, 1000, 40, 0);
```

3.6.2.2.2 Plane created by placement and rotations

Used to create a plane by the origin of the plane and its rotation in relation to the axes X and Z. Plane is created positioning it on specified origin and rotate it first around Z axis and after X axis.

Name Type Description Name String Workplane name. X0 Double X coordinate of the origin. Y0 Double Y coordinate of the origin. Z0 Double Z coordinate of the origin. ZRotation Double Angle of rotation around Z axis. XRotation Double Angle of rotation around X axis. CreateWorkplane parameters

In the example below a plane called “Workplane ” is created, with origin in point (1000, 500, 0), rotation of 45° in relation to axis Z and 45° in relation to axis X:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreateWorkplane("Workplane", 1000, 500, 0, 45, 45);
```

3.6.2.2.3 Parametric plane created by placement and rotations

Used to create a parametric plane by the origin of the plane and its rotation in relation to the axes X and Z. Plane is created positioning it on specified origin and rotate it first around Z axis and after X axis.

Name Type Description Name String Workplane name. X0 Double X coordinate of the origin, could be an expression. Y0 Double Y coordinate of the origin, could be an expression. Z0 Double Z coordinate of the origin, could be an expression. ZRotation Double Angle of rotation around Z axis, could be an expression. XRotation Double Angle of rotation around X axis, could be an expression. CreateWorkplane parameters

In the example below a plane called “Parametric plane” is created, with origin in point (x, y, z), rotation of parameter “rotZ” in relation to axis Z and rotation of parameter “rotX” in relation to axis X:

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SetWorkpieceSetupPosition(0,0,0,0);
AddVariable("x", 10, 0, 1, "", false, true);
AddVariable("y", 10, 0, 1, "", false, true);
AddVariable("z", 10, 0, 1, "", false, true);
AddVariable("rotZ", 45, 0, 0, "", false, true);
AddVariable("rotX", 45, 0, 0, "", false, true);
CreateWorkplane("Parametric plane", "x", "y", "z", "rotZ", "rotX");
```

See parametric chapter to create parameters.

3.6.3 Plane selection

Defined with instruction SelectWorkplane.

To specify the plane that will be used for the geometries and the machining operations, the active plane must be specified. It is used to select a plane as “active plane”. To do this the name of the workplane being activated is required. The workplane can be a plane created by the user with one of the previously described instructions or one of the standard machining sides (top, bottom, right, left, front, back). If the user does not activate a plane, the top plane is selected by default as the active workplane.

Plane can be selected by name

Name Type Description Name String Workplane name “Top” = upper side; “Bottom” = bottom side “Right” = right side; “Left” = left side “Front” = front side; “Back” = back side SelectWorkplane parameters

In the example below the left plane of the workpiece is selected as active plane, on which the geometry of a rhombus is performed, then the bottom plane is selected on which a cross is performed:

```
CreateFinishedWorkpieceBox("Example workpiece",1800 800 40);
SelectWorkplane("Left");
CreatePolyline("Rhombus", 300, 20);
AddSegmentToPolyline( 400, 40);
AddSegmentToPolyline( 500, 20);
```

```
AddSegmentToPolyline( 400, 0);
ClosePolyline("Rhombus");
SelectWorkplane("Bottom");
CreateSegment("Segment1", 900, 0, 900, 800);
CreateSegment("Segment2", 0, 400, 1800, 400);
```

or by index

Name Type Description index int Index of the workplane to activate.

SelectWorkplane parameters

Where index is:

0- Top plane 1- Right plane 2- Left plane

3- Front plane 4- Back plane 5- Bottom plane

In the example below the left plane of the workpiece is selected as active plane, on which the geometry of a rhombus is performed, then the bottom plane is selected on which a cross is performed:

```
CreateFinishedWorkpieceBox("Pezzo esempio", 1800, 800, 40);
SelectWorkplane("Left");
CreatePolyline("Rombo", 300, 20);
AddSegmentToPolyline( 400, 40);
AddSegmentToPolyline( 500, 20);
AddSegmentToPolyline( 400, 0);
ClosePolyline("Rombo");
SelectWorkplane("Bottom");
CreateSegment("Segmento1", 900, 0, 900, 800);
CreateSegment("Segmento2", 0, 400, 1800, 400);
```

3.6.4 Plane deletion

Defined with instruction DeleteWorkplane. The instruction deletes the plane from the project.

Plane can be deleted by an index that represents the position in the plane list. The first sequence position is 0.

Name Type Description index int Index of plane to delete.

DeleteWorkplane parameters

In the example below the workpiece in position 6 ("User plane") will be deleted. Position value is equal to 6 because for each workpiece will always be added the 6 standard planes.

```
CreateFinishedWorkpieceBox("Piece", 1600, 1200, 18);
CreateWorkplane("User plane", 10);
DeleteWorkplane(6);
```

Plane can be deleted by name

Name Type Description name string Name of plane to delete.

DeleteWorkplane parameters

In the example below the plane named "User plane" will be deleted.

```
CreateFinishedWorkpieceBox("Piece", 1600, 1200, 18);
CreateWorkplane("Piano utente", 10);
DeleteWorkplane("Piano utente");
```

3.7 2D Geometries

Remember that, as set out in chapter 2, the creation of a 2D geometry is always associated to the active workplane (current plane) and that, as it is associated to a plane, the geometry created is always expressed in local coordinates to the plane. In particular the active plane is always associated to a local reference system, consisting of a right handed Cartesian triad, integral with the plane and with origin in the left bottom end of the plane (axis X along the horizontal component of the plane, axis Y along the vertical component, axis Z exit from the plane). The geometries that the user can create are therefore always dependent on the Cartesian coordinates X and Y, related to the local reference system associated to the current plane. It is important to note that after being created the geometry becomes the "active geometry", that is, it will be the geometry on which the subsequent machining instructions will be applied.

3.7.1 Creating geometries

See below for the instructions available to create geometries available with MSL.

3.7.1.1 Point

Defined with instruction CreatPoint. The instruction is used to create a cartesian point.

Name Type Description Name string Geometry name x double Point X coordinate y double Point Y coordinate

CreatPoint parameters

In the example below, a cartesian point called "Point" will be created at coordinates (10,10). Firstly the operation workpiece is defined. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
CreatPoint("Point", 10, 10);
```

3.7.1.2 Parametric point

Defined with instruction CreatePoint. The instruction is used to create a parametric cartesian point.

Name	Type	Description
Name string	Geometry name	x object Expression of point X coordinate
y object	Expression of point Y coordinate	

CreatePoint parameters

In the example below, a parametric cartesian point called "Point" will be created at

coordinates ("X","Y"). Firstly the operation workpiece is defined. Furthermore as the

plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);  
AddVariable("X", 10, 0, 1, "", false, true);  
AddVariable("Y", 10, 0, 1, "", false, true);  
CreatPoint("Point", "X", "Y");
```

3.7.1.3 Segment

3.7.1.3.1 Segment by start point and end point

Defined with instruction CreateSegment . The instruction creates a segment from start point to the end point.

Name	Type	Description
name string	Geometry name	startX double Coordinate X of start point
startY double	Coordinate Y of start point	
endX double	Coordinate X of final point	
endY double	Coordinate Y of final point	CreateSegment parameters

In the example below a segment called "Example segment" is created with a start

point (30,70) and final point (650,500). Firstly the operation workpiece is defined.

Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece",1800 800 18);  
CreateSegment("Segment", 30, 70, 650, 500);
```

3.7.1.3.2 Parametric segment by start point and end point

Defined with instruction CreateSegment . The instruction creates a parametric segment from expressions of start point and the end point.

Name	Type	Description
name string	Geometry name	startX object Expression of coordinate X of start point
startY object	Expression of coordinate Y of start point	
endX object	Expression of coordinate X of final point	
endY object	Expression of coordinate Y of final point	CreateSegment parameters

In the example below a segment called "Parametric segment" is created with a start

point (Xi,Yi) and final point (Xf,Yf). Firstly the operation workpiece is defined.

Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece",1800 800 18);  
AddVariable("Xi", 10, 0, 1, "", false, true);  
AddVariable("Yi", 10, 0, 1, "", false, true);  
AddVariable("Xf", 100, 0, 1, "", false, true);  
AddVariable("Yf", 100, 0, 1, "", false, true);  
CreateSegment("Parametric segment", 30, 70, 650, 500);
```

3.7.1.4 Arc

3.7.1.4.1 Arc by 3 points

Defined with instruction CreateArc3Points. The instruction creates an arc by 3 points.

Name	Type	Description
name string	Geometry name	p1X double Coordinate X of point 1
p1Y double	Coordinate Y of point 1	
p2X double	Coordinate X of point 2	
p2Y double	Coordinate Y of point 2	
p3X double	Coordinate X of point 3	
p3Y double	Coordinate Y of point 3	CreateArc3Points parameters

In the example below an arc called "Arc" is created by instruction CreateArc3Points with first point P1(0,0), second point P2(100,100) and third point P3 (200, 0).Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Piece", 1800, 800, 18);  
CreateArc3Points("Arc", 0, 0, 100, 100, 200, 0);
```

3.7.1.4.2 Parametric arc by 3 points

Defined with instruction CreateArc3Points. The instruction creates a parametric arc by 3 points.

Name	Type	Description
name string	Geometry name	p1X object Expression of coordinate X of point 1
p1Y object	Expression of coordinate Y of point 1	
p2X object	Expression of coordinate X of point 2	
p2Y object	Expression of coordinate Y of point 2	
p3X object	Expression of coordinate X of point 3	
p3Y object	Expression of coordinate Y of point 3	CreateArc3Points parameters

In the example below an arc called "Parametric arc" is created by instruction CreateArc3Points with first point P1(X1,Y1), second point P2(X2,Y2) and third point P3 (X3, Y3).Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Piece", 1800, 800, 18);  
AddVariable("X1", 0, 0, 1, "", false, true);
```

```
AddVariable("Y1", 0, 0, 1, "", false, true);
AddVariable("X2", 50, 0, 1, "", false, true);
AddVariable("Y2", 100, 0, 1, "", false, true);
AddVariable("X3", 100, 0, 1, "", false, true);
AddVariable("Y3", 0, 0, 1, "", false, true);
CreateArc3Points("Parametric arc", "X1", "Y1", "X2", "Y2", "X3", "Y3");
```

3.7.1.4.3 Arc by start point end point and radius

Defined with instruction CreateArc2PointRadius. The instruction creates an arc from start point, end point and radius.

Name Type Description name string Geometry name startX double Coordinate X of start point

startY double Coordinate Y of start point endX double Coordinate X of final point endY double Coordinate Y of final point
radius double Arc radius isClockwise bool Clockwise direction isOver180 bool Arc angle greater than 180°

CreateArc2PointRadius parameters

In the example below an arc called “Arc” is created by instruction CreateArc2PointRadius with start point Pi(0,0), end point Pf(200, 0) and radius 100. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Piece", 1800, 800, 18);
CreateArc2PointRadius("Arc", 0, 0, 200, 0, 100, true, false);
```

3.7.1.4.4 Parametric arc by start point end point and radius

Defined with instruction CreateArc2PointRadius. The instruction creates a parametric arc from start point, end point and radius.

Name Type Description name string Geometry name startX object Expression of coordinate X of start point startY object
Expression of coordinate Y of start point endX object Expression of coordinate X of final point endY object Expression of
coordinate Y of final point radius object Expression of radius isClockwise bool Clockwise direction isOver180 bool Arc angle
greater than 180° CreateArc2PointRadius parameters

In the example below an arc called “Parametric arc” is created by instruction CreateArc2PointRadius with start point Pi(Xi,Yi), end point Pf(Xf, Yf) and radius R. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("Xi", 0, 0, 1, "", false, true);
AddVariable("Yi", 0, 0, 1, "", false, true);
AddVariable("Xf", 100, 0, 1, "", false, true);
AddVariable("Yf", 0, 0, 1, "", false, true);
AddVariable("R", 100, 0, 1, "", false, true);
CreateArc2PointRadius("Parametric arc", "Xi", "Yi", "Xf", "Yf", "R", true, false);
```

3.7.1.4.5 Arc by start point end point and center point

Defined with instruction CreateArc2PointCenter. The instruction creates an arc from start point, end point and center point.

Name Type Description name string Geometry name startX double Coordinate X of start point startY double Coordinate Y of
start point endX double Coordinate X of final point endY double Coordinate Y of final point centerX double Coordinate X of
centre point centerY double Coordinate Y of centre point isClockwise bool Clockwise direction CreateArc2PointCenter
parameters

In the example below an arc called “Arc” is created by instruction CreateArc2PointCenter with first point Pi(0,0), end point Pf(200, 0) and center point Pc(100, 0). Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Pezzo esempio", 1800, 800, 18);
CreateArc2PointCenter("Arco", 0, 0, 200, 0, 100, 100, 0, true);
```

Parametric arc by start point end point and center point

Defined with instruction CreateArc2PointCenter. The instruction creates a parametric arc from start point, end point and center point.

Name Type Description name string Geometry name startX object Expression of coordinate X of start point startY object
Expression of coordinate Y of start point endX object Expression of coordinate X of final point endY object Expression of
coordinate Y of final point centerX object Expression of coordinate X of centre point centerY object Expression of coordinate
Y of centre point isClockwise bool Clockwise direction CreateArc2PointCenter parameters

In the example below an arc called “Parametric arc” is created by instruction CreateArc2PointCenter with first point Pi(Xi,Yi), end point Pf(Xf, Yf) and center point Pc(Xc, Yc). Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("Xi", 0, 0, 1, "", false, true);
AddVariable("Yi", 0, 0, 1, "", false, true);
AddVariable("Xf", 100, 0, 1, "", false, true);
AddVariable("Yf", 0, 0, 1, "", false, true);
AddVariable("Xc", 50, 0, 1, "", false, true);
AddVariable("Yc", 0, 0, 1, "", false, true);
CreateArc2PointCenter("Parametric arc", "Xi", "Yi", "Xf", "Yf", "Xc", "Yc", true);
```

3.7.1.4.6 Arc by start point center point and angle

Defined with instruction CreateArcCenterAngle. The instruction creates an arc from start point, center point and arc angle.

Name Type Description name string Geometry name startX double Coordinate X of start point startY double Coordinate Y of start point centerX double Coordinate X of centre point centerY double Coordinate Y of centre point angle double Arc angle CreateArcCenterAngle parameters

In the example below an arc called "Arc" is created by instruction CreateArcCenterAngle with first point Pi(1000,500), center point Pc(1500,200) and angle 45°. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Pezzo esempio", 1800, 800, 18);
CreateArcCenterAngle ("Arc esempio", 1000, 500, 1500, 200, 45);
```

3.7.1.5 Circle

3.7.1.5.1 Circle by center point and radius

Defined with instruction CreateCircleCenterRadius . The instruction creates an anti clockwise circle from center point and radius.

Name Type Description name string Geometry name centerX double Coordinate X of centre point centerY double Coordinate Y of centre point radius double Circle radius CreateCircleCenterRadius parameters

In the example below a circle called "Circle" is created by instruction CreateCircleCenterRadius with center point Pc(1000,400), radius 300 and rotation anti clockwise. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 40);
CreateCircleCenterRadius ("Circle", 1000, 400, 300);
```

The instruction creates a circle from center point and radius and rotation direction.

Name Type Description name string Geometry name centerX double Coordinate X of centre point centerY double Coordinate Y of centre point radius double Circle radius isClockwise bool Clockwise direction CreateCircleCenterRadius parameters (with rotation direction)

In the example below a circle called "Circle" is created by instruction CreateCircleCenterRadius with center point Pc(1000,400), radius 300 and rotation clockwise. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 40);
CreateCircleCenterRadius ("Circle", 1000, 400, 300, true);
```

3.7.1.5.2 Parametric circle by center and radius

Defined with instruction CreateCircleCenterRadius . The instruction creates an anti clockwise parametric circle from center point and radius.

Name Type Description name string Geometry name centerX object Expression of coordinate X of centre point centerY object Expression of coordinate Y of centre point radius object Expression of circle radius CreateCircleCenterRadius parameters

In the example below a parametric circle called "Parametric circle" is created by instruction CreateCircleCenterRadius with center point Pc(Xc,Yc), radius R and rotation anti clockwise. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 40);
AddVariable("Xc", 50, 0, 1, "", false, true);
AddVariable("Yc", 0, 0, 1, "", false, true);
AddVariable("R", 10, 0, 1, "", false, true);
CreateCircleCenterRadius ("Parametric circle", "Xc", "Yc", "R");
```

The instruction creates a parametric circle from center point and radius and rotation direction.

Name Type Description name string Geometry name centerX object Expression of coordinate X of centre point centerY object Expression of coordinate Y of centre point radius object Expression of circle radius isClockwise bool Clockwise direction CreateCircleCenterRadius parameters (with rotation direction)

In the example below a parametric circle called "Parametric circle" is created by instruction CreateCircleCenterRadius with center point Pc(Xc,Yc), radius R and rotation clockwise. Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Pezzo", 1800, 800, 40);
AddVariable("Xc", 50, 0, 1, "", false, true);
AddVariable("Yc", 0, 0, 1, "", false, true);
AddVariable("R", 10, 0, 1, "", false, true);
CreateCircleCenterRadius ("Parametric circle", "Xc", "Yc", "R", true);
```

3.7.1.5.3 Circle by center point and a point on circumference

Defined with instruction CreateCircleCenterPoint. The instruction creates an anti clockwise circle from the center point and a point on the circumference.

Name Type Description name string Geometry name centerX double Coordinate X of centre point centerY double Coordinate Y of centre point pointX double Coordinate X of the circumference point pointY double Coordinate Y of the circumference point CreateCircleCenterPoint parameters

In the example below an anti clockwise circle called "Cerchio esempio" is created by instruction CreateCircleCenterPoint with center point Pc(1000,400), point on circumference Pcf(900, 400). Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Pezzo esempio", 1800, 800, 40);
CreateCircleCenterPoint("Cerchio esempio", 1000, 400, 900, 400);
```

The instruction creates a circle from the center point, a point on the circumference and rotation direction.

Name Type Description name string Geometry name centerX double Coordinate X of centre point centerY double Coordinate Y of centre point pointX double Coordinate X of the circumference point pointY double Coordinate Y of the circumference point isClockwise bool Clockwise direction CreateCircleCenterPoint parameters (with rotation direction)

In the example below a clockwise circle called "Cerchio esempio" is created by instruction CreateCircleCenterPoint with center point Pc(1000,400), point on circumference Pcf(900, 400). Furthermore as the plane for the geometry has not been specified, the top side plane is considered as active by default.

```
CreateFinishedWorkpieceBox("Pezzo esempio", 1800, 800, 40);
CreateCircleCenterPoint("Cerchio esempio", 1000, 400, 900, 400, true);
```

3.7.1.6 Ellipse

3.7.1.6.1 Ellipse by center and axes

Defined with instruction CreateEllipseCenterAxes . The instruction creates an ellipse from center point, major radius, minor radius and rotation angle in relation to X axis.

Name Type Description name string Geometry name centerX double X coordinate of the center point. centerY double Y coordinate of the center point. majorRadius double Value of the major radius. minorRadius double Value of the minor radius. angle double The angle of the major axis and the X axis of the active plane. CreateEllipseCenterAxes parameters

In the example below an ellipse called "Example ellipse" is created with a centre

```
point (1000,500), major semi-axis 450, minor semi-axis 200, angle 30°.
CreateFinishedWorkpieceBox("Example workpiece",1800 800 18);
CreateEllipseCenterAxes ("Example ellipse", 1000, 500, 450, 200, 30);
```

3.7.1.6.2 Parametric ellipse by center and axes

Defined with instruction CreateEllipseCenterAxes . The instruction creates an ellipse from center point, major radius, minor radius and rotation angle in relation to X axis.

Name Type Description name string Geometry name centerX object Expression of coordinate X of the center point. centerY object Expression of coordinate Y of the center point. majorRadius object Expression of the major radius. minorRadius object Expression of the minor radius. angle object Expression of the angle of the major axis and the X axis of the active plane. CreateEllipseCenterAxes parameters

In the example below an ellipse called "Example ellipse" is created with a centre

```
point (Xc,Yc), major semi-axis MR, minor semi-axis mr, angle A.
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("Xc", 50, 0, 1, "", false, true);
AddVariable("Yc", 0, 0, 1, "", false, true);
AddVariable("MR", 450, 0, 1, "", false, true);
AddVariable("mr", 200, 0, 1, "", false, true);
AddVariable("A", 30, 0, 0, "", false, true);
CreateEllipseCenterAxes ("Parametric ellipse", "Xc", "Yc", "MR", "mr", "A");
```

3.7.1.7 Polyline

Polyline is a complex geometric element composed of an ordered sequence of elements of type segment and arc of circle. Polyline creation needs the definition of an instruction sequence to create it, and to add geometric elements (segment and arc) of an ordered sequence where to respect the rule that the last point of the current element must be equal to the first point of the next element. In details, polyline creation needs a first call to an instruction to define start point (CreatePolyline), a collection of instructions to add elements and one instruction (ClosePolyline) to automatically close the polyline. If the instruction (ClosePolyline) is not used, polyline can be closed manually otherwise it is open.

3.7.1.7.1 Polyline creation

Defined with instruction CreatePolyline.

The instruction creates the polyline with start point from which next elements will be added

Name Type Description name string Geometry name. startX double X coordinate of the first point of the polyline. startY double Y coordinate of the first point of the polyline. CreatePolyline parameters

In the example below a polyline named "Polyline" is created with a start point (200, 550). This polyline is not consistent, because it defines only the start point to add segments and arcs.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
CreatePolyline("Polyline", 200, 550);
```

3.7.1.7.2 Parametric polyline creation

Defined with instruction CreatePolyline. The instruction creates the parametric polyline with start point from which next elements will be added

Name Type Description name string Geometry name. startX object Expression of X coordinate of the first point of the polyline. startY object Expression of Y coordinate of the first point of the polyline. CreatePolyline parameters

In the example below a polyline named "Parametric polyline" is created with a start

```
point (Xi, Yi). This polyline is not consistent, because it defines only the start point to
```

add segments and arcs.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("Xi", 50, 0, 1, "", false, true);
AddVariable("Yi", 0, 0, 1, "", false, true);
CreatePolyline("Parametric polyline", "Xi", "Xf");
```

3.7.1.7.3 Add to polyline a segment

Defined with instruction AddSegmentToPolyline. The instruction adds a segment to the current polyline, with start point equal to current point and end point equal to specified point.

Nome Tipo Descrizione endX double X coordinate of the last point endY double Y coordinate of the last point nameElement string Name of added element AddSegmentToPolyline parameters

In the example below a polyline named "Polyline" is created with a segment with a start point (200, 500) and end point (300, 600).

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
CreatePolyline("Polyline", 200, 500);
AddSegmentToPolyline(300, 600);
```

3.7.1.7.4 Add to polyline a parametric segment

Defined with instruction AddSegmentToPolyline. The instruction adds a parametric segment to the current polyline, with start point equal to current point and end point equal to specified point.

Nome Tipo Descrizione endX object Expression of X coordinate of the last point endY object Expression of Y coordinate of the last point nameElement string Name of added element AddSegmentToPolyline parameters

In the example below a polyline named "Parametric polyline" is created with a segment with a start point (Xi, Yi) and end point (Xf, Yf).

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("Xi", 0, 0, 1, "", false, true);
AddVariable("Yi", 0, 0, 1, "", false, true);
AddVariable("Xf", 100, 0, 1, "", false, true);
AddVariable("Yf", 0, 0, 1, "", false, true);
CreatePolyline("Parametric polyline", "Xi", "Xi");
AddSegmentToPolyline("Xf", "Xf");
```

3.7.1.7.5 Add to polyline an arc by 3 points

Defined with instruction AddArc3PointsToPolyline. The instruction adds to the polyline an arc by 3 points. First point is the current point, second point and third point are specified in the instruction.

Nome Tipo Descrizione p2X double X coordinate of point 2 p2Y double Y coordinate of point 2

p3X double X coordinate of point 3 p3Y double Y coordinate of point 3 nameElement string Name of added element AddArc3PointsToPolyline parameters

In the example below a polyline named "Polyline" is created with an arc with a start

```
point (200, 500), second point (300, 600) and end point (400, 500).
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
CreatePolyline("Polyline", 200, 500);
AddArc3PointsToPolyline(300, 600, 400, 500);
```

3.7.1.7.6 Add to polyline a parametric arc by 3 points

Defined with instruction AddArc3PointsToPolyline. The instruction adds to the polyline a parametric arc by 3 points. First point is the current point, second point and third point are specified in the instruction.

Nome Tipo Descrizione p2X object Expression of coordinate X of point 2 p2Y object Expression of coordinate Y of point 2 p3X object Expression of coordinate X of point 3 p3Y object Expression of coordinate Y of point 3 nameElement string Name of added element AddArc3PointsToPolyline parameters

In the example below a polyline named "Parametric polyline" is created with a parametric arc with a start point (X1, Y1), second point (X2, Y2) and end point (X3, Y3).

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("X1", 0, 0, 1, "", false, true);
AddVariable("Y1", 0, 0, 1, "", false, true);
AddVariable("X2", 100, 0, 1, "", false, true);
AddVariable("Y2", 0, 0, 1, "", false, true);
AddVariable("X3", 50, 0, 1, "", false, true);
AddVariable("Y3", 100, 0, 1, "", false, true);
CreatePolyline("Parametric polyline", "X1", "Y1");
AddArc3PointsToPolyline("X2", "Y2", "X3", "Y3");
```

3.7.1.7.7 Add to polyline an arc by 2 points and center

Defined with instruction AddArc2PointCenterToPolyline. The instruction adds to the polyline an arc by start point, center point and end point and rotation direction.

Nome Tipo Descrizione endX double X coordinate of final point endY double Y coordinate of final point centerX double X coordinate of centre point centerY double Y coordinate of centre point isClockwise bool true clockwise direction, false anti clockwise nameElement string Name of added element AddArc2PointCenterToPolyline parameters

In the example below a polyline named "Polilinea" is created with an arc with a start

```
point (200, 500), center point (300, 500) and end point (400, 500) and rotation
```

clockwise.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);  
CreatePolyline("Polyline", 200, 500);  
AddArc2PointCenterToPolyline(400, 500, 300, 500, true);
```

3.7.1.7.8 Add to polyline a parametric arc by 2 points and center

Defined with instruction AddArc2PointCenterToPolyline. The instruction adds to the polyline a parametric arc by start point, center point and end point and rotation direction.

Nome Tipo Descrizione endX object Expression of coordinate X of final point endY object Expression of coordinate Y of final point centerX object Expression of coordinate X of centre point centerY object Expression of coordinate Y of centre point isClockwise bool true clockwise direction, false anti clockwise nameElement string Name of added element AddArc2PointCenterToPolyline parameters

In the example below a polyline named "Parametric polyline" is created with an arc with a start point (Xi, Yi), center point (Xc, Yc) and end point (Xf, Yf) and rotation clockwise.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);  
AddVariable("Xi", 0, 0, 1, "", false, true);  
AddVariable("Yi", 0, 0, 1, "", false, true);  
AddVariable("Xc", 50, 0, 1, "", false, true);  
AddVariable("Yc", 0, 0, 1, "", false, true);  
AddVariable("Xf", 100, 0, 1, "", false, true);  
AddVariable("Yf", 0, 0, 1, "", false, true);  
CreatePolyline("Parametric polyline", "Xi", "Yi");  
AddArc2PointCenterToPolyline("Xf", "Yf", "Xc", "Yc", true);
```

3.7.1.7.9 Add to polyline an arc by 2 points and radius

Defined with instruction AddArc2PointRadiusToPolyline. The instruction adds to the polyline an arc by start point, end point, radius and rotation direction.

Nome Tipo Descrizione endX double X coordinate of final point endY double Y coordinate of final point radius double Arc radius isClockwise bool true clockwise direction, false anti clockwise isOver180 bool Arc angle greater than 180° nameElement string Name of added element AddArc2PointRadiusToPolyline parameters

In the example below a polyline named "Polyline" is created with an arc with a start

```
point (200, 500), end point (400, 500) radius 100 and rotation clockwise.  
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);  
CreatePolyline("Polyline", 200, 500);  
AddArc2PointRadiusToPolyline(400, 500, 100, true, true);
```

3.7.1.7.10 Add to polyline a parametric arc by 2 points and radius

Defined with instruction AddArc2PointRadiusToPolyline. The instruction adds to the polyline a parametric arc by start point, end point, radius and rotation direction.

Nome Tipo Descrizione endX object Expression of the X coordinate of final point endY object Expression of the Y of final point radius object Expression of the arc radius isClockwise bool true clockwise direction, false anti clockwise isOver180 bool Arc angle greater than 180° nameElement string Name of added element AddArc2PointRadiusToPolyline parameters

In the example below a polyline named "Parametric polyline" is created with an arc with a start point (Xi, Yi), end point (Xf, Yf) radius R and rotation clockwise.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);  
AddVariable("Xi", 0, 0, 1, "", false, true);  
AddVariable("Yi", 0, 0, 1, "", false, true);  
AddVariable("Xf", 100, 0, 1, "", false, true);  
AddVariable("Yf", 0, 0, 1, "", false, true);  
AddVariable("R", 100, 0, 1, "", false, true);  
CreatePolyline("Parametric polyline", "Xi", "Yi");  
AddArc2PointRadiusToPolyline("Xf", "Yf", "R", true, true);
```

3.7.1.7.11 Add to polyline an arc by center and angle

Defined with instruction AddArcCenterAngleToPolyline. The instruction adds to the polyline an arc by start point, center point and angle.

Nome Tipo Descrizione centerX double Coordinate X of centre point centerY double Coordinate Y of centre point angle double Arc angle nameElement string Name of added element AddArcCenterAngleToPolyline parameters

In the example below a polyline named "Polilinea" is created with an arc with a start

```
point (200, 500), center point (300, 500) and angle 45 degree.  
CreateFinishedWorkpieceBox("Pezzo", 1800, 800, 18);  
CreatePolyline("Polilinea", 200, 500);  
AddArcCenterAngleToPolyline(300, 500, 45);
```

3.7.1.7.12 Polyline closing

Defined with instruction ClosePolyline. The instruction closes the current polyline connecting polyline current point with polyline first point.

The example below creates a polyline rectangle. The polyline is created with 3 rectangle sides and instruction creates the last rectangle side connecting the end point of the third element with the first point of the polyline.

```
CreateFinishedWorkpieceBox("Pezzo", 1800, 800, 18);
CreatePolyline("Polilinea", 0, 0);
AddSegmentToPolyline(100, 0);
AddSegmentToPolyline(100, 100);
AddSegmentToPolyline(0, 100);
ClosePolyline();
```

the same closed polyline can be created manually by adding the last segment connecting the end point of the third element with the first point of the polyline.

```
CreateFinishedWorkpieceBox("Pezzo", 1800, 800, 18);
CreatePolyline("Polilinea", 0, 0);
AddSegmentToPolyline(100, 0);
AddSegmentToPolyline(100, 100);
AddSegmentToPolyline(0, 100);
AddSegmentToPolyline(0, 0);
```

In case of parametric polyline the instruction adds automatically the parametric of the last polyline element.

```
CreateFinishedWorkpieceBox("Workpiece", 1800, 800, 18);
AddVariable("X1", 0, 0, 1, "", false, true);
AddVariable("Y1", 0, 0, 1, "", false, true);
AddVariable("X2", 100, 0, 1, "", false, true);
AddVariable("Y2", 0, 0, 1, "", false, true);
AddVariable("X3", 100, 0, 1, "", false, true);
AddVariable("Y3", 100, 0, 1, "", false, true);
AddVariable("X4", 0, 0, 1, "", false, true);
AddVariable("Y4", 100, 0, 1, "", false, true);
CreatePolyline("Parametric polyline", "X1", "Y1");
AddSegmentToPolyline("X2", "Y2");
AddSegmentToPolyline("X3", "Y3");
AddSegmentToPolyline("X4", "Y4");
ClosePolyline();
```

3.7.1.7.13 Fillet to polyline elements

Defined with instruction `AddFilletToPolyline`. The instruction creates fillet between previous element and current element, inserting a circle of specified radius.

Nome Tipo Descrizione radius double Fillet radius option int Creation options: option = 0 => Direct fillet. option = 1 => Inverse fillet. option = 2 => Clockwise fillet. option = 3 => Counterclockwise fillet.

Parametri `AddFilletToPolyline`

The example below creates a polyline named "Polilinea", with 2 elements connected with an arc of radius 10.

```
CreateFinishedWorkpieceBox("Pezzo", 1800, 800, 18);
CreatePolyline("Polilinea", 0, 0);
AddSegmentToPolyline(100, 0);
AddFilletToPolyline(10);
AddSegmentToPolyline(100, 100);
```

3.7.1.7.14 Chamfer to polyline elements

Defined with instruction `AddChamferToPolyline`. The instruction creates a chamfer between previous element and current element, inserting a segment.

Nome Tipo Descrizione length1 double Length on first segment length2 double Length on second segment option int options : 0 direct

Parametri `AddChamferToPolyline`

The example below creates a polyline named "Polilinea", with 2 elements connected with a segment with length on first segment 10 and length on second segment 20.

```
CreateFinishedWorkpieceBox("Pezzo", 1800, 800, 18);
CreatePolyline("Polilinea", 0, 0);
AddSegmentToPolyline(100, 0);
AddChamferToPolyline(10, 20);
AddSegmentToPolyline(100, 100);
```

3.7.2 Plane reference

Defined with instruction `SetReferencePosition`. The instruction creates a geometry referred to a plane corner (bottom left, top left, bottom right, top right). This means geometric quotes are relative to plane corner and not to the standard system reference equal to the bottom left plane corner. Valid only for standard planes and not valid for user planes.

Name Type Description referencePosit int reference position: 0=Bottom Left, 1=Top Left, 2=Bottom ion Right, 3=Top Right.

`SetReferencePosition` parameters

In the example below will be created 4 cartesian points each one referred to the top plane corner. All points have the same coordinates (10,10). In this way created points will be at :

```
BottomLeft (10,10)
TopLeft (10,1190)
BottomRight (1590,10)
TopRight (1590,1190)
CreateFinishedWorkpieceBox("Workpiece", 1600, 1200, 18);
SetReferencePosition(0);
CreatPoint("BottomLeft", 10, 10);
SetReferencePosition(0);
CreatPoint("TopLeft", 10, 10);
```

```
SetReferencePosition(1);
CreatPoint("BottomRight", 10, 10);
SetReferencePosition(2);
CreatPoint("TopRight", 10, 10);
```

3.7.3 Geometry deletion

Defined with instruction DeleteGeometry. The instruction deletes the geometry from the project.

Geometry can be deleted by an index that represents the position in the geometries add sequence.

Nome Tipo Descrizione index int Index of geometry to delete.

DeleteGeometry parameters

In the example below cartesian point with index 1, corresponding to "Point2" will be deleted

```
CreateFinishedWorkpieceBox("Workpiece", 1600, 1200, 18);
CreatPoint("Point1", 10, 10);
CreatPoint("Point2", 10, 10);
DeleteGeometry(1);
```

La geometria può essere eliminata a partire dal nome

Nome Tipo Descrizione name string Nome della geometria da eliminare.

DeleteGeometry parameters

In the example below cartesian point named "Point2" will be deleted

```
CreateFinishedWorkpieceBox("Workpiece", 1600, 1200, 18);
CreatPoint("Point1", 10, 10);
CreatPoint("Point2", 10, 10);
DeleteGeometry("Point2");
```

3.8 Machining operations

As a rule for every type of operation a series of parameters can be defined to describe the mode used for the operation. For example the following can be defined:

type of tool, tool machining approach and retraction, infeed speed, tool rotation and feed, type of head and other parameters specific to the machining operation selected. Machining operation definition is made by a set of mandatory parameters that have to be always specified and a set of optional parameters that could be not specified. Optional parameters can be not specified and in this case default values will be used. Optional parameters that can be not specified are always all parameters following the last specified parameter. The example below shows an instruction with two mandatory parameters and three optional parameters

```
Instruction(MandatoryParameter1, MandatoryParameter2, OptionalParameter1=10,
```

OptionalParameter2=20, OptionalParameter3=30)

can be used specifying just mandatory parameters

```
Instruction(10,10)
```

In this case unspecified optional parameters will use default values, in the example 10, 20, 30.

The example below shows the case where the optional parameter "OptionalParameter2" will be passed with value 50

```
Instruction(10,10,10,50)
```

In this case, in addition to mandatory parameters, it is necessary to specify all optional parameters before the parameter to specify, in the example OptionalParameter1, while optional parameter OptionalParameter3 could be not specified because is after the last specified parameter.

The example below shows the instruction with all optional parameters specified by their default values.

```
Lavorazione(10,10,10,20,30)
```

About parameters that can be passed to instructions, be careful to use the tool name parameter, it must be the same name used to define it in the ToolManager tools file definition. If a tool is created with the name "Fresa D10 L20", this will be the name to use as a tool parameter in machining creation.

Machinings could be created in a way the toolpath generator generates the toolpath, or it is possible to define an explicit toolpath to execute on the machine where the toolpath generator does not generate anything.

See the Maestro manual for the specific meaning of the operations listed above. See below for a description of the most common operations.

3.8.1 Machining types

3.8.1.1 Drilling

Defined with instruction CreateDrill. The instruction creates a drill in X,Y position and with specified diameter and depth. Other parameters are optional.

Name Type Description name string Operation name x double Coordinate X of the hole y double Coordinate Y of the hole depth double Hole depth diameter double Hole diameter description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess.Drilling) tool string (opt) Tool name ("-1" for automatic selection) head string (opt) Work head number ("-1" for automatic selection) dischargeSteps int (opt def. 0) Discharge steps number rotSpeed double (opt def -1) Tool rotation speed boringSpeed double (opt def -1) Slow down speed kindOfHole string (opt

def -1) Type of hole (tool:P=flat, L=lance, S=taper. The default value is P) taperHeight double (opt def 0) Taper height securityQuote double (opt def Safety quota null) holeTaperDiameter double (opt def 0) Hole diameter of taper or bliz (valid only with taper or bliz). CreateDrill parameters

In the example below a drilling machining is created with mandatory parameters X,Y, depth and diameter

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
SelectWorkplane("Top");
CreateDrill("Drill",10,20,15,8, "",TypeOfProcess.Drilling);
```

In the example below a drilling machining is created with mandatory parameters X,Y, depth and diameter and optional parameter rotSpeed.

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
SelectWorkplane("Top");
CreateDrill("Drill",10,20,15,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

3.8.1.2 Parametric Drilling

Defined with instruction CreateDrill. The instruction creates a parametric drill in X,Y position and with specified diameter and depth. Other parameters are optional.

Name Type Description name string Hole name x object Coordinate X of the hole y object Coordinate Y of the hole depth object Hole depth diameter object Hole diameter description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess.Drilling) tool string (opt) Tool name ("-1" for automatic selection) head string (opt) Work head number ("-1" for automatic selection) dischargeSteps int (opt def. 0) Discharge steps number rotSpeed object (opt def Tool rotation speed null) boringSpeed object (opt def Slow down speed null) kindOfHole object (opt def -1) Type of hole (tool:P=flat, L=lance, S=taper. The default value is P) taperHeight object (opt def Taper height null) securityQuote object (opt def Safety quota null) holeTaperDiameter object (opt def Hole diameter of taper or bliz (valid only null) with taper or bliz). CreateDrill parameters

In the example below a parametric drilling machining is created with mandatory parameters X,Y, depth and diameter

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
AddVariable("X", 100, 0,1);
AddVariable("Y", 100, 0,1);
AddVariable("Depth", 15, 0,1);
AddVariable("Diameter", 8, 0,1);
SelectWorkplane("Top");
CreateDrill("Drill","X","Y","Depth","Diameter", "",TypeOfProcess.Drilling);
```

3.8.1.3 Through drilling

Defined with instruction CreateThroughDrill. This instruction creates a through drill in X,Y position, with specified diameter and depth automatically calculated respect to plane.

Nome Tipo Descrizione Name string Hole Name x double Coordinate X of the hole

y double Coordinate Y of the hole diameter double Hole diameter overcutLength double Hole overcut length description string Descrizione della lavorazione typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess.Drilling) tool string (opt) Tool name ("-1" for automatic selection) head string (opt) Work head number ("-1" for automatic selection) dischargeSteps int (opt def. 0) Discharge steps number rotSpeed double (opt def -1) Tool rotation speed boringSpeed double (opt def -1) Slow down speed kindOfHole string (opt def -1) Type of hole (tool:P=flat, L=lance, S=taper. The default value is P) taperHeight double (opt def 0) Taper height securityQuote double (opt def Safety quota null) holeTaperDiameter double (opt def 0) Hole diameter of taper or bliz (valid only with taper or bliz). CreateThroughDrill parameters

In the example below a through drill will is created with mandatory parameters X,Y, diameter and overcut length.

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
SelectWorkplane("Top");
CreateThroughDrill("Hole",10,20,8,14, "",TypeOfProcess.Drilling);
```

In the example below a through drill is created with mandatory parameters X,Y, diameter and overcut length and optional parameter rotSpeed.

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
SelectWorkplane("Top");
CreateThroughDrill("Hole",10,20,8,14, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

3.8.1.4 Parametric through drilling

Defined with instruction CreateThroughDrill. This instruction creates a parametric through drill in X,Y position, with specified diameter and depth automatically calculated respect to plane.

Nome Tipo Descrizione Name string Hole Name x object Coordinate X of the hole y object Coordinate Y of the hole diameter object Hole diameter overcutLength object Hole overcut length description string Descrizione della lavorazione typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess.Drilling) tool string (opt) Tool name ("-1" for automatic selection)

head string (opt) Work head number ("-1" for automatic selection) dischargeSteps int (opt def. 0) Discharge steps number rotSpeed object (opt def -1) Tool rotation speed boringSpeed object (opt def -1) Slow down speed kindOfHole object (opt def -1) Type of hole (tool:P=flat, L=lance, S=taper. The default value is P) taperHeight object (opt def 0) Taper height securityQuote object (opt def Safety quota null) holeTaperDiameter object (opt def 0) Hole diameter of taper or bliz (valid only with taper or bliz). CreateThroughDrill parameters

In the example below a through drill will is created with mandatory parameters X,Y, diameter and overcut length defined by parameters.

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
AddVariable("X", 100, 0,1);
AddVariable("Y", 100, 0,1);
AddVariable("Diameter", 8, 0,1);
AddVariable("OvercutLength", 14, 0,1);
SelectWorkplane("Top");
CreateThroughDrill("Hole",10,20,8,14, "",TypeOfProcess.Drilling);
```

In the example below a through drill is created with mandatory parameters X,Y, diameter and overcut length and optional parameter rotSpeed.

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
SelectWorkplane("Top");
CreateThroughDrill("Hole","X","Y","Diameter","OvercutLength", "",TypeOfProcess.Drilling);
```

3.8.1.5 Slanted drilling

Defined with instruction CreateSlantedDrill. This instruction creates a slanted drill, a drill not defined on a plane, but defined with X,Y,Z respect to absolute system reference and with a rotation around Z absolute

axis (Vector) and a rotation around X absolute axis (Tilting).

Name Type Description name string Operation name x double Coordinate X of the hole y double Coordinate Y of the hole z double Coordinate Z of the hole angleA double Angle (in degrees) of the tool on the horizontal plane, in relation to axis x angleB double Angle (in degrees) of the tool on the horizontal plane, in relation to axis Z depth double Hole depth diameter double Hole diameter

description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess.Drilling) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) dischargeSteps int (opt def. 0) Discharge steps number rotSpeed double (opt def -1) Tool rotation speed boringSpeed double (opt def -1) Slow down speed kindOfHole string (opt def -1) Type of hole (tool:P=flat, L=lance, S=taper. The default value is P) taperHeight double (opt def 0) Taper height securityQuote double (opt 0) Safety quote holeTaperDiameter object (opt def 0) Hole diameter of taper or bliz (valid only with taper or bliz). CreateSlantedDrill parameters

The example below creates a slanted drilling located in (10,20,18) with 60 degree Vector and 45 degree Tilting.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
CreateSlantedDrill("Hole",10,20,18,45,60, "",TypeOfProcess.Drilling);
```

3.8.1.6 Routing

Defined with instruction CreateRoughFinish. The instruction creates a 2D routing on active geometry associated with a plane. Routing can be created only with milling tools, different tools are not allowed like saw tools. The tool name is a mandatory parameter. Geometry could be whatever of 2D geometry type, open or closed.

Nome Tipo Descrizione Name string Name of machining Depth double Depth of the operation. description string Description of the operation (optional parameter). typeOfProcess TypeOfProcess Type of operation, if not set a generic operation will be created (optional parameter). Tool String (opt) Tool, if null or -1, tool will be selected

automatically (optional parameter).

Head string (opt) Head to be used, if null or -1 head will be selected automatically (optional parameter). correction int (opt def 0) Tool correction. C = 0, Tool center (default) C = 1, left correction C = 2, right correction C = 3, in depth correction C = 13, left correction + in depth

correction C = 23, right correction + in

depth correction inputSpeed double (opt def -1) Entry speed into piece. (V in XGO Xilog), if not set the value programmed in the tooling will be used (optional parameter).

rotSpeed double (opt def -1) Rotation speed of tool (S in XGO Xilog), if not set the value programmed in the tooling will be used (optional parameter). Speed double (opt def -1) Milling speed, if not set the value programmed in the tooling will be used (optional parameter). overMaterial double (opt def 0) Overmaterial, if not set 0 will be used (optional parameter).

CreateRoughFinis parameters

The example below creates 2D routings width depth 10 and tool named "EndMill D20" on the same geometry associated with the top plane. Created routings are on center tool, left tool compensation and right tool compensation. With compensation is possible to define an over material.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Plane selection
SelectWorkplane("Top");
// Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
// Operation creation in center tool
CreateRoughFinish("Routing1",10,"", TypeOfProcess.GeneralRouting,"EndMill
```

D20","-1",0,-1,-1,3,0);

```
// Operation creation in left tool compensation
CreateRoughFinish("Routing2",10,"", TypeOfProcess.GeneralRouting,"EndMill
```

D20","-1",1,-1,-1,3,0);

```
// Operation creation in right tool compensation
CreateRoughFinish("Routing3",10,"", TypeOfProcess.GeneralRouting,"EndMill
```

D20","-1",2,-1,-1,3,0);

The example below creates 2D routings width depth 10 and tool named “EndMill D20” on different geometries associated with the top plane. Each routing has is own geometry. Created routings are on center tool, left tool compensation and right tool compensation. With compensation is possible to define an over material.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Plane selection
SelectWorkplane("Top");
// Geometry creation
CreatePolyline("Geometry1",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
// Operation creation in center tool
CreateRoughFinish("Routing1",10,"", TypeOfProcess.GeneralRouting,"EndMill
```

D20","-1",0,-1,-1,3,0);

```
// Geometry creation
CreatePolyline("Geometry2",0,0);
AddSegmentToPolyline(0,200);
AddSegmentToPolyline(200,200);
// Operation creation in left tool compensation
CreateRoughFinish("Routing2",10,"", TypeOfProcess.GeneralRouting,"EndMill
```

D20","-1",1,-1,-1,3,0);

```
// Geometry creation
CreatePolyline("Geometry3",0,0);
AddSegmentToPolyline(0,300);
AddSegmentToPolyline(300,300);
// Operation creation in right tool compensation
CreateRoughFinish("Routing3",10,"", TypeOfProcess.GeneralRouting,"EndMill
```

D20","-1",2,-1,-1,3,0);

3.8.1.7 Slanted routing

Defined with instruction CreateSlantedRoughFinish. The instruction creates a 2D slanted routing on active geometry associated with a plane that must be the top plane or a plane parallel to the top plane. Routing can be created only with milling tools, different tools are not allowed like saw tools. The tool name is a mandatory parameter. Geometry could be whatever of 2D geometry type, open or closed.

The instruction creates a center tool slanted routing.

Name Type Description name string Operation name angleA double Angle (in degrees) of routing rotation angleB double Angle (in degrees) between tool axis and vertical plane toolApproach int Tool approach to the geometry: 0 = tool parallel to axis X positive of the absolute reference system; 1 = tool perpendicular to the programmed

```
trajectory (tool right side of path);
```

2 = tool perpendicular to the programmed

```
trajectory (tool left side of path);
```

3 = tool parallel to the tangent to the programmed trajectory, in the path start

```
point (angle remains fixed during the
```

operation); 4 = tool parallel to the tangent to the programmed trajectory, in the current point (tool remains parallel to the path during the operation); depth double Operation depth description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection)

inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial (the default value is 0) CreateSlantedRoughFinish parameters

The example below creates 2D slanted routings width depth 20 and tool named “EndMill D20” on the same geometry associated with the top plane. Created routings have different tool approaches.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Plane selection
SelectWorkplane("Top");
// Geometry creation
CreateSegment("Segment1", 0, 0, 100, 0);
// Slanted routing creation with tool approach 0
CreateSlantedRoughFinish("SlantedRouting1", 60, 45, 0, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Slanted routing creation with tool approach 1
CreateSlantedRoughFinish("SlantedRouting2", 60, 45, 1, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Slanted routing creation with tool approach 2
CreateSlantedRoughFinish("SlantedRouting3", 60, 45, 2, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Slanted routing creation with tool approach 3
CreateSlantedRoughFinish("SlantedRouting4", 60, 45, 3, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Slanted routing creation with tool approach 4
CreateSlantedRoughFinish("SlantedRouting5", 60, 45, 4, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

The example below creates 2D slanted routings width depth 20 and tool named "EndMill D20" on different geometries associated with the top plane. Created routings have different tool approaches.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Plane selection
SelectWorkplane("Top");
// Geometry creation
CreateSegment("Segment1", 0, 0, 100, 0);
// Slanted routing creation with tool approach 0
CreateSlantedRoughFinish("SlantedRouting1", 60, 45, 1, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment2", 0, 50, 100, 50);
// Slanted routing creation with tool approach 1
CreateSlantedRoughFinish("SlantedRouting2", 60, 45, 2, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment3", 0, 100, 100, 100);
// Slanted routing creation with tool approach 2
CreateSlantedRoughFinish("SlantedRouting3", 60, 45, 3, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment4", 0, 100, 100, 100);
// Slanted routing creation with tool approach 3
CreateSlantedRoughFinish("SlantedRouting4", 60, 45, 4, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment5", 0, 100, 100, 100);
// Slanted routing creation with tool approach 4
CreateSlantedRoughFinish("SlantedRouting5", 60, 45, 5, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

The example below creates 2D slanted routings width depth 20 and tool named "EndMill D20" on the same geometry associated with a plane parallel to the top plane. Created routings have different tool approaches.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Top plane selection
SelectWorkplane("Top");
// Creation of a plane parallel to top plane
CreateWorkplane("Parallel plane", 20);
// Geometry creation
CreateSegment("Segment1", 0, 0, 100, 0);
// Slanted routing creation with tool approach 0
CreateSlantedRoughFinish("SlantedRouting1", 60, 45, 0, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment2", 0, 50, 100, 50);
// Slanted routing creation with tool approach 1
CreateSlantedRoughFinish("SlantedRouting2", 60, 45, 1, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment3", 0, 100, 100, 100);
// Slanted routing creation with tool approach 2
CreateSlantedRoughFinish("SlantedRouting3", 60, 45, 2, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment4", 0, 100, 100, 100);
// Slanted routing creation with tool approach 3
CreateSlantedRoughFinish("SlantedRouting4", 60, 45, 3, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

```
// Geometry creation
CreateSegment("Segment5", 0, 100, 100, 100);
// Slanted routing creation with tool approach 4
CreateSlantedRoughFinish("SlantedRouting5", 60, 45, 4, 20, "Slanted Routing",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", -1, -1, -1, 0);

The instruction creates a slanted routing with tool compensation. The Z axis rotation angle must be equal to 0 (angleA).

Name Type Description name string Operation name angleA double Angle (in degrees) of routing rotation angleB double Angle (in degrees) between tool axis and vertical plane toolApproach int Tool approach to the geometry: 0 = tool parallel to axis X positive of the absolute reference system; 1 = tool perpendicular to the programmed

trajectory (tool right side of path);

2 = tool perpendicular to the programmed

trajectory (tool left side of path);

3 = tool parallel to the tangent to the programmed trajectory, in the path start

```
point (angle remains fixed during the
```

operation); 4 = tool parallel to the tangent to the programmed trajectory, in the current point (tool remains parallel to the path during the operation); depth double Operation depth description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) correction double (opt def 0) Tool correction. C = 0, Tool center (default) C = 1, left correction C = 2, right correction C = 3, in depth correction C = 13, left correction + in depth

```
correction C = 23, right correction + in
```

depth correction inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial (the default value is 0)

CreateSlantedRoughFinish parameters

The example below creates 2D slanted routings width depth 20 and tool named "EndMill D20" on the same geometry associated with the top plane. Created routings have different tool compensations.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
// Plane selection  
SelectWorkplane("Top");  
// Geometry creation  
CreateSegment("Segmento", 0, 0, 100, 0);  
// Slanted routing creation in center tool  
CreateSlantedRoughFinish("SlantedRouting1", 60, 45, 0, 20, "Slanted Routing",
```

```
TypeOfProcess.GeneralRouting, "Fresa D20", "-1", 0, -1, -1, -1, 0);
```

```
// Slanted routing creation in left compensation  
CreateSlantedRoughFinish("SlantedRouting2", 60, 45, 1, 20, "Slanted Routing",
```

```
TypeOfProcess.GeneralRouting, "Fresa D20", "-1", 1, -1, -1, -1, 0);
```

```
// Slanted routing creation in right compensation  
CreateSlantedRoughFinish("SlantedRouting3", 60, 45, 2, 20, "SlantedRouting",
```

```
TypeOfProcess.GeneralRouting, "Fresa D20", "-1", 2, -1, -1, -1, 0);
```

3.8.1.8 Parametric slanted routing

Defined with instruction CreateSlantedRoughFinish. The instruction creates a parametric slanted routing on active geometry associated with a plane that must be the top plane or a plane parallel to the top plane. Routing can be created only with milling tools, different tools are not allowed like saw tools. The tool name is a mandatory parameter. Allowed parameters are angles and depth. Geometry could be whatever of 2D geometry type, open or closed.

Name Type Description name string Operation name angleA object Angle (in degrees) of routing rotation angleB object Angle (in degrees) between tool axis and vertical plane toolApproach int Tool approach to the geometry: 0 = tool parallel to axis X positive of the absolute reference system; 1 = tool perpendicular to the programmed

```
trajectory (tool right side of path);
```

2 = tool perpendicular to the programmed

```
trajectory (tool left side of path);
```

3 = tool parallel to the tangent to the programmed trajectory, in the path start

```
point (angle remains fixed during the
```

operation); 4 = tool parallel to the tangent to the programmed trajectory, in the current point (tool remains parallel to the path during the operation); depth object Operation depth description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) correction int (opt def 0) Tool correction. C = 0, Tool center (default) C = 1, left correction C = 2, right correction C = 3, in depth correction

C = 13, left correction + in depth

```
correction C = 23, right correction + in
```

depth correction inputSpeed object (opt def Input speed null) rotSpeed object (opt def Tool rotation speed null) speed object (opt def Feed speed null) overMaterial object (opt def Overmaterial null) isCnc bool (opt def true if Cnc compensation, false if Cad false) compensation(optional parameter). CreateSlantedRoughFinish parameters

The example below creates 2D slanted routings width depth 20 and tool named "EndMill D20" on the same geometry associated with the top plane. Created routings have angles and depth parametrized.

```
// WorkPiece creation  
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);  
// Parameter definition  
AddVariable("A", 100, 0,0);  
AddVariable("B", 100, 0,0);  
AddVariable("Depth", 20, 0,1);  
// Plane selection  
SelectWorkplane("Top");  
// Geometry creation  
CreateSegment("Segment", 0, 0, 100, 0);  
// Slanted routing creation with angles and depth parametrized  
CreateSlantedRoughFinish("SlantedRouting", "A", "B", 1, "Depth", "Slanted routing",
```

```
TypeOfProcess.GeneralRouting, "Fresa D20", "-1", 0, -1, -1, -1, 0, false);
```

3.8.1.9 Chamfer

Defined with instruction CreateChamfer. The instruction creates a chamfer on the top plane of workpiece. Chamfer is applied to a geometry that must be defined on the top plane. Geometry could be whatever of 2D geometry type, open or closed.

Name Type Description name string Operation name chamferWidth double Chamfer width chamferHeight double Chamfer height overcutLength double Overcut length toolPosition int Tool position in relation to the geometry: 0 = tool on left of geometry on the top plane ;

1 = tool on left of geometry on the bottom plane ; 2 = tool on right of geometry on the top plane ; 3 = tool on right of geometry on the bottom plane ; description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial (the default value is 0) CreateChamfer parameters

The example below creates a chamfer with height and width 10 and a tool named "EndMill D20" on a geometry associated with the top plane.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
CreateSegment("Segment", 0, 0, 0, 800);
CreateChamfer("Chamfer", 10, 10, 0, 0, "Chamfer", TypeOfProcess.GeneralRouting, "Fresa
```

D20", "-1", -1, -1, -1, 0);

3.8.1.10 Slot

Defined with instruction CreateSlot. The instruction creates a slot, a machining with tool of type mill or saw, on a linear geometry. Geometry could be defined in whatever plane. Machining could be defined with tool compensation, a different start and end depth, and could be inclined with respect to Z axis reference. Inclination is allowed only on standard planes and planes parallel to standard planes. Slot made by saw tool can be only horizontal or vertical.

Name Type Description name string Operation name depth double Slot depth description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) correction int (opt def 0) Tool correction (0=central, 1=left, 2=right. The default value is 0) inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial (the default value is 0) angle double (opt def 90) Operation angle in relation to the workplane. endDepth double (opt null) End depth width double (opt null) Width of operation, if not set the value programmed in the tooling will be used (optional parameter). CreateSlot parameters

The example below creates slots with tool named "Fresa D20" depth 10 on geometry defined on top plane with different tool compensation values

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Geometry creation
CreateSegment("Segment", 0, 0, 100, 0);
// Slot creation in center tool
CreateSlot("Slot1",10,"Slot", TypeOfProcess.GeneralRouting , "Fresa
```

D20", "-1", 0, -1, -1, -1, 0, 90, null, null);

```
// Slot creation in left compensation
CreateSlot("Slot2",10,"Slot", TypeOfProcess.GeneralRouting , "Fresa
```

D20", "-1", 1, -1, -1, -1, 0, 90, null, null);

```
// Slot creation in right compensation
CreateSlot("Slot3",10,"Slot", TypeOfProcess.GeneralRouting , "Fresa
```

D20", "-1", 2, -1, -1, -1, 0, 90, null, null);

The example below creates slot with tool named "Fresa D20" start depth 10 and end depth 20 on geometry defined on top plane in center tool

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Geometry creation
CreateSegment("Segment", 0, 0, 100, 0);
// Slot creation in center tool
CreateSlot("Slot",10,"Slot", TypeOfProcess.GeneralRouting , "Fresa
```

D20", "-1", 0, -1, -1, -1, 0, 90, 20, null);

3.8.1.11 Saw cut

Defined with instruction CreateBladeCut. The instruction creates a saw cut, a through machining made by a saw cut tool on a linear geometry. Geometry must be defined only on the top plane. Machining could be inclined, it is possible to define tool compensation, cut reference and material position. Material position defines workpiece uncut material. A plane is automatically generated with reference to material position. Cut reference defines cut position along Z absolute axis.

Name Type Description name string Operation name description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) angle double (opt def 90) Inclination angle respect to Z absolute axis correction int (opt def 0) Tool correction (0=central, 1=left, 2=right, 3=depth , 13=left/ depth , 23=right/depth. The default value is 0) inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial (the default value is 0)

cutPositionUpper bool (opt def true) Blade cut reference position (true = upper position reference) materialPositionLeft bool (opt def true) Workpiece position for subsequent operation (true = workpiece on the left of the cut) referenceOffset double (opt def 0) Distance from the blade cut reference position extraDepth double (opt def 0) Extra cut depth CreateBladeCut parameters

The example below creates saw cuts with saw tool named "Lama D300" with different parameter combinations

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Geometry creation
CreateSegment("Segment", 0, 0, 100, 0);
//Saw Cut machining in center tool orthogonal to top plane,
//top plane cut reference, left material
CreateBladeCut("SawCut1", "Saw Cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
```

90, 0, -1, -1, -1, 0, true, true, 0, 0);

```
//Saw Cut machining in left tool compensation orthogonal to top plane,
//top plane cut reference, left material
CreateBladeCut("SawCut2", "Saw Cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
```

90, 1, -1, -1, -1, 0, true, true, 0, 0);

```
//Saw Cut machining in right tool compensation orthogonal to top plane,
//top plane cut reference, left material
CreateBladeCut("SawCut3", "Saw Cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
```

90, 2, -1, -1, -1, 0, true, true, 0, 0);

```
//Saw Cut machining in center tool inclination 45 degree, top plane cut reference,
//left material
CreateBladeCut("SawCut4", "Saw Cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
```

45, 0, -1, -1, -1, 0, true, true, 0, 0);

```
//Saw Cut machining in left tool compensation inclination 45 degree, top plane cut
//reference, left material
CreateBladeCut("SawCut5", "Saw Cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
```

45, 1, -1, -1, -1, 0, true, true, 0, 0);

```
//Saw Cut machining in right tool compensation inclination 45 degree, top plane cut
//reference, left material
CreateBladeCut("SawCut6", "Saw Cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
```

45, 2, -1, -1, -1, 0, true, true, 0, 0);

3.8.1.12 Contour

Defined with instruction CreateContour. The instruction creates a contour, a machining with mill tool working on a side. Machining could be applied to a workpiece, in this case reference geometry is created from the workpiece profile on too plane, or can be applied to a geometry where it is possible to specify internal or external side to machine.

Name Type Description name string Operation name depth double Operation depth typeOfContour int Type of contour (0 = workpiece, 1 = selected geometry) sideOfContour int Contour side (0 = internal, 1 = external) description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. GeneralRouting) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) correction int (opt def 0) Tool correction (0=central, 1=left, 2=right. The default value is 0) inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial (the default value is 0) CreateContour parameters

The example below creates a contour on workpiece with a mill tool named "Fresa D20"

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
//Workpiece contour
CreateContour("Contour", 18, 0, 1, "Workpiece contour", TypeOfProcess.GeneralRouting,
```

"Fresa D20", "-1", 0, -1, -1, -1, 0);

The example below creates an external contour on the geometry used to create the workpiece extrusion with a mill tool named "Fresa D20"

```
CreatePolyline("WorkPiece profile",0,0);
AddSegmentToPolyline(1600,0);
AddSegmentToPolyline(1600,1200);
AddSegmentToPolyline(0,1200);
ClosePolyline();
CreateFinishedWorkpieceFromExtrusion("WorkPiece",18);
//Contour on geometry
CreateContour("Contour", 18, 1, 1, "Contour on geometry",
```

TypeOfProcess.GeneralRouting, "Fresa D20", "-1", 0, -1, -1, -1, 0);

3.8.1.13 Pocket

Defined with instruction CreateContourPocket. The instruction creates a pocket, a machining that removes all material inside a closed geometry. Machining can be defined only with mill tools. By default, pocket strategy is contour parallel. It is possible to specify bosses, closed geometry inside the outer geometry indicating part of material to not machine.

Name Type Description name string Operation name depth double Operation depth description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess. ConcentricPocket) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overlap double (opt def

50) Overlap percentage finalPass bool (opt false) Final pass application bossNames string[] Name of the geometries that define the bosses CreateContourPocket parameters

The example below creates a closed pocket on a circle with mill tool named "Fresa D20"

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
CreateCircleCenterRadius ("Cerchio", 25, 25, 10, false);
CreateContourPocket("Svuotatura", 10, "Svuotatura cerchio",
```

TypeOfProcess.ConcentricalPocket, "Fresa D20", "-1", -1, -1, -1, 50, false);

The example below creates a closed pocket on a circle containing a boss with mill tool named "Fresa D20"

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
CreateCircleCenterRadius ("Isola", 40, 40, 15, false);
CreateCircleCenterRadius ("Cerchio esterno", 40, 40, 30, false);
CreateContourPocket("Svuotatura", 10, "Svuotatura cerchio",
```

TypeOfProcess.ConcentricalPocket, "Fresa D20", "-1", -1, -1, -1, 50, false, "Isola");

3.8.1.14 Toekick pocket

Defined with instruction CreateToeKickPocket. The instruction creates a Toekick pocket, a machining that completely removes a workpiece corner or a workpiece open side. Machining can be applied only to closed geometries defined on workpiece top plane. Machining can be defined only with mill tools. By default, pocket strategy is Toekick.

Name Type Description Name string Operation name description string Operation description typeOfProcess TypeOfProcess Type of process (specify TypeOfProcess.ToekickPocket) tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection)

OvercutLength double (opt def 0) Overcut length. inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed Parametri CreateToeKickPocket

The example below creates a toekcik pocket on a rectangle on a workpiece corner with a tool named "Fresa D20"

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry
CreatePolyline("Geometry", 0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
ClosePolyline();
CreateToeKickPocket("ToekickNoStrategy", "", TypeOfProcess.ToekickPocket, "Fresa D20",
```

"-1");

3.8.1.15 Dowel

Defined with the instruction CreateDowelOperation. The instruction creates a doweling, a machining that allows inserting a dowel in a hole in the workpiece. No tool is necessary. Doweling can be used by herself if hole is already done, or in combination with dowel drilling operation.

Name Type Description Name string Name of doweling x double Coordinate X of the dowel y double Coordinate Y of the dowel dowelOuterLength double Dowel length out of workpiece description string Description of the operation (optional parameter). retractDistance double (opt def. 0) Retract distance from dowel outer tip, if not set 0 will be used (optional parameter). dowelReleaseSpeed double (opt def -1) Speed on dowel releasing, if not set rapid speed will be used (optional parameter). glueThickness double (opt def Glue Layer Thickness. 0.5) dowelDiameter double (opt null) Diameter of the dowel. dowelLength double (opt null) Length of the dowel. approachDistance double (opt null) Approach distance from work plane. head string (opt) Head to be used, if null or -1 head will be selected automatically (optional parameter). dowelInsertionDista double (opt null) Distance of dowel insertion, if not set nce default value will be used (optional parameter).

isDowelAlreadyInser bool (opt false) true if the dowel is already inserted, ted otherwise false.

dowelReleaseDistanc double (opt 0) Distance on dowel realesing added to e standard release distance (optional parameter).

CreateDowelOperation parameters

The example below creates a dowel with outer length 15.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Left");
CreateDowelOperation("Dowel", 100, 100, 15, "Dowel", 0, -1,0, 8, 30);
```

3.8.1.16 Parametric dowel

Defined with the instruction CreateDowelOperation. The instruction creates a parametric doweling, a machining that allows inserting a dowel in a hole in the workpiece where X and Y coordinates are parametrized. No tool is necessary. Doweling can be used by herself if hole is already done, or in combination with dowel drilling operation.

Name Type Description Name string Name of doweling x double Coordinate X of the dowel y double Coordinate Y of the dowel dowelOuterLength double Dowel length out of workpiece description string Description of the operation (optional parameter). retractDistance double (opt def. 0) Retract distance from dowel outer tip, if not set 0 will be used (optional parameter). dowelReleaseSpeed double (opt def -1) Speed on dowel releasing, if not set rapid speed will be used (optional parameter). glueThickness double (opt def Glue Layer Thickness. 0.5) dowelDiameter double (opt null) Diameter of the dowel. dowelLength double (opt null) Length of the dowel. approachDistance double (opt null) Approach distance from work

plane. head string (opt) Head to be used, if null or -1 head will be selected automatically (optional parameter). dowelInsertionDista double (opt null) Distance of dowel insertion, if not set nce default value will be used (optional parameter).

CreateDowelOperation parameters

The example below creates a parametric dowel with outer length 15.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
AddVariable("X", 100, 0,0);
AddVariable("Y", 100, 0,0);
SelectWorkplane("Left");
//Dowel with X and Y parametrized
CreateDowelOperation("Dowel", "X", "Y", 15, "Dowel", 0, -1,0, 8, 30);
```

3.8.1.17 Dowel drilling

Defined with the instruction CreateDowelDrillingOperation. The instruction creates the hole to insert dowel and must be used in association with the instruction to insert dowel.

Name Tipo Descrizione Name string Name of the doweling x double Coordinate X of the dowel. y double Coordinate Y of the dowel. dowelOuterLength double Dowel length out of workpiece. description string Description of the operation (optional parameter). tool string (opt) Tool, if null or -1, tool will be selected

automatically (optional parameter).

head string (opt) Head to be used, if null or -1 head will be selected automatically (optional parameter). dischargeSteps int (opt def. 0) Number of chip discharge steps. rotSpeed double (opt def -1) Rotation speed of tool (S in Xilog), if not set the value programmed in the tooling will be used (optional parameter). boringSpeed double (opt def -1) Boring speed. securityQuote double (opt null) Security quote. glueThickness double (opt 0.5) Glue Layer Thickness. dowelDiameter double (opt def Diameter of the dowel. null) dowelLength double (opt def Length of the dowel. null) CreateDowelDrillingOperation parameters

Drilling and doweling define a complex process, where there will be created an association between doweling that is master machining and drilling that is dependent machining. In this way, after process building, modifying dowel operation, drilling will be modified as a consequence A complex process is defined by an initialization process instruction InitializeComplexProcess and a create process instruction CreateComplexProcess. The example below creates a doweling process, made by drilling and doweling.

```
CreateFinishedWorkpieceBox("WorkPiece",1600, 1200, 18);
SelectWorkplane("Left");
//Complex process initialization
InitializeComplexProcess();
//Dowel drilling creation
CreateDowelDrillingOperation("Dowel drilling", 100, 100, 15, "Dowel drilling");
//Dowel operation
CreateDowelOperation("Dowel", 100, 100, 15, "Dowel", 0, -1,0, 8, 30);
//Create complex process
CreateComplexProcess();
```

3.8.1.18 3D Routing

Defined with instruction Create3DRoughFinish. The instruction creates a 3D routing made by a sequence of linear elements identifying linear movement and orientation of the tool along each of them. Each linear element is defined by an absolute position X,Y,Z and by rotation with respect to the Z axis and to the X axis of the absolute reference system. Machining has not associated a geometry.

This kind of instruction defines an explicit toolpath, the toolpath will not be generated by the toolpath generator and will be executed exactly as is defined.

The instruction creates the 3D machining lavorazione 3D

Nome Tipo Descrizione Name string Name of machining description string Description of the operation typeOfProcess TypeOfProcess Type of operation, if not set a generic operation will be created (use TypeOfProcess.GeneralRouting) (optional parameter). Tool String (opt) Tool, if null or -1, tool will be selected

automatically (optional parameter).

Head string (opt) Head to be used, if null or -1 head will be selected automatically (optional parameter). inputSpeed double (opt def -1) Entry speed into piece. (V in XGO Xilog), if not set the value programmed in the tooling will be used (optional parameter). rotSpeed double (opt def -1) Rotation speed of tool (S in XG0 Xilog), if not set the value programmed in the tooling will be used (optional parameter). speed double (opt def -1) Milling speed, if not set the value programmed in the tooling will be used (optional parameter). inputZRotation Double (opt null) Angle of rotation around the Z-axis (optional parameter). inputXRotation double (opt null) Angle of rotation around the X-axis (optional parameter).

Create3DRoughFinish parameters

the instruction CreateToolpath3D creates the toolpath start point

Nome Tipo Descrizione name string Name of the toolpath. startX double X coordinate of the first point of the toolpath. startY double Y coordinate of the first point of the toolpath.

startZ double Z coordinate of the first point of the toolpath.

CreateToolpath3D parameters

and the instruction AddSegmentToToolpath creates the toolpath element

Nome Tipo Descrizione endX double X coordinate of the last point of the toolpath. endY double Y coordinate of the last point of the toolpath. endZ double Z coordinate of the last point of the toolpath. ZRotation double (opt null) Rotation around Z

applied to the current segment. XRotation double (opt null) Rotation around X applied to the current segment.

Parametri AddSegmentToToolpath

The example below creates a 3D routing with a tool named "Fresa D20" with linear elements with rotation change in each element

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);
//3D Routing
Create3DRoughFinish("3D Routing", "3D Routing", TypeOfProcess.GeneralRouting, "Fresa D20",
"-1", -1, -1, -1, 90, 45);

//Creating first toolpath point
CreateToolpath3D("PercorsoUtensile3D", 90, 45, 20);
//Create toolpath elements
AddSegmentToToolpath(1600, 0, 18, 180, 45);
AddSegmentToToolpath(1600, 1200, 0, 270, 45);
AddSegmentToToolpath(0, 1200, 18, 0, 45);
AddSegmentToToolpath(0, 0, 20, 90, 45);
```

3.8.1.19 Pattern

3.8.1.19.1 Pattern creation

Defined with instruction CreatePattern. The instruction creates a rectangular pattern to the associated machining. Applicable to drilling and doweling. The instruction is active until it will be reset.

Nome Tipo Descrizione NumberOfRows int Number of rows. NumberOfColumns int Number of columns. RowDistance double Distance between rows. ColumnDistance double Distance between columns. RotationAngle Double Rotation 1 RowLayoutAngle double Rotation 2 (required for oblique patterns).

CreatePattern parameters

The example below creates a rectangular pattern of holes with 3 rows, 2 columns, rows distance 32, column distance 32

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreatePattern(3,2,32,32,0,90);
CreateDrill("Drill",10,20,15,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

The example below creates a rectangular pattern of dowels with 3 rows, 2 columns, rows distance 32, column distance 32

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Left");
CreatePattern(3,2,32,32,0,90);
CreateDowelOperation("Dowel", 100, 100, 15, "Dowel", 0, -1,0, 8, 30);
```

3.8.1.19.2 Pattern reset

Defined with instruction ResetPattern. The instruction reset the pattern, following machinings will be created single.

The example below creates a rectangular repetition drill followed by a single drill

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreatePattern(3,2,32,32,0,90);
CreateDrill("Drill1",10,20,15,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
ResetPattern();
CreateDrill("Drill2",10,20,15,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

3.8.1.20 Trimming

Defined with instruction CreateTrimming. The instruction creates a Trimming on active geometry

Name Type Description name string Name of the operation description string Description of the operation

trimmingType int Type of Trimming : 0 Single Feature 1 Joined to EdgeBanding tool String (opt) Tool name ("-1" for automatic selection) head String (opt) Work head number ("-1" for automatic selection) approachAngle int (opt def. 0) Approach tool inclination Angle to the normal to the path (degrees)

infeedAdvance double (opt def 0) Shifting of the start point of the trajectory: negative value reduce of that value the first segment of profile, positive

value (only per closed profile) extend of

that value the first segment of profile outfeedDelay double (opt def 0) Shifting of the end point of the trajectory: negative value reduce of that value the last segment of profile, positive value (only per closed profile) extend of that value the last segment of profile inputSpeed double (opt def -1) Input speed rotSpeed double (opt def -1) Tool rotation speed speed double (opt def -1) Feed speed overMaterial double (opt def 0) Overmaterial retractAngle double (opt def 0) Retract tool inclination Angle to the normal to the path (degrees) CreateTrimming parameters

The example below creates a trimming operation on active geometry.

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
//Geometry creation
CreateSegment("Segment", 0, 0, 100, 0);
//Creation of trimming independent to edge banging
CreateTrimming("Trimming", "Trimming", 0, "Refilatore", "-1", 0, 0, 0, -1, -1, -1, 0,
0);
```

3.8.2 Lead in and lead out

3.8.2.1 Lead in

Defined with instruction SetApprochStrategy. The instruction applies a lead in to a machining, it specifies the way the tool approach to the first working point. Instruction is modal, it is active until it is reset.

Nome Tipo Descrizione isLinear bool If true approach is linear, otherwise is an arc of circle. false = approccio ad arco di cerchio. isQuote bool If true approach is in quote from the start point, otherwise is in descent starting from the security Z. Distance double Distance of the approach segment/arc. Speed double (opt def null) Speed to be used for the approach segment/arc. Parametri SetApprochStrategy parameters

The example below creates a machining with different kind of lead in

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
// Plane selection
SelectWorkplane("Top");
// Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
//Linead down lead in
SetApprochStrategy(true, true, 1.2, );
//Fresatura
CreateRoughFinish("Routing1",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

```
//Linear quote lead in
SetApprochStrategy(true, false, 1.2);
//Fresatura
CreateRoughFinish("Routing2",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

```
//Arc down lead in
SetApprochStrategy(false, false, 1.2);
//Fresatura
CreateRoughFinish("Routing3",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

```
//Arc quote lead in
SetApprochStrategy(false, true, 1.2);
//Fresatura
CreateRoughFinish("Routing4",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

The instruction ResetApprochStrategy resets the active lead in.

3.8.2.2 Lead out

Defined with instruction SetRetractStrategy. The instruction applies a lead out to a machining, it specifies the way the tool retract from the last working point. Instruction is modal, it is active until it is reset.

Name Type Description isLinear bool Type of retraction: true = linear retraction; false = circle arc retraction. isQuote bool Type of retraction start: true = in quota, from the start point; false = in upstroke, from security height along Z.

distance double Radius multiplier for segment/retraction arc segment (distance = tool radius multiplier). overlapLength double Profile overlap length speed double (opt) Segment/ retraction arc speed SetRetractStrategy parameters

The example below creates a machining with different kind of lead out

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
// Plane selection
SelectWorkplane("Top");
// Creazione geometria
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
//Allontanamento lineare in discesa
SetApprochStrategy(true, true, 1.2, );
//Fresatura
CreateRoughFinish("Fresatura1",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

```
//Allontanamento lineare in quota
SetApprochStrategy(true, false, 1.2);
//Fresatura
CreateRoughFinish("Fresatura2",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

```
//Allontanamento ad arco in discesa
SetApprochStrategy(false, false, 1.2);
//Fresatura
CreateRoughFinish("Fresatura3",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20",-1",0,-1,-1,3,0);
```

```
//Allontanamento ad arco in quota
SetApprochStrategy(false, true, 1.2);
//Fresatura
```

```
CreateRoughFinish("Fresatura4",10,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","-1",0,-1,-1,3,0);
```

The instruction `ResetRetractStrategy` resets the active lead out.

3.8.3 Strategies

Machining strategy defines the way a machining is executed to reach the machining depth and are applied only to the first following machining.

3.8.3.1 Drilling strategies

Strategies applicable to drilling machinings.

3.8.3.1.1 Single step drilling strategy

Defined with instruction `CreateSingleStepDrillingStrategy`. The instruction applies to drilling a single step strategy, in one pass depth is reached.

3.8.3.1.2 Multi step drilling strategy

Defined with instruction `CreateMultiStepDrillingStrategy`. The instruction applies to drilling a multi step strategy. Depth will be reached in N steps. It is possible to specify the number of steps or the steps depth.

Nome Tipo Descrizione `isStepDepth` bool If true step depth parameter is used, else false number of steps parameter is used. `stepNumber` int (opt def 1) Number of steps. `stepDepth` double (opt def 0.0) Depth of each step.

`isSafetyQuoteReturn` bool (opt def false) true if each step return is at safety quote, false if each step return is inside piece. `CreateMultiStepDrillingStrategy` parameters

The example below creates a drilling executed in 3 steps with return inside piece

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
CreateMultiStepDrillingStrategy(true, 0, 3);  
CreateDrill("Drilling1",10,20,20,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

The example below creates a drilling executed with step depth 4 with return inside piece

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
CreateMultiStepDrillingStrategy(false, 4, 0);  
CreateDrill("Drill2",10,20,20,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

The example below creates a drilling executed in 3 steps with return at safety quote

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
CreateMultiStepDrillingStrategy(true, 0, 3, true);  
CreateDrill("Drilling1",10,20,20,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

The example below creates a drilling executed with step depth 4 with return at safety quote

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
CreateMultiStepDrillingStrategy(false, 4, 0, true);  
CreateDrill("Drill2",10,20,20,8, "",TypeOfProcess.Drilling, "-1", "-1", 0 , 15000);
```

3.8.3.2 Milling strategies

Strategies applicable to drilling machinings.

3.8.3.2.1 Unidirectional strategy

Defined with instruction `CreateUnidirectionalMillingStrategy`. The instruction applies a strategy to the machining where the final depth is reached with passes in the same direction, stroke connection at safety quote or inside the workpiece at the end of each pass. In case of lead in or lead out it is possible to apply it at each pass.

Nome Tipo Descrizione `allowMultiplePasses` bool If true, this is the standard roughing operation with multiple passes, i. e. several layers of material are removed sequentially, taking into account the maximum cutting depth. If false, this is the special roughing operation for pre-cast features with one pass `cuttingDepth` double Pass depth in the direction axial to the tool `finishCuttingDepth` double Final pass depth in the direction axial to the tool. If programmed is the pass that allows to reach the final depth programmed. `strokeConnectionType` int (opt def 1) Tool behavior when it moves between two passes. 0 = Tool remains into the workpiece and moves at the start point of the next pass (straghtline) 1 = Tool exits from the workpiece, moves in rapid and moves at the start point of the next pass (liftshiftplunge).

`isApproachRetractPas` bool If true for each depth step an sStrategy approach/retract will be applied, if false the only approach/retract applied is at starting and ending of machining. `CreateUnidirectionalMillingStrategy` parameters

The example below creates a routing with step depth 3, final depth 5, stroke connection at safety quote, lead in lead out only on first pass.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
//Plane selection  
SelectWorkplane("Top");  
//Geometry creation  
CreateSegment("Geometry",0,0, 100, 0);  
//Strategy creation  
CreateUnidirectionalMillingStrategy(true, 3, 5, 1, false);  
//Routing  
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The example below creates a routing with step depth 3, final depth 5, stroke connection at safety quote, lead in and lead out arc type at quote on each pass.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Lead in
SetApproachStrategy(false,true,1.5)
//Strategy creation
CreateUnidirectionalMillingStrategy(true, 3, 5, 1, true);
//Routing
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

3.8.3.2.2 Bidirectional strategy

Defined with instruction CreateBidirectionalMillingStrategy. The instruction applies a strategy to the machining where the final depth is reached with passes in the same direction, stroke connection at safety quote or inside the workpiece at the end of each pass. In case of lead in or lead out is possible to apply it at each pass.

Nome Tipo Descrizione allowMultiplePasses bool If true, this is the standard roughing operation with multiple passes, i. e. several layers of material are removed sequentially, taking into account the maximum cutting depth. If false, this is the special roughing operation for pre-cast features with one pass cuttingDepth double Pass depth in the direction axial to the tool finishCuttingDepth double Final pass depth in the direction axial to the tool. If programmed is the pass that allows to reach the final depth programmed.

CreateBidirectionalMillingStrategy parameters

The example below creates a routing with step depth 3, final depth 5.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Strategy creation
CreateBidirectionalMillingStrategy(true, 3, 5, 1, false);
//Routing
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

3.8.3.2.3 Helicoidal strategy

Defined with instruction CreateHelicMillingStrategy. The instruction applies a strategy to the machining where the final depth is reached with a helicoidal toolpath.

Nome Tipo Descrizione cuttingDepth double Pass depth in the direction axial to the tool. allowsFinishCutting bool True to execute the final pass, false otherwise finishCuttingDepth double Final pass depth in the direction axial to the tool. If programmed is the pass that allows to reach the final depth programmed. CreateHelicMillingStrategy parameters

The example below creates a routing with step depth 3, final depth 5.

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Strategy
CreateHelicMillingStrategy(3, 5, true);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

3.8.3.2.4 Zig zag strategy

Defined with instruction CreateZigZagMillingStrategy.

The instruction applies a strategy to the machining where the final depth is reached with a zig zag toolpath.

Nome Tipo Descrizione feedCuttingDepth double Depth of feed pass.

returnCuttingDepth double Depth of return pass.

finishCuttingDepth double Final pass depth in the direction axial to the tool. If programmed is the pass that allows to reach the final depth programmed.

CreateZigZagMillingStrategy parameters

The example below creates a routing with feed depth 3, return depth 4 and final depth 5.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Strategy
```

```
CreateZigZagMillingStrategy(3, 4, 5);
//Routing
CreateRoughFinish("Routing", 20, "", TypeOfProcess.GeneralRouting, "Fresa
D20", "-1", 0, -1, -1, 3, 0);
```

3.8.3.3 Saw strategies

3.8.3.3.1 Sectioning strategy

Defined by the instruction `CreateSectioningMillingStrategy`. The instruction applies to a saw cut machining a sectioning strategy, with a first cut to the specified depth along the cutting direction and a return pass at final depth. Strategy lets to keep the saw tool inside the workpiece between first cut and final cut, or to modify this position.

Name Type Description firstCutDepth double First cut depth outDistance double Out distance from sideMovingDistance double (opt def 0) Movement at right angles to the programmed linear trajectory, based on the current

tool correction (positive value for outside movement and negative for inside). `reateSectioningMillingStrategy` parameters

The example below applies to a saw cut machining a sectioning strategy with first cut depth 3 and out distance 10.

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry", 0, 0, 100, 0);
//Strategy
CreateSectioningMillingStrategy(3, 10, 0);
//Saw cut machining
CreateBladeCut("SawCut", "Saw cut", TypeOfProcess.GeneralRouting, "Lama D300", "-1",
90, 0, -1, -1, -1, 0, true, true, 0, 0);
```

3.8.3.4 Pocket strategies

3.8.3.4.1 Contour parallel pocket strategy

Defined with the instruction `CreateContourParallelStrategy`. The instruction applies to a closed pocket a contour parallel strategy, where tool passes follow the geometry profile. Strategy defines the way to execute pocketing, direction, rotation and applies if enabled multi pass along tool axis.

Name Type Description insideToOutside int (opt def true) Operation direction: true = operation performed from inside to outside; false = operation performed from outside to inside; rotationDirection int (opt def 1) Operation rotation direction: 0 = clockwise; 1 = counterclockwise. allowMultiplePasses bool (opt def false) Multiple pass roughing: true = standard roughing with multiple

passages (different layers of material are

removed in sequence, complying with the maximum cutting depth); false = special roughing with a single

pass (for prefabricated)

cuttingDepth double (opt def 0) Passage depth in the tool axial direction. finishCuttingDepth double (opt def 0) Final passage depth in the tool axial

direction (if used corresponds to the

final pass, used to reach the final programmed depth). strokeConnectionType int (opt def 1) Tool movement between two passages: 0 = tool remains inside the workpiece and moves to the start point of the next pass; 1 = tool exits the workpiece and moves quickly to the start point of the next pass; isHelicStrategy bool (opt def false) Enables the helic strategy

isApproachRetractPas bool (opt false) If true for each depth step an sStrategy approach/retract will be applied, if false the only approach/retract applied is at starting and ending of machining.

allowanceSide double (opt def 0) Layer of material which will be left on side of profile, positive value to leave material inside profile, negative value to create pass outside profile.

`CreateContourParallelStrategy` parameters

The example below applies a contour parallel strategy to a closed pocket with multi passes enabled, passes depth 5 and final pass 2

```
CreateFinishedWorkpieceBox("WorkPiece", 1600, 1200, 18);
//Geometry creation
CreateCircleCenterRadius("Circle", 25, 25, 200, false);
//Strategy
CreateContourParallelStrategy(true, 0 true, 5, 2);
//Pocket
CreateContourPocket("Pocket", 10, "Circle pocketing",
TypeOfProcess.ConcentricPocket, "Fresa D20", "-1", -1, -1, -1, 50, false);
```

3.8.3.4.2 ToeKick strategy

Defined with the instruction `CreateToekickMillingStrategy`. The instruction applies to a closed pocket a Toekick strategy to completely remove a workpiece corner or a workpiece open side with rough passes and a final finish pass.

Name Type Description radialFinishMaterial double (opt def 0) Width of material to remove with radial Width finish pass. isRadialRoughPassHor bool (opt def true) <c>true</c> if rough passes are executed izontal horizontally, <c>false</c>

vertically. rotationDirection double (opt 0) Direction used to perform machining. 0 = clockwise 1 = counterclockwise.

finishPassInputDist double (opt def 0) Lead in distance to apply at first point nce of radial finish pass.

finishPassOutputDist double (opt def 0) Lead out distance to apply at first point ance of radial finish pass.

finishPassSlowDistan double (opt def 0) Lead in distance slow speed to apply to ce radial finish pass. finishPassSlowSpeed

double (opt def -1) Lead in slow speed to apply to radial finish pass. allowMultiplePasses bool (opt def false) If true, this is the standard roughing operation with multiple passes, i. e. several layers of material are removed sequentially, taking into account the maximum cutting depth, if false, this is the special roughing operation for pre-cast features with one pass.

axialCuttingDepth double (opt def 0) Depth pass along tool axis axialFinishCuttingDe double (opt def 0) Finish depth pass along tool axis. It is pth the pass to reach the machining final depth

CreateContourParallelStrategy parameters

The example below applies a Toekick strategy to pocket.

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry
CreatePolyline("Geometry", 0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
ClosePolyline();
//Strategy
CreateToekickMillingStrategy(10, false, 1, 20, 30, 5, 7, true, 4, 5);
CreateToeKickPocket("ToekickNoStrategy", "", TypeOfProcess.ToekickPocket, "Fresa D20",
"-1");
```

3.8.4 Machine functions

To program the machine and perform specific procedures during the operations the user can use certain instructions called machine functions. This category includes, for example, functions used to enable, disable and configure the positioning of pneumatic hoods (PneumaticHood), the dustpan (Dustpan), etc.... Please note that this type of instructions are usually treated as “modal” functions inside the scripting. This means that the configurations set with these functions remain active for the subsequent operations, unless the settings are changed (reset or new setting). The operations that use these configurations do not “consume” the

setting: the user must remember to reset or change the configuration set if he no longer wants to use it in the subsequent operations, otherwise it remains active. There are also functions called “non-modal”. These functions set configurations whose validity is linked only to the first use: the first operation (after setting) uses and “consumes” the configuration. In this case, after the operation, the configuration is no longer available for the subsequent operations and it must be reset. By way of an example, many instructions defined “operation strategies”, described later, are part of this category.

See below for the main machine functions available to the user and the main input parameters that they accept.

3.8.4.1 Jerk machine function

Defined with instruction SetJerk. The instruction sets jerk.

Nome Tipo Descrizione gain double Valore di Jerk (0 : ripristino valori di default; >= 60 : valore consigliato per foratura; >= 90 : valore consigliato per sgrossatura; >=160 : valore consigliato per finitura) Parametri SetJerk

The example below shows an application of the Jerk instruction for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometria",0,0, 100, 0);
//Jerk
SetJerk(100);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
D20",-1,0,-1,-1,3,0);
```

The instruction ResetJerk resets the machine function.

3.8.4.2 Jerk 3D machine function

Defined with instruction SetJerk3D. The instruction sets jerk 3D.

Name Type Description gain double Jerk3D value: 0 = disabled; 1 = roughing; 2 = intermediate; 3 = finish. SetJerk3D parameters

The example below shows an application of the Jerk 3D instruction for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometria",0,0, 100, 0);
//Jerk
SetJerk3D(100);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
D20",-1,0,-1,-1,3,0);
```

The instruction ResetJerk3D resets the machine function.

3.8.4.3 NC Auto speed machine function

Defined with instruction SetCNAutoSpeed. The instruction sets the NC automatic speed.

The example below shows an application of the NC Auto Speed instruction for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
//Plane selection  
SelectWorkplane("Top");  
//Geometry creation  
CreateSegment("Geometria",0,0, 100, 0);  
//CN Auto speed
```

SetCNAutoSpeed();

```
//Routing  
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetCNAutoSpeed resets the machine function.

3.8.4.4 Main Pneumatic hood machine function

Defined with instruction SetPneumaticHoodPosition. The instruction sets the main hood pneumatic position. Position is a discrete value.

Name Type Description position int Pneumatic hood position: 0 = high hood 1 = low hood in position 1 2 = low hood in position 2 3 = low hood in position 3 4 = low hood in position 4 5 = low hood in position 5 6 = low hood in position 6 7 = low hood in position 7 8 = low hood in position 8 SetPneumaticHoodPosition parameters

The example below shows an application of the instruction to set hood position for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
//Plane selection  
SelectWorkplane("Top");  
//Geometry creation  
CreateSegment("Geometria",0,0, 100, 0);  
//Hood  
SetPneumaticHoodPosition(3);  
//Routing  
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetJerk3D resets the machine function.

3.8.4.5 Auxiliary pneumatic hood

Defined with instruction SetAuxiliaryHoodPosition. The instruction sets the main hood pneumatic position. Position is a discrete value.

Name Type Description position int Auxiliary hood position: 0 = high hood 1 = low hood in position 1 2 = low hood in position 2 3 = low hood in position 3 4 = low hood in position 4

5 = low hood in position 5 6 = low hood in position 6 7 = low hood in position 7 8 = low hood in position 8

SetAuxiliaryHoodPosition parameters

The example below shows an application of the instruction to set hood position for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
//Plane selection  
SelectWorkplane("Top");  
//Geometry creation  
CreateSegment("Geometria",0,0, 100, 0);  
//Hood  
SetAuxiliaryHoodPosition(3);  
//Routing  
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetAuxiliaryHoodPosition resets the machine function.

3.8.4.6 Opposite pneumatic hood

Defined with instruction SetOppositePneumaticHoodPosition. The instruction sets the main hood pneumatic position. Position is a discrete value.

Name Type Description position int Opposite hood position: 0 = high hood 1 = low hood in position 1 2 = low hood in position 2 3 = low hood in position 3 4 = low hood in position 4 5 = low hood in position 5 6 = low hood in position 6 7 = low hood in position 7 8 = low hood in position 8 SetOppositePneumaticHoodPositionparameters

The example below shows an application of the instruction to set hood position for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);  
//Plane selection  
SelectWorkplane("Top");  
//Geometry creation  
CreateSegment("Geometria",0,0, 100, 0);  
//Hood
```

```
SetOppositePneumaticHood(3);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetOppositePneumaticHood resets the machine function.

3.8.4.7 Main Motorized hood machine function

Defined with instruction SetMotorizedHoodPosition. The instruction sets the main motorized hood position. Positions are continue. If value is null, the position is automatically calculated, otherwise the hood will be set at specified value.

Nome Tipo Descrizione position double (def null) Hood position

SetMotorizedHoodPosition parameters

The example below shows an application of the instruction to set main hood position for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometria",0,0, 100, 0);
//Hood
SetMotorizedHoodPosition(100);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetMotorizedHood resets the machine function.

3.8.4.8 Automatic motorized hood machine function

Defined with instruction SetAutomaticMotorizedHood. The instruction sets the main motorized hood in automatic mode. The position is automatically calculated.

Name Type Description isUpperReference bool (def null) true to refere the hood to upper reference, false to refere the hood to lower reference.

offsetTopPlane double (def null) Offset respect to upper reference, allowed only with upper reference, with lower reference will be ignored. isAutoAdaptiveEn bool (def null) true to enable CN auto adaptive mode, abled otherwise false. Valid only with upper reference. autoAdaptiveOffs double (def null) Offset applied on auto adaptive mode is et enabled, if auto adaptive mode is disabled will be ignored. SetAutomaticMotorizedHood parameters

The example below shows an application of the instruction to set main hood in automatic mode for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Hood
SetAutomaticMotorizedHood(true, 10, true, 5);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetMotorizedHood resets the machine function.

3.8.4.9 Generic hood machine function

Defined with instruction SetHoodPosition. The instruction sets the main hood in relation of main hood type (motorized or pneumatic)

Name Type Description position double (def null) Hood position, if null hood position will be automatic calculated, otherwise hood is placed at specified value in relation of main hood type.

SetHoodPosition parameters

The example below shows an application of the instruction to set main hood in automatic mode for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Hood
SetHoodPosition(null);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetHood resets the machine function.

3.8.4.10 Clean mode machine function

Defined with instruction SetCleaningMode. The instruction sets clean machine function.

The example below shows an application of the instruction to set clean mode for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometria",0,0, 100, 0);
//Clean mode
SetCleaningMode();
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","-1",0,-1,-1,3,0);

The instruction ResetCleaningMode resets the machine function.

3.8.4.11 Pneumatic DustPan machine function

Defined with instruction SetDustpanOffset. The instruction enables the pneumatic dustpan. Position is a discrete value.

The instruction below lets to manage the dustpan by herself. It is possible to set the dustpan offset, the position and the lead in lead out.

Name Type Description offset double Dustpan distance (between dustpan and measured net radius/net radius measured at right angles to the tool path) position int Dustpan position: 0 = off; 1 = on in position 1; 2 = on in position 2; isInOutEnable bool (opt def null) true to enable approach and retract; otherwise false isTrajectoryForc bool (opt def null) true to force trajectory to avoid unexpected ed movement due to discontinuity segments; otherwise false

SetDustpanOffset parameters

The example below shows an application of the instruction to set dustpan for a routing operation

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//DustPan
SetDustpanOffset(10, 1, false);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","-1",0,-1,-1,3,0);

There is another possibility to set the dustpan using both the instructions SetDustpanOffset e SetDustpanPosition at the same time. The instruction SetDustpanOffset sets the offset without enabling the device and without setting the position.

Name Type Description offset double Dustpan distance (between dustpan and measured net radius/net radius measured at right angles to the tool path) SetDustpanOffset parameters

The instruction SetDustpanPosition enables the device to set position and lead in lead out.

Name Type Description position int Dustpan position: 0 = off; 1 = on in position 1; 2 = on in position 2; isInOutEnable bool (opt def null) true to enable approach and retract; otherwise false isTrajectoryForc bool (opt def null) true to force trajectory to avoid unexpected ed movement due to discontinuity segments; otherwise false

SetDustpanPosition parameters

The example below shows an application of the instruction to set dustpan for a routing operation

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//DustPan
SetDustpanPosition(1, false);
SetDustpanOffset(10);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","-1",0,-1,-1,3,0);

L'istruzione ResetDustpan resetta la funzione macchina.

3.8.4.12 Motorized DustPan machine function

Defined with instruction SetMotorizedDustpanPosition. The instruction enables the motorized dustpan. Position is a continuous value

The instruction below enables the dustpan device, setting the position and lead in lead out.

Nome Tipo Descrizione position double (opt def DustPan position. null) isInOutEnable bool (opt def true to enable approach and retract; otherwise null) false isTrajectoryForc bool (opt def true to force trajectory to avoid unexpected ed null) movement due to discontinuity segments; otherwise false

SetMotorizedDustpanPosition parameters

The example below shows an application of the instruction to set dustpan for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//DustPan
```

```
SetMotorizedDustpanPosition(100, false);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction below SetMotorizedDustpanOffset sets the 'offset and must be used in combination with previous instruction

Nome Tipo Descrizione offset double Dustpan distance (between dustpan and measured net radius/net radius measured at right angles to the tool path)

SetMotorizedDustpanOffset parameters

The example below shows an application of the instruction to set dustpan for a routing operation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//DustPan
SetMotorizedDustpanPosition(100, false);
SetMotorizedDustpanOffset(10);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20",-1",0,-1,-1,3,0);

The instruction ResetMotorizedDustpan resets the machine function.

3.8.4.13 Five axis head unroll machine function

Defined with instruction SetUnrollHeadMode.

The instruction enables the machine function to recover 5 axis angular recover.

Nome Tipo Descrizione mode int (opt def 0) Mode. 0 = disabled; 1 = perpendicular input and output; 2 = tangential input and perpendicular output; 3 = perpendicular input and tangential output; 4 = tangential input and output; radiusMultiplier double (opt def 1.0) Radius multiplier SetUnrollHeadMod parameters

The example below creates a routing with 5 axis head unroll.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//5 axis head unroll
SetUnrollHeadMode(0, 1.2);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20,"3",0,-1,-1,3,0);

The instruction ResetMotorizedDustpan resets the machine function.

3.8.4.14 Axis brake machine function

Defined with instruction SetBrakes. The instruction enables the machine function to brake the rotative axis.

Nome Tipo Descrizione activeBrakes string Active brakes (es. "B", "B,C").

SetBrakes parameters

The example below creates a routing with rotative axis brake

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Brake axis
SetBrakes("B,C");
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20,"3",0,-1,-1,3,0);

3.8.4.15 ToolBlower machine function

Defined with instruction SetToolBlower. The instruction enables the machine function to blow the tool.

Nome Tipo Descrizione toolBlowerMode int Tool blowing mode 0 Ari. 1 Air and oil. 2 Head. 3 Air tank. 4 Air and oil tank.

SetToolBlower parameters

The example below creates a routing with tool blowing

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Tool blowing
```

```
SetToolBlower(0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.4.16 Electronic zero prober machine function

Defined with instruction SetElectronicZeroProber. The instruction enables the machine function to probe zero.

Nome Tipo Descrizione gain double Gain offset double Offset compression double Compression

SetElectronicZeroProber parameters

The example below creates a routing with electronic zero probing

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Electronic zero prober
SetElectronicZeroProber(10, 20, 1);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

The instruction ResetElectronicZeroProber resets the machine function.

3.8.4.17 Pressure rollers machine function

Defined with instruction SetPressureRollers. The instruction enables the machine function to use the pressure rollers during machining execution..

Nome Tipo Descrizione offset double Positioning offset of pressure device

SetPressureRollers parameters

The example below creates a routing with pressure rollers

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Pressure rollers
SetPressureRollers(10);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

The instruction ResetPressureRollers resets the machine function.

3.8.4.18 Mechanical pressure machine function

Defined with instruction SetMechanicalPresser. The instruction enables the machine function to use the mechanical pressure.

The example below creates a routing with mechanical pressure

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreateSegment("Geometry",0,0, 100, 0);
//Mechanical pressure
SetMechanicalPresser();
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

The instruction ResetMechanicalPresser resets the machine function.

3.8.5 Machining attributes

Machining attributes are properties that can be associated to geometry elements or toolpath elements. Machining attributes can be specified in each geometry element or toolpath element to modify the toolpath.

3.8.5.1 Machine attributes types

3.8.5.1.1 Feed attribute (FEED)

Feed attribute (FEED) modifies the machining feed in the specified position of the geometry element or toolpath element.

The attribute can be applied to element start point

Name Type Description name string Attribute name "FEED" value double Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE). SetAttribute parameters

The example below applies a feed attribute to a routing in the start point of the second geometry element. Feed starts from this point.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
SetAttribute("FEED", 5);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

or can be applied in a position of the element

Name Type Description name string Name of the attribute. "FEED" value double Attribute value (for the boolean use 0 for FALSE, different than zero for TRUE). uPar Double Position attribute (allowable values 0.0-1.0) Parameters
SetParametricAttribute

The example below applies a feed attribute to a routing in the middle point of the second geometry element. Feed starts from this point.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
SetParametricAttribute("FEED", 5, 0.5);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.5.1.2 Depth attribute (DEPTH)

Depth attribute (FEED) modifies the machining depth in the specified position of the geometry element or toolpath element.

The attribute can be applied to element end point

Name Type Description name string Attribute name "DEPTH" value double Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE). SetAttribute parameters

The example below applies a depth attribute to a routing in the end point of the second geometry element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
SetAttribute("DEPTH", 5);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

or can be applied in a position of the element

Name Type Description name string Name of the attribute. "DEPTH" value double Attribute value (for the boolean use 0 for FALSE, different than zero for TRUE). uPar Double Position attribute (allowable values 0.0-1.0) Parameters
SetParametricAttribute

The example below applies a depth attribute to a routing in the middle point of the second geometry element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
SetAttribute("DEPTH", 5, 0.5);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.5.1.3 Micro junction attribute (TAB)

Micro junction attribute (TAB) modifies the machining toolpath to leave a portion of unmachined material.

The attribute can be applied to element end point

Name Type Description name string Attribute name "TAB" value double Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE). SetAttribute parameters

The example below applies a micro junction attribute to a routing in the end point of the second geometry element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
SetAttribute("TAB", 5, 2);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

or can be applied in a position of the element

Name Type Description name string Name of the attribute. "TAB" value double Attribute value (for the boolean use 0 for FALSE, different than zero for TRUE). uPar Double Position attribute (allowable values 0.0-1.0) Parameters

SetParametricAttribute2

The example below applies a micro junction attribute to a routing in the middle point of the second geometry element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
SetParametricAttribute2("TAB", 5, 2, 0.5);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.5.1.4 Start point attribute

Defined with instruction SetStartPointAttribute. Start point attribute modifies the machining start point.

Nome Tipo Descrizione elementIndex int Index of the element to set start point value double Start point relative position on the element expressed in percentage

SetStartPointAttribute parameters

The example below applies a start point on a routing in the middle point of the second element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
SetStartPointAttribute(1, 50);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.5.1.5 Start point position attribute

Defined with instruction SetStartMachiningPosition. Start point position attribute modifies the machining start point position. The application is relative to machining and not relative to geometry or toolpath.

Nome Tipo Descrizione position int Start point position of machining : 0 not defined 1 middle of first side 2 middle of longest side 3 vertex of first side

Parametri SetStartMachiningPosition

The example below applies a start point position on a routing in the middle point of the longest side. .

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
SetStartMachiningPosition(1);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.5.1.6 End point attribute

Defined with instruction SetEndPointAttribute. End point attribute modifies the machining end point

Name Type Description elementIndex int Index of the element to set end point value double End point relative position on the element expressed in percentage

SetEndPointAttribute parameters

The example below applies an end point on a routing in the middle point of the second element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
SetEndPointAttribute(1, 50);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.6 Correction reference

Defined with instruction SetProbingReferenceReferencePoint.

The instruction allows you to specify the correction reference for machining operations on PowerFlex machines. If part measurement is enabled on the machine, the machining will be corrected, to keep its position unchanged from the reference point. The method must be inserted into the script before any machining operation on which it is intended to specify a reference correction that differs from the standard

one. Remember that on PowerFlex machines, the standard correction gives as a reference point: ● the upper left corner for processing on the upper face: ● the left lower corner for processing on the left, front and bottom faces; ● the lower right corner for processing on the right and rear sides.

Name Type Description referencePoint int Identifies the reference point on the active work plane for machining corrections after measuring the part. Instruction used on PowerFlex machines.

0 = no dot (standard correction) 1 = upper left corner 2 = lower left corner 3 = upper right corner 4 = lower right corner 5 = central point

SetProbingReferenceReferencePoint parameters

The example below creates a routing with probing correction upper left.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
SetProbingReferenceReferencePoint(1);
//Fresatura
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.7 Machining inversion

Defined with instruction SetMachiningDirection. The instruction modifies the machining direction with respect to geometry direction. Generated toolpath could be in the same direction of geometry or could be in the opposite direction.

Name Type Description sameDirection bool Type of direction: true = Machining in the same direction of the geometry; false = machining in the opposite direction of the geometry;

SetMachiningDirection parameters

The example below creates a routing with direction opposite to geometry. Toolpath will be clockwise.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
SetMachiningDirection(false);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.8 Tool compensation

Defined with instruction SetToolCorrection. The instruction modifies the tool compensation of a machining with respect to geometry. Compensation is relative to radial tool position with respect to geometry (central, left, right) and is relative to axial position (length compensation).

Name Type Description correction int Tool compensation: 0 - Center tool 1 - Left compensation 2 - Right compensation 3 - Length compensation 13 - Left and length compensation 23 - Right and length compensation overMaterial double (opt def 0) Overmaterial to apply to compensation

SetToolCorrection parameters

The example below creates routing with different compensation values.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing center tool
SetToolCorrection(0);
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Routing with left compensation
SetToolCorrection(1);
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Routing with right compensation
SetToolCorrection(2);
CreateRoughFinish("Routing3",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Routing with length compensation
SetToolCorrection(3);
CreateRoughFinish("Routing4",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Routing with left compensation and length compensation
SetToolCorrection(13);
CreateRoughFinish("Routing5",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Routing with right compensation and length compensation
SetToolCorrection(23);
CreateRoughFinish("Routing6",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.9 Toolpath generation mode

Defined with instruction SetCompensationMode. The instruction modifies the toolpath generator for the machining compensation. Tool compensation could be generated by Cad or by NC at machine execution time.

Name Type Description isCnc bool true for tool compensation generated by CN è generata dal CN, false for tool compensation generated by Cad

SetCompensationMode parameters

The example below creates routings with different toolpath generation modes.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing with NC toolpath generation
SetCompensationMode(true);
CreateRoughFinish("Fresatura1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Routing with Cad toolpath generation
SetCompensationMode(false);
CreateRoughFinish("Fresatura2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.10 Breakeable machining

Defined with instruction SetBreakableMode. The instruction indicates to optimizer that the machining could be broken and replaced with N instructions in order to execute it on the machine.

Name Type Description isBreakable bool true machining can be broken, false machining can not be broken.
 approachAndRetractOn bool true if milling lead in and lead out must Parts be applied to sub-parts, false otherwise.

SetBreakableMode parameters

The example below creates routing that can be breaked.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Breakable Routing
SetBreakableMode(true, false);
CreateRoughFinish("Fresatura1",20,"", TypeOfProcess.GeneralRouting,"Fresa
D20","3",0,-1,-1,3,0);
```

3.8.11 Machining lead in safety distance

Defined with instruction SetApproachSecurityDistance. The instruction modifies the lead in safety distance. If not defined, value is the options default value.

Name Type Description value double (def opt null) Lead in safety distance

SetApproachSecurityDistance parameters

The example below creates a routing with lead in safety distance 10..

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Breakable Routing
SetBreakableMode(true, false);
CreateRoughFinish("Fresatura1",20,"", TypeOfProcess.GeneralRouting,"Fresa
D20","3",0,-1,-1,3,0);
```

```
//Routing with lead in safety distance 10
SetApproachSecurityDistance(10);
CreateRoughFinish("Fresatura2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.12 Machining lead out safety distance

Defined with instruction SetRetractSecurityDistance. The instruction modifies the lead out safety distance. If not defined, value is the options default value.

Name Type Description value double (def opt null) Lead out safety distance

SetRetractSecurityDistance parameters

The example below creates a routing with lead out safety distance 10.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Breakable Routing
SetBreakableMode(true, false);
CreateRoughFinish("Fresatura1",20,"", TypeOfProcess.GeneralRouting,"Fresa
D20","3",0,-1,-1,3,0);
```

```
//Routing with lead out safety distance 10
SetRetractSecurityDistance(10);
CreateRoughFinish("Fresatura2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

3.8.13 Machine selection

Defined with instruction SetMachine. The instruction selects the machine to execute the machining. Allowed only in multi machine program.

Name Type Description id int Machine index starting from 1

SetMachine parameters

The example below creates a routing with lead out safety distance 10.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Breakable Routing
SetBreakableMode(true, false)
CreateRoughFinish("Fresatura1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//First machine selection
SetMachine(1);
CreateRoughFinish("Fresatura2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

3.8.14 Drilling spindles selection

Defined with instruction SetSpindles. The instruction selects the spindles to use to create drilling. First spindle is the reference spindle

Name Type Description id int Machine index starting from 1

SetMachine parameters

The example below creates a drilling with spindle 1, 5, 7.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SetSpindles(1, 5, 7);
CreateDrill("Hole",10,20,15,8, "",TypeOfProcess.Drilling);
```

3.8.15 Machining priority

Defined with instruction SetPriority. The instruction sets the execution priority to a machining. Machining priority is used to define the way the optimizers modifies the working sequence. Priority value must be greater or equal 0, 0 value means the machining is not prioritized. Priority behaviour is different from a working center machine and a clamps machine.

Name Type Description priority int Machining priority (from 1 to N)

SetPriority parameters

3.8.15.1 Working center machine

In the working center machine case, priorities are used to indicate to the optimizers which machinings have the same technology and must be executed on the same group and the group execution sequence. All machinings with the same technology must have the same priority value. Priority value defines the sequence with 2 or more machinings have to be executed. It is positively increasing and the execution sequence is from 1 to N.

Starting from a project like this

Machining 1 Machining 2 Machining 3 Machining 4 Machining 5 Machining 6

where odd machinings have to be executed first and even machinings after

Machining 1 Machining 3 Machining 5

Machining 2 Machining 4 Machining 6

it is necessary to set odd machinings with same priority values and even machinings with same priority value

Machining 1 - Priority 1 Machining 3 - Priority 1 Machining 5 - Priority 1

Machining 2 - Priority 2 Machining 4 - Priority 2 Machining 6 - Priority 2

In this way, odd machinings have priority 1 and even machinings have priority 2, the optimizers must optimize all machinings with priority 1 in the same group and all machinings with priority 2 in another group, respecting the group sequence.

A sample result could be

Machining 5 Machining 1 Machining 3

Machining 6 Machining 2 Machining 4

where it is shown that inside the same group the machining sequence is changed, but the priority sequence is respected, odd machinings before even machinings.

The example below shows the complete example priority definition

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing 1 priority 1
SetPriority(1);
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 2 priority 2
SetPriority(2);
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 3 priority 1
SetPriority(1);
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 4 priority 2
SetPriority(2);
CreateRoughFinish("Routing4",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 5 priority 1
SetPriority(1);
CreateRoughFinish("Routing5",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 6 priority 2
SetPriority(2);
CreateRoughFinish("Routing6",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

3.8.15.2 Clamps machine

In the clamps machine case, priorities are used to indicate to optimizers which machinings are to be inserted in the same clamping and which sequence they have to respect. To have machinings in the same clamping, it is necessary to create groups, groups are created setting incremental values to priorities, the first next machining with a priority value lower or equal to last priority defines a new group. Machining with priority value equal to 0 could be in whatever clamping.

Starting from a project like this

Machining 1 Machining 2 Machining 3 Machining 4 Machining 5 Machining 6

where odd machinings have to be executed in the same clamping and even machinings have to be in another clamping

First clamping

Machining 1 Machining 3 Machining 5

Second clamping

Machining 2 Machining 4

Machining 6

it is necessary to set to odd machinings priority increasing starting from 1 and to set to even machining priority increasing starting from 1

Machining 1 - Priority 1 Machining 3 - Priority 2 Machining 5 - Priority 3

Machining 2 - Priority 1 Machining 4 - Priority 2 Machining 6 - Priority 3

In this way, odd machinings will be inserted in the same clamping and executed in the specified sequence,

Machining 1 Machining 3 Machining 5

and even machinings will be inserted in the next clamping and executed in the specified sequence

Machining 2 Machining 4 Machining 6

The example below shows the complete example priority definition

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing 1 priorità 1
SetPriority(1);
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 3 priorità 2
SetPriority(2);
CreateRoughFinish("Routing3",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 5 priorità 3
SetPriority(3);
CreateRoughFinish("Routing5",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
//Routing 2 priorità 1
SetPriority(1);
```

```
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 4 priorità 2
```

```
SetPriority(2);
```

```
CreateRoughFinish("Routing4",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 6 priorità 3
```

```
SetPriority(3);
```

```
CreateRoughFinish("Routing6",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

Groups definition must be done in an incremental sequence, all machinings with priority lower or equal to last priority value creates a new group.

Starting from previous example machinings

Machining 1 Machining 2 Machining 3 Machining 4 Machining 5 Machining 6

where is necessary to have one clamping for odd machinings and one clamping for even machinings

Machining 1 - Priority 1 Machining 3 - Priority 2 Machining 5 - Priority 3

Machining 2 - Priority 1 Machining 4 - Priority 2 Machining 6 - Priority 3

if the priority sequence is not respected the result will be very different. For example changing the machining sequence definition

Machining 1 - Priority 1 Machining 2 - Priority 1

Machining 3 - Priority 2 Machining 4 - Priority 2

Machining 5 - Priority 3 Machining 6 - Priority 3

the result will be very different than expected and will be

First clamping

Machining 1 - Priority 1

Secon clamping

Machining 2 - Priority 1 Machining 3 - Priority 2

Third clamping

Machining 4 - Priority 2 Machining 5 - Priority 3

Forth clamping

Machining 6 - Priority 3

The example below shows the complete example priority definition

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
```

```
//Plane selection
```

```
SelectWorkplane("Top");
```

```
CreatePolyline("Geometry",0,0);
```

```
AddSegmentToPolyline(0,100);
```

```
AddSegmentToPolyline(100,100);
```

```
AddSegmentToPolyline(0,100);
```

```
AddSegmentToPolyline(0,0);
```

```
//Routing 1 priority 1
```

```
SetPriority(1);
```

```
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 2 priority 1
```

```
SetPriority(1);
```

```
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 3 priority 2
```

```
SetPriority(2);
```

```
CreateRoughFinish("Routing3",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 4 priority 2
```

```
SetPriority(2);
```

```
CreateRoughFinish("Routing4",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 5 priority 3
```

```
SetPriority(3);
```

```
CreateRoughFinish("Routing5",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Routing 6 priority 3
```

```
SetPriority(3);
```

```
CreateRoughFinish("Routing6",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

3.8.16 Explicit toolpath

Explicit toolpath is a 2.5D toolpath that can be explicit defined and is not affected by toolpath generation calculation. It can be executed directly on the machine as is defined. The toolpath is defined with a sequence of linear and arc elements with X,Y,Z position relative to the associated machining plane system reference. The explicit toolpath definition needs first to create a machining in the classic way (plane, geometry, machining) and after assigning to it a toolpath. It is important to define a machining to create the shape of machining ("what to do") and the toolpath to define the way to machine it ("how to do"). In this case it is not allowed to use instructions to define lead in , lead out, strategies etc.

3.8.16.1 Explicit toolpath creation

3.8.16.1.1 Explicit toolpath first point

Defined with instruction CreateToolpath. The instruction creates the toolpath start point from which will be inserted next elements. Start point coordinates are local to the machining plane. This instruction must be always followed by instructions to insert linear and arc elements else the toolpath is not consistent.

Name Type Description name string Tool path name startX double Coordinate X of the first tool path point. startY double Coordinate Y of the first tool path point. startZ double Coordinate Z of the first tool path point. CreateToolpath parameters

The example below creates a routing with an explicit toolpath with first point

```
coordinates (0,0,-10).
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
```

The instruction creates the toolpath start point from which will be inserted next elements and lets you define if it is necessary to generate the tool compensation of the toolpath. Start point coordinates are local to the machining plane. This instruction must be always followed by instructions to insert linear and arc elements else the toolpath is not consistent.

Name Type Description name string Tool path name startX double Coordinate X of the first tool path point. startY double Coordinate Y of the first tool path point. startZ double Coordinate Z of the first tool path point. isCompensationEnable bool true if is requesting to generate tool d compensation on provided toolpath, otherwise false.

CreateToolpath parameters

The example below creates a routing with an explicit toolpath with first point

```
coordinates (0,0,-10).
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10, true);
```

3.8.16.1.2 Adding a linear toolpath to explicit toolpath

Defined with instruction AddSegmentToToolpath. The instruction adds to the active toolpath a linear toolpath where the first point is the last point of the current toolpath and the end point is the specified point.

Name Type Description endX Double Coordinate X of the last tool path point. endY Double Coordinate Y of the last tool path point. endZ Double Coordinate Z of the last tool path point. AddSegmentToToolpath parameters

The example below creates a routing with an explicit toolpath made by a linear toolpath with first point coordinates (0,0,-10) and end point coordinates (0,100,-20).

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
```

```
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
D20","3",0,-1,-1,3,0);
```

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddSegmentToToolpath(0,100,-20);
```

3.8.16.1.3 Add an arc toolpath by 3 points to explicit toolpath

Defined with instruction AddArc3PointsToToolpath. The instruction adds to the active toolpath an arc by 3 points. First point is the last point of the current toolpath and the second and third point are the specified points.

Name Type Description p2X Double Coordinate X of the second point. p2Y Double Coordinate Y of the second point. p3X Double Coordinate X of the third point. p3Y Double Coordinate Y of the third point. p3Z Double Coordinate Z of the third point. AddArc3PointsToToolpath parameters

The example below creates a routing with an explicit toolpath made by a 3 points arc

```
toolpath (0,0,-10), (100, 50), (0, 100, -30)
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddArc3PointsToToolpath(100,50, 0, 100, -30);
```

3.8.16.1.4 Add an arc toolpath by 2 points and center to explicit toolpath

Defined with instruction AddArc2PointCenterToToolpath. The instruction adds to the active toolpath an arc by 2 points and center. First point is the last point of the current toolpath, the second and center point are the specified points.

Name Type Description p2X Double Coordinate X of the second point. p2Y Double Coordinate Y of the second point. p3X Double Coordinate X of the third point. p3Y Double Coordinate Y of the third point. p3Z Double Coordinate Z of the third point. AddArc2PointCenterToToolpath parameters

The example below creates a routing with an explicit toolpath made by an anticlockwise arc by 2 points and center (0,0,-10), (100, 50), (0, 100, -30)

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddArc2PointCenterToToolpath(0,100, -30, 0, 50, false);
```

3.8.16.1.5 Add an arc toolpath by 2 points and radius to explicit toolpath

Defined with instruction AddArc2PointRadiusToToolpath.

The instruction adds to the active toolpath an arc by 2 points and radius. First point is the last point of the current toolpath, the second point and radius are specified.

Name Type Description endX double Coordinate X of the last tool path point. endY double Coordinate Y of the last tool path point. endZ double Coordinate Z of the last tool path point. radius double Arc radius isClockwise bool Arc constructed clockwise isOver180 bool Arc angle greater than 180° AddArc2PointRadiusToToolpath parameters

The example below creates a routing with an explicit toolpath made by an anticlockwise arc by 2 points (0,0,-10), (0, 100, -30), and radius 50.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
```

```
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddArc2PointCenterToToolpath(0,100, -30, 50, false, false);
```

3.8.16.1.6 Add an arc toolpath by center and angle

Defined with instruction `AddArcCenterAngleToToolpath`. The instruction adds to the active toolpath an arc by center point and angle. First point is the last point of the current toolpath, the center point and angle are specified.

Name Type Description centerX double Coordinate X of centre point centerY double Coordinate Y of centre point endZ double Coordinate Z of the last tool path point. angle double Arc angle `AddArcCenterAngleToToolpath` parameters

The example below creates a routing with an explicit toolpath made by an anticlockwise arc by center point (0,50, 0), and angle 180 degree.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
```

3.8.16.2 Explicit toolpath attributes

3.8.16.2.1 Explicit toolpath feed attribute (FEED)

Feed attribute (FEED) modifies the toolpath feed in the specified position of the toolpath element.

The attribute can be applied to element start point

Name Type Description name string Attribute name "FEED" value double Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE). `SetAttribute` parameters

The example below applies a feed attribute to an explicit toolpath in the start point of the second toolpath element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

```
D20","3",0,-1,-1,3,0);
```

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddSegmentToToolpath(100,0);
SetToolpathAttribute("FEED", 30);
AddSegmentToToolpath(100,100);
AddSegmentToToolpath(0,100);
AddSegmentToToolpath(0,0);
```

3.8.16.2.2 Explicit toolpath depth attribute (DEPTH)

Depth attribute (DEPTH) modifies the toolpath depth in the specified position of the toolpath element.

The attribute can be applied to element end point

Name Type Description name string Attribute name "DEPTH" value double Attribute value (for the Booleans use 0 for FALSE, other than zero for TRUE). `SetAttribute` parameters

The example below applies a depth attribute to an explicit toolpath in the end point of the second toolpath element.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
```

```
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath", 0,0,-10);
AddSegmentToToolpath(100,0);
SetToolpathAttribute("DEPTH", 30);
AddSegmentToToolpath(100,100);
AddSegmentToToolpath(0,100);
AddSegmentToToolpath(0,0);
```

3.8.16.3 Explicit toolpath selection

Defined with instruction SelectToolpath. The instruction selects the toolpath as active toolpath.

Toolpath can be selected by index

Name Type Description index int Index of toolpath to select SelectToolpath parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry1",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath1", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//Geometry creation
CreatePolyline("Geometry2",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Fresatura
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath2", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//First toolpath selection
SelectToolpath("0");
```

or by name

Name Type Description name string Name of the toolpath to select SelectToolpath parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry1",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath1", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//Geometry creation
CreatePolyline("Geometry2",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath2", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//First toolpath selection
SelectToolpath("Toolpath1");
```

3.8.16.4 Explicit toolpath deletion

Defined with instruction DeleteToolpath. The instruction deletes the toolpath from the project.

Toolpath can be deleted by index

Name Type Description index int Index of toolpath to delete. DeleteToolpath parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry1",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath1", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//Geometry creation
CreatePolyline("Geometry2",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Fresatura
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath2", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//First toolpath deletion
DeleteToolpath("0");
```

or by name

Name Type Description name string Name of the toolpath to select DeleteToolpath parameters

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometry1",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing1",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath1", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//Geometry creation
CreatePolyline("Geometry2",0,0);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Routing
CreateRoughFinish("Routing2",20,"", TypeOfProcess.GeneralRouting,"Fresa
```

D20","3",0,-1,-1,3,0);

```
//Explicit toolpath creation
CreateToolpath("Toolpath2", 0,0,-10);
AddArcCenterAngleToToolpath(0,50, -30, 180);
//First toolpath deletion
DeleteToolpath("Toolpath1");
```

3.8.17 Machining technologies

Machining technologies let the user to create machinings applying to a reference geometry one of available technologies in Maestro. Technologies are machining information containers

3.8.17.1 Machining technologies application

Defined with instruction ApplyTechnology. The instruction creates a routing on active geometry from all machining information contained inside the technology.

Name Type Description name string Name of the technology to apply ApplyTechnology parameters

L'esempio seguente mostra l'applicazione di una tecnologia di nome "Fresatura D10.tchx" a una geometria

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Geometry creation
CreatePolyline("Geometria",0,0);
AddSegmentToPolyline(100,0);
AddSegmentToPolyline(100,100);
AddSegmentToPolyline(0,100);
AddSegmentToPolyline(0,0);
//Technology application
ApplyTechnology("Fresatura D10");
```

3.8.18 Feedrate profiles

Feedrate profiles are applicable to drilling machinings and are used to manage speeds to execute the operation to lead in the material, to work inside the material and the lead out from material.

3.8.18.1 Feedrate profile application

Defined with instruction SetFeedrateProfile. The instruction applies to a drilling machining the feedrate profile associated to specified name. Feedrate profile (.fbpx) must be previously created. It is still active until it is reset.

Name Type Description feedrateProfileName string Name of feedrate profile to apply. SetFeedrateProfile parameters

The example below applies a feedrate profile named "Depth drill feedrate profile" to drilling machining.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Drilling
CreateDrill("Drilling",10,20,15,8, "",TypeOfProcess.Drilling);
//Feedrate profile application
SetFeedrateProfile("Depth drill feedrate profile");
```

3.8.18.2 Feedrate profile reset

Defined with instruction ResetFeedrateProfile. The instruction reset the active feedrate profile.

3.9 Macro

Inside a MSL file is possible to use Macros installed on Maestro or Macro created by the user.

3.9.1 Macro usage

3.9.1.1 Macro inserting

Defined with the instruction CreateMacro or in equal mode with instruction ExecMacro. The instruction inserts a Macro with default values inside the project. Macro is created on active plane.

Name Type Description name string Nome of the Macro inside project macroName string Identification name of the Macro CreateMacro parameters

The example below inserts a Macro with default values inside the project.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Macro creation
CreateMacro("Macro", "Cerniera3");
```

or it is possible to pass parameters in input to override Macro default values. Parameters must be passed respecting the definition sequence, in a Scripting Macro (.cs), sequence is defined by AddParameter inside the constructors, not considering workpiece parameters ("WorkpieceLength", "WorkpieceWidth", "WorkpieceHeight").

Name Type Description Name string Name of the Macro inside project macroName string Name of Macro macroParameters Object[] Array of Macro input parameters

Parametri CreateMacro

For example, a Macro with this parameter in the constructor

```
AddParameter("WorkpieceLength", 1200, typeof(Double), "WorkpieceLengthDescription",
```

```
ParameterType.WorkpieceLength, false, FisicalUnitType.Lenght, false);
```

```
AddParameter("WorkpieceWidth", 800, typeof(Double), "WorkpieceWidthDescription",
```

```
ParameterType.WorkpieceWidth, false, FisicalUnitType.Lenght, false);
```

```
AddParameter("WorkpieceHeight", 18, typeof(Double), "WorkpieceHeightDescription",
```

```
ParameterType.WorkpieceHeight, false, FisicalUnitType.Lenght, false);
```

```
AddParameter("P1",0, typeof(Double), "", ParameterType.UserDefined, true,
```

```
FisicalUnitType.Lenght, false);
```

```
AddParameter("P2",0, typeof(Double), "", ParameterType.UserDefined, true,
```

```
FisicalUnitType.Lenght, false);
```

```
AddParameter("P3",0, typeof(Double), "", ParameterType.UserDefined, true,
```

FisicalUnitType.Lenght, false);

To override P2 parameter, it is necessary to pass all preceding parameters, the following parameters can be omitted and default values will be used.

In the case of a Macro with this parameters in the constructor

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
AddParameter("WorkpieceLength", 1200, typeof(Double), "WorkpieceLengthDescription",
ParameterType.WorkpieceLength, false, FisicalUnitType.Lenght, false);
AddParameter("WorkpieceWidth", 800, typeof(Double), "WorkpieceWidthDescription",
ParameterType.WorkpieceWidth, false, FisicalUnitType.Lenght, false);
AddParameter("WorkpieceHeight", 18, typeof(Double), "WorkpieceHeightDescription",
ParameterType.WorkpieceHeight, false, FisicalUnitType.Lenght, false);
AddParameter("ReferencePosition", 0, typeof(Int32), "Spigolo di riferimento",
ParameterType.ReferencePosition, true, FisicalUnitType.UnitLess, false);
AddParameter("X", 126, typeof(Double), "Centro in X foratura cerniera",
ParameterType.UserDefined, true, FisicalUnitType.Lenght, false);
AddParameter("Y", 22.5, typeof(Double), "Centro in Y foratura cerniera",
ParameterType.UserDefined, true, FisicalUnitType.Lenght, false);
AddParameter("Z", 13, typeof(Double), "Profondità foratura cerniera",
ParameterType.UserDefined, true, FisicalUnitType.Lenght, false);
AddParameter("H", 13, typeof(Double), "Profondità foratura gruppo",
ParameterType.UserDefined, true, FisicalUnitType.Lenght, false);
AddParameter("J", 3, typeof(Double), "Diametro gruppo foratura", ParameterType.UserDefined,
true, FisicalUnitType.Lenght, false);
AddParameter("V", 2, typeof(Double), "Avanzamento cerniera", ParameterType.UserDefined,
true, FisicalUnitType.Speed, false);
AddParameter("X1", 48, typeof(Double), "Distanza foratura gruppo in X",
ParameterType.UserDefined, true, FisicalUnitType.Lenght, false);
AddParameter("Y1", 6, typeof(Double), "Distanza foratura gruppo in Y",
ParameterType.UserDefined, true, FisicalUnitType.Lenght, false);
AddParameter("D", 35, typeof(Double), "Diametro cerniera", ParameterType.UserDefined, true,
FisicalUnitType.Lenght, false);
AddParameter("V2", 2, typeof(Double), "Avanzamento gruppo foratura",
ParameterType.UserDefined, true, FisicalUnitType.Speed, false);
AddParameter("RotationAngle", 0, typeof(Double), "Angolo di rotazione",
ParameterType.UserDefined, true, FisicalUnitType.UnitLess, false);
AddParameter("Machine", 0, typeof(Int32), "Macchina", ParameterType.Machine, true,
FisicalUnitType.UnitLess, false);
AddParameter("HeadsGroupType", "0", typeof(String), "Tipo gruppo teste",
ParameterType.HeadsGroupType, true, FisicalUnitType.UnitLess, false);
AddParameter("HeadsGroupNumber", 0, typeof(Int32), "Numero gruppo teste",
ParameterType.HeadsGroupNumber, true, FisicalUnitType.UnitLess, false);
AddParameter("Head", "0", typeof(String), "Testa", ParameterType.Head, true,
FisicalUnitType.UnitLess, false);
```

to override X,Y position is necessary to pass the "ReferencePosition" parameter before the X, Y parameters.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Macro creation
CreateMacro("Macro", "Cerniera3", 0, 200, 300);
```

3.9.1.2 Macro parameters passing

Defined with instruction SetMacroParam. The instruction passes a parameter value by name and must be called before the instruction CreateMacro to override default values. Parameters are specified by name and the instruction can be used with Scripting Macros and with User Macro.

Name Type Description parName string Name of parameter value Object Value of parameter (Double,Int32,String,Bool)
Parametri SetMacroParam parameters

The example below creates a Macro specifying X, Y parameters. Macro is created with all default values except X and Y that will assume specified values.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Plane selection
SelectWorkplane("Top");
//Macro creation
SetMacroParam("X", 100);
SetMacroParam("Y", 200);
CreateMacro("Macro", "Cerniera3");
```

3.10 NC Functions

NC functions can be inserted in the project flow to execute accessories operations to machinings.

3.10.1 WorkPiece pressure

Defined with instruction CreateWorkpiecePressureOperation. The instruction creates the workpiece pressure operation.

Name Type Description name string NC function name. startX double X coordinate of pressure start point. startY double Y coordinate of pressure start point. endX double X coordinate of pressure end point. endY double Y coordinate of pressure end point. compression double Workpiece compression description string (opt def Nc function description. vuoto) approachSpeed double (opt def -1) Lead in speed, if not set, pressure head rapid speed will be used. workingSpeed double (opt def -1) Feed speed, if not set, pressure head feed speed will be used.

CreateWorkpiecePressureOperation parameters

The example below creates a workpiece pressure nc function.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
CreateWorkpiecePressureOperation("NC function", 10, 10, 20, 20, 1);
```

3.10.2 Machine parking

Defined with instruction CreatePark. The instruction creates the workpiece pressure operation.

Name Type Description name string NC function name. stopType String Type of stop to perform: Nothing = No stop NoUnlock = Stop with start wait Unlock = Stop with release and start wait toMinQuote bool Type of parking false = no park at minimum quota true = park at minimum quota CreatePark parameters

The example below creates a machine parking nc function.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
CreatePark("NC function", "Nothing", false);
```

3.10.3 WorkPiece probing

Defined with instruction CreateWorkPieceProbing. The instruction creates the workpiece probing operation.

Name Type Description name string NC function Name. X double Probing X quota Y double Probing Y quota tool string Tool for probing head string Probing head type int (opt def 0) Type of probing 0 = All faces 1 = Point headReturnAfterPro int (opt def 0) Return mode after probing: bing 0 = Parking quota 1 = Probing quota 2 = Skimming quota CreateWorkPieceProbing parameters

The example below creates a workpiece probing nc function.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateWorkPieceProbing("NC function", 10, 10, "Probing tool", "3");
```

3.10.4 Clean table

Defined with instruction CreateCleanPlaneHood. The instruction creates the workpiece cleaning operation with hood.

Name Type Description name string NC function Name. cycleSpeed double (opt def 25) Clean cycle speed cleaningType int (opt def 0) Type of cleaning 0 = None 1 = Workpiece 2 = Spoilboard startX double (opt def null) X coordinate of start cleaning point relative to workpiece origin. endX double (opt def null) Y coordinate of start cleaning point relative to workpiece origin. hoodPosition double (opt def null) Hood position cleanAllPlane bool (def opt true) Clean mode, true to clean all table, false to clean a part of table. CreateCleanPlaneHood parameters

The example below creates a workpiece cleaning nc function.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateCleanPlaneHood("NC function", 25, 0, 100, 100);
```

3.10.5 Tool length probing

Defined with instruction CreatePrismaProbing. The instruction creates the tool length probing to correct the tool length.

Name Type Description name string NC function Name. X object Probing X quota. Y object Probing Y quota. tool string Tool to correct. speed object (opt def null) Probing speed (optional) CreatePrismaProbing parameters

The example below creates a tool length probing nc function to probe the tool "Fresa D20" length.

```
CreateFinishedWorkpieceBox("Pezzo",1600,1200,18);
SelectWorkplane("Top");
CreatePrismaProbing("NC function", 100, 100, "Fresa D20");
```

3.10.6 WorkPiece translation and rotation probing

Defined with instruction CreateWorkPieceTranslationRotationProbing. The instruction creates the probing of workpiece translation and rotation. .

Name Type Description name string NC function Name. probingXFirstPointQuote object First probing point X probingXSecondPointQuote object Second probing point X probingYPointQuote object Probing Y position probingZQuote object Probing Z position tool string Tool for probing Head String Head to use CreateWorkPieceTranslationRotationProbing parameters

The example below creates the probing of workpiece translation and rotation

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateWorkPieceTranslationRotationProbing("NC function", 100, 100, 10, 10, "Probing tool");
```

3.10.7 User message

Defined with instruction CreateMessage. The instruction creates a message for the user displayed on the machine panel. .

Name Type Description

name string NC function name. text string Text displayed for the operator

```
waitForUserInput bool = true Wait for operator
releaseWorkpiece bool = true workpiece release request
```

CreateMessage parameters

The example below creates a message for the user.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateMessage("NC function", "Workpiece has to be overturned", true, true);
```

3.10.8 Null operation

Defined with instruction CreateNullOperation. The instruction creates a special operation that can be used to abort some active

```
operation (head lift up, head park).
```

Name Type Description name string NC function name. X double (opt def null) Parking X quota (optional) Y double (opt def null) Parking Y quota (optional) Q double (opt def null) Quotas reference: (optional) 0 = Quotas referred to machine zero 1 = Quotas referred to workpiece zero speed double (opt def 0) Movement speed (optional) spindleEnable bool (opt def false) Spindle state (optional) false = Spindle off true = Spindle on Tool string (opt def "-1") Tool to fit (optional) Head string (opt def "-1") Head (optional) CreateNullOperation parameters

The example below creates a null operation.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateNullOperation("NC function", 100, 200, 0, 0, true, "Fresa D20");
```

3.10.9 Iso operation

Defined with instruction CreateIso. The instruction creates a Iso instruction. .

Name Type Description name string NC function name. Instruction String ISO instruction to execute optionalParamete String Optional parameters to the ISO instruction rs

isXiso bool Xiso instruction false = it's not a Xiso instruction true = it is a Xiso instruction CreateIso Parameters

The example below creates a Iso instruction

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateIso("G0 X0 Y0","G0 X0 Y0","", true);
```

3.10.10 CombiFlex unload

Defined with instruction CreateWorkPieceWidthProbing. The instruction creates CombiFlex unload. .

Nome Tipo Descrizione name string NC function name. zQuote double Unload quote along Z axis. yOffset double Offset along Y axis. CreateWorkPieceWidthProbing parameters

The example below creates CombiFlex unload

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateWorkPieceWidthProbing("NC function", 10, 10);
```

3.10.11 Check Y dimension

Defined with instruction CreateCombiflexUnloadUnitClamp. The instruction creates CombiFlex unload. .

Name Type Description name string NC function name. distance double Y distance where check dimension CreateWorkPieceWidthProbingparameters

The example below creates CombiFlex unload

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
CreateWorkPieceWidthProbing("NC function", 10);
```

3.10.12 Workpiece center

Defined with instruction CreateCenterWorkpiece.

The instruction can be used to place the workpiece on the machine table with a robot loader. .

Name Type Description name string NC function name. referencePointX double X picking coordinate to center the workpiece. referencePointY double Y picking coordinate to center the workpiece. CreateCenterWorkpiece parameters

The example below creates a workpiece centering

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
//Workpiece alignment
CreateCenterWorkpiece("Workpiece alignment", 900, 400);
```

3.10.13 Workpiece lifting

Defined with instruction CreateLiftWorkpiece. The instruction can be used to lift the workpiece from the machine table with a robot loader. .

Name Type Description name string NC function name. overturnWorkpiece bool true to overturn the workpiece, otherwise false referencePointX double X picking coordinate to center the workpiece. referencePointY double Y picking coordinate to center the workpiece. CreateLiftWorkpiece parameters

The example below creates a workpiece lifting

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
//Workpiece lifting
CreateLiftWorkpiece("Workpiece lifting", true, 900, 400);
```

3.10.14 Tool integrity check

Defined with instruction CreateToolIntegrityCheck. The instruction can be used to check the tool integrity and must be placed at the end of the program.

Name Type Description name string Nc function name. tool string Name of the tool to check

head string (def -1) Head number to execute the check.(Optional) CreateToolIntegrityCheck parameters

The example below creates the integrity tool check for a tool named "Fresa D20" with automatic head selection

```
CreateFinishedWorkpieceBox("Workpiece",1600,1200,18);
SelectWorkplane("Top");
//Tool integrity check
CreateToolIntegrityCheck("Tool integrity check", "-1", "Fresa D20");
```

3.11 Workpiece Blocking

Workpiece blocking defines the layout of blocking devices used to block the workpiece on the machine table and could be divided into supports blocking or suction cups blocking.Type of disposition and blocking is related to machine table type and to blocking devices used.

3.11.1 Blocking profile

Support blocking needs the definition of a blocking profile, a profile used to indicate where is possible to put supports. The blocking profile defines which side or sides are useful to block a specific workpiece. It must be used in the case of calling the automatic blocking disposition. Blocking profile must be created on each workplan inside the project to allow the automatic support disposition.

3.11.1.1 Blocking profile creation

Defined with instruction CreateBlockingProfile. The instruction creates a blocking profile with start point and defines the minimum clamping distance for support.

Name Type Description name string Name of the blocking profile startX double Coordinate X of the first blocking profile point. startY double Coordinate Y of the first blocking profile point. offset double Minimum clamping expressed with negative values.

CreateBlockingProfile parameters

The example below creates a blocking profile in the Setup workplan with start point and a minimum clamping value 5. This profile is not consistent, it is just the first point to insert segment and arcs.

```
CreateFinishedWorkpieceBox("FinishedWorkPiece",1200,70,70);
CreateRawWorkpiece("RawWorkPiece",10,10,2,0,0,0);
//Blocking profile creation
CreateBlockingProfile("SetupBlockingProfile", 1210, 70,-5);
```

3.11.1.2 Add to blocking profile a segment

Defined with instruction AddSegmentToBlockingProfile. The instruction adds a segment to the active blocking profile with specified end point.

Name Type Description endX double Coordinate X of the final blocking profile point. endY double Coordinate Y of the final blocking profile point. nameElement string Name of added element

AddSegmentToBlockingProfile parameters

The example below creates a segment blocking profile in the Setup workplan on the back side of the workpiece with a minimum clamping value 5.

```
CreateFinishedWorkpieceBox("FinishedWorkPiece",1200,70,70);
CreateRawWorkpiece("RawWorkPiece",10,10,2,0,0,0);
//Blocking profile creation
CreateBlockingProfile("SetupBlockingProfile", 1210, 70,-5);
AddSegmentToBlockingProfile(-10, 70);
```

3.11.1.3 Add to blocking profile an arc by 3 points

Defined with instruction AddArc3PointsToBlockingProfile. The instruction adds an arc by 3 points to the active blocking profile.

Name Type Description p2X double Coordinate X of point 2 p2Y double Coordinate Y of point 2 p3X double Coordinate X of point 3 p3Y double Coordinate Y of point 3 nameElement string Name of added element AddArc3PointsToBlockingProfile parameters

3.11.1.4 Add to blocking profile an arc by 2 points and center

Defined with instruction AddArc2PointCenterToBlockingProfile.

The instruction adds an arc by 2 points and center to the active blocking profile.

Name Type Description endX double Coordinate X of final point endY double Coordinate Y of final point centerX double Coordinate X of centre point centerY double Coordinate Y of centre point isClockwise bool Clockwise direction nameElement string Name of added element AddArc2PointCenterToBlockingProfile parameters

3.11.1.5 Add to blocking profile an arc by 2 points and radius

Defined with instruction AddArc2PointRadiusToBlockingProfile. The instruction adds an arc by 2 points and radius the active blocking profile.

Name Type Description endX double Coordinate X of final point endY double Coordinate Y of final point radius double Arc radius isClockwise bool Clockwise direction isOver180 bool Arc angle greater than 180° nameElement string Name of added element AddArc2PointRadiusToBlockingProfile parameters

The example below creates an arc blocking profile in the Setup workplan.

```
CreatePolyline("WorkPieceGeometry",1000,1000);
AddArc2PointRadiusToPolyline(1000,0,566.667,true,false);
AddSegmentToPolyline(0,0);
AddSegmentToPolyline(0,1000);
AddSegmentToPolyline(1000,1000);
CreateFinishedWorkpieceFromExtrusion("FinishedWorkPiece",68,"");
//Blocking profile creation
CreateBlockingProfile("SetupBlockingProfile",979.1878,920,-42);
AddArc2PointRadiusToBlockingProfile(979.1878,80,486.667,true,false, "");
AddSegmentToBlockingProfile(80.,80, "");
AddSegmentToBlockingProfile(80,920, "");
AddSegmentToBlockingProfile(979.187,920, "");
```

3.11.1.6 Add to blocking profile an arc by center and angle

Defined with instruction AddArcCenterAngleToBlockingProfile. The instruction adds an arc by center and angle to the active blocking profile.

Name Type Description centerX double Coordinate X of centre point centerY double Coordinate Y of centre point angle double Arc angle nameElement string Name of added element AddArcCenterAngleToBlockingProfile parameters

3.11.1.7 Add to blocking profile a tangent segment

Defined with instruction AddArcCenterAngleToBlockingProfile. The instruction adds a tangent segment to the active blocking profile.

Name Type Description length double Segment length nameElement string Name of added element AddSegmentTanToBlockingProfile parameters

3.11.1.8 Add to blocking profile a tangent arc

Defined with instruction AddArcTanToBlockingProfile. The instruction adds a tangent arc to the active blocking profile.

Name Type Description endX double Coordinate X of final point endY double Coordinate Y of final point option int The following values are allowed: 0 = direct tangent; 1 = inverse tangent; 2 = clockwise arc; 3 = counterclockwise arc. nameElement string Name of added element AddArcTanToBlockingProfile parameters

3.11.1.9 Blocking profile closing

Defined with instruction CloseBlockingProfile. The instruction closes with a segment the active blocking profile.

Name Type Description nameElement string Name of added element CloseBlockingProfile parameters

3.11.2 Blocking devices positioning

Blocking devices positioning could be done referring to workpiece origin or referring to working field origin.

3.11.2.1 Positioning reference definition

Defined with instruction `ConfigFixtureSetupPositioningMode`. The instruction lets you specify if blocking devices are to be placed referring to workpiece origin or referring the working field origin. By default blocking devices are referred to workpiece origin.

Name Type Description
mode int Reference mode : 0 standard, referred to the active workpiece 1 absolute coordinates referred to working field origin

`ConfigFixtureSetupPositioningMode` parameters

The example below shows the positioning of bars referring to workpiece origin

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SetWorkpieceSetupPosition(100,0,0,0);
//Bar 1 at position 100 from workpiece origin
SetBarPosition(1, 100);
//Bar 2 at position 200 from workpiece origin
SetBarPosition(2, 200);
//Bar 3 at position 300 from workpiece origin
SetBarPosition(3, 300);
```

The example below shows the same previous positioning of bars referring to working field origin

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SetWorkpieceSetupPosition(100,0,0,0);
ConfigFixtureSetupPositioningMode(1);
//Bar 1 at position 200 from working field origin
SetBarPosition(1, 200);
//Bar 2 at position 300 from working field origin
SetBarPosition(2, 300);
//Bar 1 at position 400 from working field origin
SetBarPosition(3, 400);
```

3.11.2.2 Blocking bar positioning

Defined with instruction `SetBarPosition`. The instruction defines the blocking bar position on the machine table. Bar is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description
id int Position bar id (value between 1 and the max number of bars).

xPosition double Bar position along axis X (referred to the selected reference system for the workpiece being machined).
`SetBarPosition` parameters

The example below set up the bars on machine table referring to workpiece origin

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Bar 2 at position 200 to workpiece origin
SetBarPosition(2, 200);
//Bar 3 at position 300 to workpiece origin
SetBarPosition(3, 300);
```

3.11.2.3 Mobile bar positioning

Defined with instruction `SetMobileBarPosition`. The instruction enables or disables a mobile bar on the machine table. Bar is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description

id int Position bar id (value between 1 and the max : number of bars). position double Bar position : -1 not used 0 OFF 1 ON

`SetMobileBarPosition` parameters

The example below set up on the mobile bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Mobile bar enable
SetMobileBarPosition(2, 1);
```

3.11.2.4 Support base positioning

Defined with instruction `SetSupportBasePosition`. The instruction set up along the active blocking bar a support base to host blocking devices. Support base is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description
id int Support base id (value between 1 and the max number of support bases). yPosition double Support base position along axis Y (referred to the selected reference system for the workpiece being machined).

`SetSupportBasePosition` parameters

The example below set up the support base on bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Support base 1
SetSupportBasePosition(1, 100);
//Support base 2
SetSupportBasePosition(2, 200);
//Support base 3
SetSupportBasePosition(3, 300);
```

3.11.2.5 Clamp positioning

Defined with instruction `SetClampPosition`. The instruction sets up a clamp along the active blocking bar and defines the blocking status. Clamp is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description supportId int Clamp id (if the table is motor driven: identifies the support base where the clamp must be fitted or moved if the table is manual: identifies if there is the clamp or it creates it). yPosition double Clamp position along axis Y (referred to the selected reference system for the workpiece being machined). status int Blocking status: 0 = closed; 1 = open; 2 = closed on the workpiece. code string Clamp code `SetClampPosition` parameters

The example below set up clamps on bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Clamp 1
SetClampPosition(1, 100, 2, "H110-A-145");
//Clamp 2
SetClampPosition(2, 200, 2, "H110-A-145");
//Clamp 3
SetClampPosition(3, 300, 2, "H110-A-145");
```

3.11.2.6 Horizontal clamp positioning

Defined with instruction `SetHorizontalClampPosition`. The instruction sets up a horizontal clamp along the active blocking bar and defines the blocking status. Clamp is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description supportId int Clamp id (if the table is motor driven: identifies the support base where the clamp must be fitted or moved if the table is manual: identifies if there is the clamp or it creates it). yPosition double Clamp position along axis Y (referred to the selected reference system for the workpiece being machined). status int Blocking status: 0 = closed; 1 = open; 2 = closed on the workpiece. code string Clamp code `SetClampPosition` parameters

The example below set up clamps on bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Clamp 1
SetHorizontalClampPosition(1, 100, 2, "H110-MF-150x294");
//Clamp 2
SetHorizontalClampPosition(2, 200, 2, "H110-MF-150x294");
//Clamp 3
SetHorizontalClampPosition(3, 300, 2, "H110-MF-150x294");
```

3.11.2.7 Clamp for jamb positioning

Defined with instruction `SetClampForJambPosition`.

The instruction sets up a clamp for jamb along the active blocking bar and defines the blocking status. Clamp is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description supportId int Clamp id (if the table is motor driven: identifies the support base where the clamp must be fitted or moved if the table is manual: identifies if there is the clamp or it creates it). yPosition double Clamp position along axis Y (referred to the selected reference system for the workpiece being machined). status int Blocking status: 0 = closed; 1 = open; 2 = closed on the workpiece. code string Clamp code `SetClampForJambPosition` parameters

The example below set up clamps for jamb on bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Clamp 1
SetClampForJambPosition(1, 100, 2, "H162-A-145-YM");
//Clamp 2
SetClampForJambPosition(2, 200, 2, "H162-A-145-YM");
//Clamp 3
SetClampForJambPosition(3, 300, 2, "H162-A-145-YM");
```

3.11.2.8 Suction cup positioning

Defined with instruction `SetSuctionCupPosition`. The instruction sets up a suction cup along the active blocking bar and defines the rotation. Suction cup is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description supportId int Suction cup id (if the table is motor driven: identifies the support base where the suction cup must be fitted or moved if the table is manual: identifies the suction cup, if it exists or it creates it). yPosition double Suction cup position along axis Y (referred to the selected reference system for the workpiece being machined). angle double Suction cup rotation angle (if it supports rotation). code string Suction cup code SetSuctionCupPosition parameters

The example below set up suction cup on bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Suction cup 1
SetSuctionCupPosition(1, 100, 90, "H75-M-145x55");
//Suction cup 2
SetSuctionCupPosition(2, 200, 90, "H75-M-145x55");
//Suction cup 3
SetSuctionCupPosition(3, 300, 90, "H75-M-145x55");
```

3.11.2.9 Hepod suction cup positioning

Defined with instruction SetPopupSuctionCupPosition. The instruction sets up a suction cup along the active blocking bar and defines the rotation. Suction cup is identified by a logical index from 1 to N and grows moving in the direction of the active area's specularly.

Name Type Description supportId int Suction cup id (if the table is motor driven: identifies the support base where the suction cup must be fitted or moved if the table is manual: identifies the suction cup, if it exists or it creates it). yPosition double Suction cup position along axis Y (referred to the selected reference system for the workpiece being machined). angle double Suction cup rotation angle (if it supports rotation). isUp bool true to lift up the suction cup, false to low down the suction cup code string Suction cup code SetPopupSuctionCupPosition parameters

The example below set up the Hepod suction cup on bar

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Bar 1 at position 100 to workpiece origin
SetBarPosition(1, 100);
//Suction cup 1
SetPopupSuctionCupPosition(1, 100, 90, true, "H75_165-M-145x230-XM");
//Suction 2
SetPopupSuctionCupPosition(2, 200, 90, true, "H75_165-M-145x230-XM");
//Suction 3
SetPopupSuctionCupPosition(3, 300, 90, true, "H75_165-M-145x230-XM");
```

3.11.2.10 Suction cup positioning on multi functional table

Defined with instruction SetMLTFSuctionCupPosition. The instruction sets up a suction cup on the multi functional table.

Name Type Description xPosition double Suction cup position along axis X (referred to the selected reference system for the workpiece being machined). yPosition double Suction cup position along axis Y (referred to the selected reference system for the workpiece being machined). angle double Suction cup rotation angle (if it supports rotation). code string Suction cup code SetMLTFSuctionCupPosition parameters

The example below set up suction cup on the multi functional table

```
SetMachiningParameters("A", 1, 0, 0, false);
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Suction cup 1
SetMLTFSuctionCupPosition(100, 100, 90, "H25-MF-S-90x90");
//Suction cup 2
SetMLTFSuctionCupPosition(200, 200, 90, "H25-MF-S-90x90");
//Suction cup 3
SetMLTFSuctionCupPosition(300, 300, 90, "H25-MF-S-90x90");
```

3.12 Mathematical functions

In a MSL project the following mathematical functions can be used.

Function Description and type of return double abs(double value) Returns the absolute value of a specific value. double ABS(double value) double acos(double value) Returns the arc-cosine of a specific value. double ACOS(double value) double asin(double value) Returns the arc-sine of a specific value. double ASIN(double value) double atan(double value) Returns the arc-tangent of a specific value. double ATAN(double value) double sin(double value) Returns the sine of a specific value. double SIN(double value) double cos(double value) Returns the cosine of a specific value. double COS(double value) double tan(double value) Returns the tangent of a specific value. double TAN(double value) double ln(double value) Returns the logarithm of a specific value. double LN(double value) double log10(double value) Returns the logarithm to base ten of a specific value. double LOG10(double value)

double exp(double value) Returns the exponential of a specific value. double EXP(double value) double sqrt(double value) Returns the square root of a specific value. double SQRT(double value) double pow(double x, double y) Returns the elevation to base power X and exponent Y. double POW(double x, double y) double rd(double value) Returns the rounding off by defect for a specific double RD(double value) decimal value. double ru(double value) Returns the rounding off by excess for a specific double RU(double value) decimal value.

3.13 Additional functions

In a MSL project the following additional functions can be used.

Function Parameters Description and type of return string
GetVersion() None Reads the Maestro version in execution and returns it as string
Int GetProgrammingOrigine() No uno Returns an integer value that identified the programming origin: 0 = SCM 1 = Morbidelli
void Print(string message) Message to print string. Used to print an error message. (if the string starts with "@" the parameter is the identification key of the message to print).
bool NDEF(double? value) Double type test variable. Used to check if the variable contains a valid value. Returns true if the variable does not contain a valid value.
bool DEF(double? value) Double type test variable. Used to check if the variable contains a valid value. Returns true if the variable contains a valid value.
bool NDEF(string value) String type test variable. Used to check if the variable contains a valid value. Returns true if the variable does not contain a valid value.
bool DEF(string value) String type test variable. Used to check if the variable contains a valid value. Returns true if the variable contains a valid value.

double V(double? par) Double type variable Returns the value in the variable (if the variable is not defined it returns zero).

double VALUE(double? par) Double type variable Returns the value in the variable (if the variable is not defined it returns zero).

4 Tools In a MSL project it is possible to get tool properties.

Defined with instruction GetTool. The instruction returns a Tool object containing tool properties.

Name Type Description name string Name to identify tool GetTool parameters

Available tool properties are :

Nome Tipo Descrizione Description string Tool description TotalLength double Tool overall assembly length PilotLength double Tool pilot length CutterLength double Tool sinking length TotalDiameter double Tool overall assembly diameter CuttingDiameter double Tool overall cutting diameter Radius double Tool radius BladeThickness double Tool thickness (only for saw tool) MortiserWidth double Mortising chain length (only for mortising tool) MortiserThickness double Mortising chain thickness (only for mortising tool) MortiserRoundLength double Mortising chain round length (only for mortising tool) MaxSpindleSpeed double Maximum tool rotation speed MaxFeedSpeed double Maximum tool feed StandardFeedSpeed double Standard tool feed ToolFamily int Tool family 0- Not set 1- Drill tool 2- Mill tool 3- Saw tool 4- Probe tool 5- Pressure tool 6- Generic tool 7- Mortising tool

Tool properties

The example below shows how to get tool properties.

```
CreateFinishedWorkpieceBox("WorkPiece",620,800,18);
SelectWorkplane("Top");
//Get of tool Fresa D20
Tool tool = GetTool("Fresa D20");
//Check tool existence
if(tool != null)

{
    //Check mill tool
    int toolFamily = tool.ToolFamily;
    if(toolFamily == 2)

    {
        //Get tool radius
        double radius = tool.Radius;
    }
}
```

5 Edge banding MSL lets you create the whole edge banding process using technologies. Technologies must be created in the way each edge banding has to be associated with all joined machinings and this is done defining colors in technology creation. For example edge banding and end trimming must have the same color because they are directly joined.

Technology application process is made by the definition of geometries where to apply technology, technology selection and indication of geometry to apply and for each geometry the definition of edge to apply.

5.1 Technology selection

Defined with instruction SelectTechnology. The instruction selects the technology to be applied on geometry. Geometry can be one and in this case the same edge is applied on each geometry element, or can be expressed as a set of joined elements and in this case is possible to apply different edge on each element. .

Name Type Description name string Name of technology to apply. geometryNames string[] Names of geometries where to apply the edge SelectTechnology parameters

The example below selects a technology named "EdgeBanding.tchx" to be applied on rectangle sides.

```
CreateFinishedWorkpieceBox("WorkPiece",620,800,18);
SelectWorkplane("Top");
//Rectangle defined by 4 independent sides
CreateSegment("Top", 0, 800, 620, 800);
CreateSegment("Bottom", 620, 0,0, 0);
CreateSegment("Right", 620, 800, 620, 0);
CreateSegment("Left", 0, 0, 0, 800);
//Selection of technology and definition of geometries
SelectTechnology("EdgeBanding", "Top", "Bottom", "Right", "Left");
```

5.2 Edge selection for geometry

Defined with instruction `SelectGeometryTechnology`. The instruction selects for the geometry the edge to apply. Edge selection identify edge banding to apply to geometry and all joined machinings. Machinings are identified from color. .

Name Type Description name string Name of technology to apply. r int Red color component for machining inside the technology g int Green color component for machining inside the technology b int Blu color component for machining inside the technology `SelectGeometryTechnology` parameters

The example below applies a technology named "EdgeBanding.tchx" on rectangle sides. The rectangle is defined by 4 joined segments (not a polyline). On the side "Top", "Right" is applied the same edge, on the "Left" a different edge. If presents inside the technology, for each selected side all joined machinings will be

```
applied (for example end trimming).
CreateFinishedWorkpieceBox("WorkPiece",620,800,18);
SelectWorkplane("Top");
//Rectangle defined by 4 independent sides
CreateSegment("Top", 0, 800, 620, 800);
CreateSegment("Bottom", 620, 0,0, 0);
CreateSegment("Right", 620, 800, 620, 0);
CreateSegment("Left", 0, 0, 0, 800);
//Selection of technology and definition of geometries
SelectTechnology("EdgeBanding", "Top", "Bottom", "Right" , "Left");
//Selection of edge to apply on each geometry
SelectGeometryTechnology("Top", 255, 0, 0);//Red
SelectGeometryTechnology("Right", 255, 0, 0);//Red
SelectGeometryTechnology("Left", 255, 255, 0);//Yellow
```

5.3 Technology application

Defined with instruction `ApplyTechnology`. The instruction creates an edge banding from a technology process.

The example below creates an edge banding process from a technology.

```
CreateFinishedWorkpieceBox("WorkPiece",620,800,18);
SelectWorkplane("Top");
//Rectangle defined by 4 independent sides
CreateSegment("Top", 0, 800, 620, 800);
CreateSegment("Bottom", 620, 0,0, 0);
CreateSegment("Right", 620, 800, 620, 0);
CreateSegment("Left", 0, 0, 0, 800);
//Selection of technology and definition of geometries
SelectTechnology("EdgeBanding", "Top", "Bottom", "Right" , "Left");
//Selection of edge to apply on each geometry
SelectGeometryTechnology("Top", 255, 0, 0);//Red
SelectGeometryTechnology("Right", 255, 0, 0);//Red
SelectGeometryTechnology("Left", 255, 255, 0);//Yellow
//Technology application
ApplyTechnology();
```

5.4 Full Examples

The example below creates an edge banding process on a rectangle defined by 4 independent sides process. On side "Top" and "Right" is applied the edge corresponding to red color, and on side "Left" the edge corresponding to yellow color. Side "Bottom" is not edged.

```
CreateFinishedWorkpieceBox("WorkPiece",620,800,18);
SelectWorkplane("Top");
//Rectangle defined by 4 independent sides
CreateSegment("Top", 0, 800, 620, 800);
CreateSegment("Bottom", 620, 0,0, 0);
CreateSegment("Right", 620, 800, 620, 0);
CreateSegment("Left", 0, 0, 0, 800);
//Selection of technology and definition of geometries
SelectTechnology("EdgeBanding", "Top", "Bottom", "Right" , "Left");
//Selection of edge to apply on each geometry
SelectGeometryTechnology("Top", 255, 0, 0);//Red
SelectGeometryTechnology("Right", 255, 0, 0);//Red
SelectGeometryTechnology("Left", 255, 255, 0);//Yellow
//Technology application
ApplyTechnology();
```

The example below creates an edge banding process on a rectangle defined by 4 independent sides process. On side "Top" and "Right" is applied the edge corresponding to red color, and on side "Left" and "Bottom" the edge corresponding to yellow color.

```
CreateFinishedWorkpieceBox("WorkPiece",620,800,18);
SelectWorkplane("Top");
//Rectangle defined by 4 independent sides
CreateSegment("Top", 0, 800, 620, 800);
CreateSegment("Bottom", 620, 0,0, 0);
CreateSegment("Right", 620, 800, 620, 0);
CreateSegment("Left", 0, 0, 0, 800);
//Selection of technology and definition of geometries
SelectTechnology("EdgeBanding", "Top", "Bottom", "Right" , "Left");
//Selection of edge to apply on each geometry
SelectGeometryTechnology("Top", 255, 0, 0);//Red
SelectGeometryTechnology("Right", 255, 0, 0);//Red
```

```
SelectGeometryTechnology("Left", 255, 255, 0);//Yellow
SelectGeometryTechnology("Bottom", 255, 255, 0);//Yellow
//Technology application
ApplyTechnology();
```

The example below creates an edge banding process on a rectangle defined by a polyline to use the same edge corresponding to red color.

```
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
SelectWorkplane("Top");
//Rectangle defined by a polyline
CreatePolyline("Geometry",0,1200);
AddSegmentToPolyline(1600,1200);
AddSegmentToPolyline(1600,0);
AddSegmentToPolyline(0,0);
ClosePolyline();
//Selection of technology and definition of geometries
SelectTechnology("EdgeBanding", "Geometry");
//Selection of edge to apply on each geometry
SelectGeometryTechnology("Geometry", 255, 0, 0);//Red
//Technology application
ApplyTechnology();
```

6 Labeling In MSL it is possible to create a project applying a label to a workpiece. Labeling needs the global definition of label name to use and for each workpiece position and rotation of label.

6.1 Label definition

Defined with instruction SetProjectLabel. The instruction defines the label name to be used in the project.

Name Type Description labelName string Label name to be used in the project.

SetProjectLabel parameters

The example below defines the label to be used for workpieces inside the project.

```
SetMachiningParameters("A", 1, 0, 0, false);
SetProjectLabel("Label");
//Workpiece
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
```

6.2 WorkPiece label definition

Defined with instruction SetWorkPieceLabel. The instruction places the project label on the active workpiece.

Name Type Description xStart double X position of label referred to workpiece origin. yStart double Y position of label referred to workpiece origin. rotationDegree double Label rotation in degrees.

SetWorkPieceLabel parameters

The example below defines the label position on the active workpiece

```
SetMachiningParameters("A", 1, 0, 0, false);
SetProjectLabel("Label");
//Workpiece
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Label positioning
SetWorkPieceLabel(200, 200, 0);
```

6.3 Label parameters definition

Defined with instruction AddLabelParams.

The instruction adds label parameter to active workpiece. Parameter must respect the number of available parameters and their order. If a parameter is not necessary pass empty.

Name Type Description labelParameters string[] Label parameters.

AddLabelParams parameters

The example below defines the label position on the active workpiece

```
SetMachiningParameters("A", 1, 0, 0, false);
SetProjectLabel("Label");
//Workpiece
CreateFinishedWorkpieceBox("WorkPiece",1600,1200,18);
//Label positioning
SetWorkPieceLabel(200, 200, 0);
//Workpiece label parameters
AddLabelParams("A", "B", "", "", "C");
```

7 Nesting In MSL it is possible to create a Nesting project that defines a cutting scheme and all labeling information In the project it is possible to define nested parts, remnants to recover and scraps to remove. Parts and remnants can be labeled. Parts, remnants and scrap can have associated machinings.

7.1 Part

Part is a closed profile inside the sheet that identifies a workpiece.

7.1.1 Part creation

Part creation needs the definition of process beginning, the definition of the part with outer boundary and internal holes boundary, the process ending.

Process beginning is defined by the instruction BeginPartCreation. Process ending is defined by the instruction EndPartCreation. Process boundary and hole boundaries creation is defined by the instruction CreatePart.

Name Type Description name string Name of the part to create. boundaryGeometryNa string Name of the boundary geometry of the part. me holeGeometryNames string[] (opt def Names of hole geometries of the parte null) CreatePart parameters

The example below creates a rectangular part with a circular hole inside.

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
//Part creation beginning
BeginPartCreation();
//Part Boundary
CreatePolyline("PartBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Part hole
CreateCircleCenterRadius("PartHole", 50, 50, 30);
//Part creation with boundary and hole
CreatePart("Part", "PartBoundary", "PartHole");
//Part creation ending
EndPartCreation();
```

7.1.2 Part labeling

A label can be associated with a Part. Label must be previously defined in the sheet.

7.1.2.1 Part label definition

Defined with instruction SetNestingPartLabel. The instruction places the label inside the part.

Name Type Description x double Label X position referred to part origin

y double Label Y position referred to part origin

angle double Label rotation in degree

SetNestingPartLabel parameters

The example below creates a rectangular part with a circular hole inside and label placed at (50,50)

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
SetProjectLabel("Label");
//Part creation beginning
BeginPartCreation();
//Part Boundary
CreatePolyline("PartBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Part hole
CreateCircleCenterRadius("PartHole", 50, 50, 30);
//Part creation with boundary and hole
CreatePart("Part", "PartBoundary", "PartHole");
//Etichetta associata alla parte
SetNestingPartLabel(50, 50, 0);
//Part creation ending
EndPartCreation();
```

7.1.2.2 Part label parameters definition

Defined with instruction AddNestingPartLabelParameter. The instruction set the label parameter to the active part. Parameter is positional if not necessary, set empty value.

Name Type Description labelParameter string Label parameter.

AddNestingPartLabelParameter parameters

The example below creates a rectangular part with a circular hole inside and label placed at (50,50) and 4 label parameters.

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
SetProjectLabel("Label");
//Part creation beginning
BeginPartCreation();
//Part Boundary
CreatePolyline("PartBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
```

```

AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Part hole
CreateCircleCenterRadius("PartHole", 50, 50, 30);
//Part creation with boundary and hole
CreatePart("Part", "PartBoundary", "PartHole");
//Part label
SetNestingPartLabel(130, 100, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("D");
//Part creation ending
EndPartCreation();

```

7.1.3 Part selection

Defined with instruction `SelectPart`. The instruction selects the specified part as the active part.

Name Type Description name string Name of the part to select.

`SelectPart` parameters

The example below creates a part named "Part1" and a part named "Part2" and selects "Part1"

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet", 3000, 1200, 18);
//Part 1 creation beginning
BeginPartCreation();
//Part 1 Boundary
CreatePolyline("Part1Boundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Part 1 creation with boundary
CreatePart("Part1", "Part1Boundary");
//Part 1 creation ending
EndPartCreation();
//Part 2 creation beginning
BeginPartCreation();
//Part 2 Boundary
CreatePolyline("Part2Boundary", 500, 10);
AddSegmentToPolyline(700, 10);
AddSegmentToPolyline(700, 200);
AddSegmentToPolyline(500, 200);
ClosePolyline();
//Part 2 creation with boundary
CreatePart("Part2", "Part2Boundary");
//Part 2 creation ending
EndPartCreation();
//Select part 1
SelectPart("Part1");

```

7.1.4 Part removal from machine table

Defined with instruction `CreateRemovePart`. The instruction removes the active part from the machine table using a robot loader.

Name Type Description name string Name of the NC function.

referencePointX double X picking coordinate to remove part.

referencePointY double Y picking coordinate to remove part. pickingDimensionX double (default null) X part rectangle picking dimension, if null part length will be used. pickingDimensionY double (default null) Y part rectangle picking dimension, if null part length will be used.

`CreateRemovePart` parameters

The instruction below remove the active part from machine table

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet", 3000, 1200, 18);
//Part 1 creation beginning
BeginPartCreation();
//Part 1 Boundary
CreatePolyline("Part1Boundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Part 1 creation with boundary
CreatePart("Part1", "Part1Boundary");
//Part 1 creation ending
EndPartCreation();
//Part 2 creation beginning
BeginPartCreation();

```

```
//Part 2 Boundary
CreatePolyline("Part2Boundary", 500, 10);
AddSegmentToPolyline(700, 10);
AddSegmentToPolyline(700, 200);
AddSegmentToPolyline(500, 200);
ClosePolyline();
//Part 2 creation with boundary
CreatePart("Part2", "Part2Boundary");
//Part 2 creation ending
EndPartCreation();
//Select part 1
SelectPart("Part1");
//Remove part 1
CreateRemovePart("RemovePart1", 100, 100, 100, 100);
```

7.2 Remnant

Remnant is a rectangular closed profile inside the sheet that identifies a portion of material to be recovered and placed in the material store.

7.2.1 Remnant creation

Remnant creation needs the definition of process beginning, the definition of the remnant with outer boundary, the process ending.

Process beginning is defined by the instruction `BeginRemnantCreation`. Process ending is defined by the instruction `EndRemnantCreation`. Process boundary creation is defined by the instruction `CreateRemnant`.

Name Type Description name string Name of the remnant to create. boundaryGeometryNa string Name of the boundary geometry of the remnant. CreateRemnant parameters

The example below creates a rectangular remnant.

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet", 3000, 1200, 18);
//Remnant creation beginning
BeginRemnantCreation();
//Remnant Boundary
CreatePolyline("RemnantBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Remnant creation with boundary
CreateRemnant("Remnant", "RemnantBoundary");
//Remnant creation ending
EndRemnantCreation();
```

7.2.2 Remnant labeling

A label can be associated with a Remnant. Label must be previously defined in the sheet.

7.2.2.1 Remnant label definition

Defined with instruction `SetNestingRemnantLabel`. The instruction places the label inside the remnant.

Name Type Description x double Label X position referred to remnant origin

y double Label Y position referred to remnant origin

angle double Label rotation in degree

`SetNestingRemnantLabel` parameters

The example below creates a rectangular remnant and label placed at (50,50)

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet", 3000, 1200, 18);
SetProjectLabel("Label");
//Remnant creation beginning
BeginRemnantCreation();
//Remnant Boundary
CreatePolyline("RemnantBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Remnant creation with boundary
CreateRemnant("Remnant", "RemnantBoundary");
//Remnant label
SetNestingRemnantLabel(50, 50, 0);
//Remnant creation ending
EndRemnantCreation();
```

7.2.2.2 Remnant label parameters definition

Defined with instruction AddNestingRemnantLabelParameter. The instruction set the label parameter to the active remnant. Parameter is positional if not necessary, set empty value.

Name Type Description labelParameter string Label parameter.

AddNestingRemnantLabelParameter parameters

The example below creates a rectangular remnant and label placed at (50,50) and 4 label parameters.

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
SetProjectLabel("Label");
//Remnant creation beginning
BeginRemnantCreation();
//Remnant Boundary
CreatePolyline("RemnantBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Remnant creation with boundary
CreateRemnant("Remnant", "RemnantBoundary");
//Remnant label
SetNestingPartLabel(50, 50, 0);
//Label parameters
AddNestingRemnantLabelParameter("A");
AddNestingRemnantLabelParameter("B");
AddNestingRemnantLabelParameter("");
AddNestingRemnantLabelParameter("D");
//Remnant creation ending
EndRemnantCreation();
```

7.2.3 Remnant selection

Defined with instruction SelectRemnant. The instruction selects the specified remnant as the active remnant.

Name Type Description name string Name of the remnant to select.

SelectRemnant parameters

The example below creates a remnant named "Remnant1" and a remnant named "Remnant2" and selects "Remnant1"

```
SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
//Remnant 1 creation beginning
BeginRemnantCreation();
//Remnant 1 Boundary
CreatePolyline("Remnant1Boundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Remnant 1 creation with boundary
CreateRemnant("Remnant1", "Remnant1Boundary");
//Remnant 1 creation ending
EndRemnantCreation();
//Remnant 2 creation beginning
BeginRemnantCreation();
//Remnant 2 Boundary
CreatePolyline("Remnant2Boundary", 500, 10);
AddSegmentToPolyline(700, 10);
AddSegmentToPolyline(700, 200);
AddSegmentToPolyline(500, 200);
ClosePolyline();
//Remnant 2 creation with boundary
CreateRemnant("Remnant2", "Remnant2Boundary");
//Remnant 2 creation ending
EndRemnantCreation();
//Select remnant 1
SelectRemnant("Remnant1");
```

7.2.4 Remnant removal from machine table

Defined with instruction CreateRemoveRemnant. The instruction removes the active remnant from the machine table using a robot loader.

Name Type Description name string Name of the NC function.

referencePointX double X picking coordinate to remove remnant.

referencePointY double Y picking coordinate to remove remnant. pickingDimensionX double (default null) X remnant rectangle picking dimension, if null remnant length will be used.

pickingDimensionY double (default null) Y remnant rectangle picking dimension, if null remnant length will be used.

CreateRemoveRemnant parameters

The instruction below remove the active remnant from machine table

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
//Remnant 1 creation beginning
BeginRemnantCreation();
//Remnant 1 Boundary
CreatePolyline("Remnant1Boundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Remnant 1 creation with boundary
CreateRemnant("Remnant1", "Remnant1Boundary");
//Remnant 1 creation ending
EndRemnantCreation();
//Remnant 2 creation beginning
BeginRemnantCreation();
//Remnant 2 Boundary
CreatePolyline("Remnant2Boundary", 500, 10);
AddSegmentToPolyline(700, 10);
AddSegmentToPolyline(700, 200);
AddSegmentToPolyline(500, 200);
ClosePolyline();
//Remnant 2 creation with boundary
CreateRemnant("Remnant2", "Remnant2Boundary");
//Remnant 2 creation ending
EndRemnantCreation();
//Select remnant 1
SelectRemnant("Remnant1");
//Remove remnant 1
CreateRemoveRemnant("RemoveRemnant1", 100, 100, 100, 100);

```

7.3 Scrap

Scrap is a closed profile inside the sheet that identifies a waste portion of material.

7.3.1 Scrap creation

Scrap creation needs the definition of process beginning, the definition of the scrap with outer boundary, the process ending.

Process beginning is defined by the instruction BeginScrapCreation. Process ending is defined by the instruction EndScrapCreation. Process boundary creation is defined by the instruction CreateScrap.

Name Type Description name string Name of the scrap to create. boundaryGeometryNa string Name of the boundary geometry of the scrap. me CreateScrap parameters

The example below creates a rectangular scrap.

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
//Scrap creation beginning
BeginScrapCreation();
//Scrap Boundary
CreatePolyline("ScrapBoundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Scrap creation with boundary
CreateScrap("Scrap", "ScrapBoundary");
//Scrap creation ending
EndScrapCreation();

```

7.3.2 Scrap selection

Defined with instruction SelectScrap. The instruction selects the specified scrap as the active scrap.

Name Type Description name string Name of the scrap to select.

SelectScrap parameters

The example below creates a scrap named "Scrap1" and a scrap named "Scrap2" and selects "Scrap1"

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
//Scrap 1 creation beginning
BeginScrapCreation();
//Scrap 1 Boundary
CreatePolyline("Scrap1Boundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Scrap 1 creation with boundary
CreateScrap("Scrap1", "Scrap1Boundary");
//Scrap 1 creation ending
EndScrapCreation();
//Scrap 2 creation beginning

```

```

BeginScrapCreation();
//Scrap 2 Boundary
CreatePolyline("Scrap2Boundary", 500, 10);
AddSegmentToPolyline(700, 10);
AddSegmentToPolyline(700, 200);
AddSegmentToPolyline(500, 200);
ClosePolyline();
//Scrap 2 creation with boundary
CreateScrap("Scrap2", "Scrap2Boundary");
//Scrap 2 creation ending
EndScrapCreation();
//Select scrap 1
SelectScrap("Scrap1");

```

7.3.3 Scrap removal from machine table

Defined with instruction CreateRemoveScrap. The instruction removes the active scrap from the machine table using a robot loader.

Name Type Description name string Name of the NC function.

referencePointX double X picking coordinate to remove scrap.

referencePointY double Y picking coordinate to remove scrap. pickingDimensionX double (default null) X scrap rectangle picking dimension, if null scrap length will be used. pickingDimensionY double (default null) Y scrap rectangle picking dimension, if null scrap length will be used.

CreateRemoveScrap parameters

The instruction below remove the active Scrap from machine table

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Sheet",3000,1200,18);
//Scrap 1 creation beginning
BeginScrapCreation();
//Scrap 1 Boundary
CreatePolyline("Scrap1Boundary", 10, 10);
AddSegmentToPolyline(200, 10);
AddSegmentToPolyline(200, 200);
AddSegmentToPolyline(10, 200);
ClosePolyline();
//Scrap 1 creation with boundary
CreateScrap("Scrap1", "Scrap1Boundary");
//Scrap 1 creation ending
EndScrapCreation();
//Scrap 2 creation beginning
BeginScrapCreation();
//Scrap 2 Boundary
CreatePolyline("Scrap2Boundary", 500, 10);
AddSegmentToPolyline(700, 10);
AddSegmentToPolyline(700, 200);
AddSegmentToPolyline(500, 200);
ClosePolyline();
//Scrap 2 creation with boundary
CreateScrap("Scrap2", "Scrap2Boundary");
//Scrap 2 creation ending
EndScrapCreation();
//Select scrap 1
SelectScrap("Scrap1");
//Remove scrap 1
CreateRemoveScrap("RemoveScrap1", 100, 100, 100, 100);

```

7.4 Full Examples

The example below creates a cut scheme containing a part with outer profile incision, hole incision, outer profile cut, hole cut, machinings, a remnant with outer profile cut, a scrap with outer profile cut. Parts and remnants are labeled. Machinings are always referred to sheet origin, labels are always referred to parts or remnants origin. In this example, all machinings relative to part, remnant or scrap are created inside part, remnant, scrap block definition.

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Foglio",3000,1200,18);
//Project label
SetProjectLabel("LabelNesting");
//Part creation beginning
BeginPartCreation();
//Part 1 start point
double part1XCoordinate = 10;
double part1YCoordinate = 10;
//Part Boundary
CreatePolyline("Part1Boundary", part1XCoordinate, part1YCoordinate);
AddSegmentToPolyline(part1XCoordinate + 200, part1YCoordinate);
AddSegmentToPolyline(part1XCoordinate + 200, part1YCoordinate + 200);
AddSegmentToPolyline(part1XCoordinate, part1YCoordinate + 200);
ClosePolyline("Part1Boundary");
//Part hole
CreateCircleCenterRadius("Part1Hole", part1XCoordinate + 50, part1YCoordinate + 50, 30);
//Part creation with boundary and hole

```

```

CreatePart("Part1", "Part1Boundary", "Part1Hole");
//Part 1 machining
CreateSegment("Segment1", part1XCoordinate,part1YCoordinate, part1XCoordinate + 100,
part1YCoordinate + 100);
CreateRoughFinish("Segment1Machining",5,"Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1", 2);
//Part 1 outer boundary incision
SelectGeometry("Part1Boundary");
CreateRoughFinish("Part1BoundaryIncisionCut",5,"Part1BoundaryCut",
TypeOfProcess.GeneralRouting ,"E034",-1", 2);
//Part 1 hole boundary incision
SelectGeometry("Part1Hole");
CreateRoughFinish("Part1HoleIncisionCut",5,"Part1HoleCut", TypeOfProcess.GeneralRouting
,"E034",-1", 1);
//Part 1 outer boundary cut
SelectGeometry("Part1Boundary");
CreateRoughFinish("Part1BoundaryCut",18,"Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1", 2);
//Part 1 hole boundary cut
SelectGeometry("Part1Hole");
CreateRoughFinish("Part1HoleCut",18,"Part1HoleCut", TypeOfProcess.GeneralRouting
,"E034",-1", 1);
//Part label
SetNestingPartLabel(50, 50, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("D");
//Part creation ending
EndPartCreation();
//Part creation beginning
BeginPartCreation();
//Part 2 start point
double part2XCoordinate = 500;
double part2YCoordinate = 10;
//Part Boundary
CreatePolyline("Part2Boundary", part2XCoordinate, part2YCoordinate);
AddSegmentToPolyline(part2XCoordinate + 200, part2YCoordinate);
AddSegmentToPolyline(part2XCoordinate + 200, part2YCoordinate + 200);
AddSegmentToPolyline(part2XCoordinate, part2YCoordinate + 200);
ClosePolyline("Part2Boundary");
//Part creation beginning
CreatePart("Part2", "Part2Boundary");
//Part 2 machining
CreateSegment("Segment2", 10,10,100,100);
CreateRoughFinish("Segment2Machining",5,"Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1", 2);
//Part 2 outer boundary cut
SelectGeometry("Part2Boundary");
CreateRoughFinish("Part2BoundaryCut",18,"Part2BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1", 2);
//Part label
SetNestingPartLabel(100, 100, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
//Part creation ending
EndPartCreation();
//Remnant creation beginning
BeginRemnantCreation();
//Remnant Boundary
CreatePolyline("Remnant1Boundary", 800, 0);
AddSegmentToPolyline(3000, 0);
AddSegmentToPolyline(3000, 1200);
AddSegmentToPolyline(800, 1200);
ClosePolyline("Remnant1Boundary");
//Remnant creation with boundary
CreateRemnant("Remnant1", "Remnant1Boundary");
//Remnant outer cut
SelectGeometry("Remnant1Boundary");
CreateRoughFinish("Remnant1BoundaryCut",18,"Remnant1BoundaryCut",
TypeOfProcess.GeneralRouting ,"E034",-1", 2);
//Remnant label
SetNestingRemnantLabel(100, 100, 0);
//Label parameters

```

```

AddNestingPartLabelParameter("F");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("G");
AddNestingPartLabelParameter("");
//Remnant creation ending
EndRemnantCreation();
//Scrap creation beginning
BeginScrapCreation();
//Scrap Boundary
CreatePolyline("Scrap1Boundary", 0, 500);
AddSegmentToPolyline(500, 500);
AddSegmentToPolyline(700, 700);
AddSegmentToPolyline(0, 1200);
ClosePolyline("Scrap1Boundary");
//Scrap creation with boundary
CreateScrap("Scrap1", "Scrap1Boundary");
//Scrap outer cut
SelectGeometry("Scrap1Boundary");
CreateRoughFinish("Scrap1BoundaryCut", 18, "Scrap1BoundaryCut", TypeOfProcess.GeneralRouting

```

```

,"E034",-1", 2);

```

```

//Scrap creation ending
EndScrapCreation();

```

The example below creates a cut scheme containing a part with outer profile incision, hole incision, outer profile cut, hole cut, machinings, a remnant with outer profile cut, a scrap with outer profile cut. Parts and remnants are labeled. Machinings are always referred to sheet origin, labels are always referred to parts or remnants origin. In this example, first, parts, remnants and scraps are created with label information, and after all machinings are created using selection instructions.

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Foglio", 3000, 1200, 18);
//Project label
SetProjectLabel("LabelNesting");
//Part creation beginning
BeginPartCreation();
//Part 1 start point
double part1XCoordinate = 10;
double part1YCoordinate = 10;
//Part Boundary
CreatePolyline("Part1Boundary", part1XCoordinate, part1YCoordinate);
AddSegmentToPolyline(part1XCoordinate + 200, part1YCoordinate);
AddSegmentToPolyline(part1XCoordinate + 200, part1YCoordinate + 200);
AddSegmentToPolyline(part1XCoordinate, part1YCoordinate + 200);
ClosePolyline("Part1Boundary");
//Part hole
CreateCircleCenterRadius("Part1Hole", part1XCoordinate + 50, part1YCoordinate + 50, 30);
//Part creation with boundary and hole
CreatePart("Part1", "Part1Boundary", "Part1Hole");
//Part label
SetNestingPartLabel(50, 50, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("D");
//Part creation ending
EndPartCreation();
//Part creation beginning
BeginPartCreation();
//Part 2 start point
double part2XCoordinate = 500;
double part2YCoordinate = 10;
//Part Boundary
CreatePolyline("Part2Boundary", part2XCoordinate, part2YCoordinate);
AddSegmentToPolyline(part2XCoordinate + 200, part2YCoordinate);
AddSegmentToPolyline(part2XCoordinate + 200, part2YCoordinate + 200);
AddSegmentToPolyline(part2XCoordinate, part2YCoordinate + 200);
ClosePolyline("Part2Boundary");
//Part creation beginning
CreatePart("Part2", "Part2Boundary");
//Part label
SetNestingPartLabel(100, 100, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
//Part creation ending
EndPartCreation();
//Remnant creation beginning
BeginRemnantCreation();
//Remnant Boundary
CreatePolyline("Remnant1Boundary", 800, 0);
AddSegmentToPolyline(3000, 0);
AddSegmentToPolyline(3000, 1200);
AddSegmentToPolyline(800, 1200);
ClosePolyline("Remnant1Boundary");
//Remnant creation with boundary

```

```

CreateRemnant("Remnant1", "Remnant1Boundary");
//Remnant label
SetNestingRemnantLabel(100, 100, 0);
//Label parameters
AddNestingPartLabelParameter("F");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("G");
AddNestingPartLabelParameter("");
//Remnant creation ending
EndRemnantCreation();
//Scrap creation beginning
BeginScrapCreation();
//Scrap Boundary
CreatePolyline("Scrap1Boundary", 0, 500);
AddSegmentToPolyline(500, 500);
AddSegmentToPolyline(700, 700);
AddSegmentToPolyline(0, 1200);
ClosePolyline("Scrap1Boundary");
//Scrap creation with boundary
CreateScrap("Scrap1", "Scrap1Boundary");
//Scrap creation ending
EndScrapCreation();
//Part 1 selection
SelectPart("Part1");
//Part 1 machining
CreateSegment("Segment1", part1XCoordinate, part1YCoordinate, part1XCoordinate + 100,
part1YCoordinate + 100);
CreateRoughFinish("Segment1Machining", 5, "Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1, 2);
//Part 1 outer boundary incision
SelectGeometry("Part1Boundary");
CreateRoughFinish("Part1BoundaryIncisionCut", 5, "Part1BoundaryCut",
TypeOfProcess.GeneralRouting, "E034",-1, 2);
//Part 1 hole boundary incision
SelectGeometry("Part1Hole");
CreateRoughFinish("Part1HoleIncisionCut", 5, "Part1HoleCut", TypeOfProcess.GeneralRouting
,"E034",-1, 1);
//Part 1 outer boundary cut
SelectGeometry("Part1Boundary");
CreateRoughFinish("Part1BoundaryCut", 18, "Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1, 2);
//Part 1 hole boundary cut
SelectGeometry("Part1Hole");
CreateRoughFinish("Part1HoleCut", 18, "Part1HoleCut", TypeOfProcess.GeneralRouting
,"E034",-1, 1);
//Part 2 selection
SelectPart("Part2");
//Part 2 machining
CreateSegment("Segment2", 10, 10, 100, 100);
CreateRoughFinish("Segment2Machining", 5, "Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1, 2);
//Part 2 outer boundary cut
SelectGeometry("Part2Boundary");
CreateRoughFinish("Part2BoundaryCut", 18, "Part2BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1, 2);
//Remnant selection
SelectRemnant("Remnant1");
//Remnant outer cut
SelectGeometry("Remnant1Boundary");
CreateRoughFinish("Remnant1BoundaryCut", 18, "Remnant1BoundaryCut",
TypeOfProcess.GeneralRouting, "E034",-1, 2);
//Scrap selection
SelectScrap("Scrap1");
//Scrap outer cut
SelectGeometry("Scrap1Boundary");
CreateRoughFinish("Scrap1BoundaryCut", 18, "Scrap1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1, 2);

```

The example below creates a cut scheme containing a part with outer profile incision, hole incision, outer profile cut, hole cut, machinings, a remnant with outer profile cut, a scrap with outer profile cut. Parts and remnant are labeled. Machinings are always referred to sheet origin, labels are always referred to parts or remnants origin. In this example, first, parts, remnants and scraps are created with label information, and after all machinings are created using selection instructions to define an ordered machining sequence.

```

SetMachiningParameters("A", 1, 0, 0, false);
//Sheet definition
CreateFinishedWorkpieceBox("Foglio", 3000, 1200, 18);

```

```

//Project label
SetProjectLabel("LabelNesting");
//Part creation beginning
BeginPartCreation();
//Part 1 start point
double part1XCoordinate = 10;
double part1YCoordinate = 10;
//Part Boundary
CreatePolyline("Part1Boundary", part1XCoordinate, part1YCoordinate);
AddSegmentToPolyline(part1XCoordinate + 200, part1YCoordinate);
AddSegmentToPolyline(part1XCoordinate + 200, part1YCoordinate + 200);
AddSegmentToPolyline(part1XCoordinate, part1YCoordinate + 200);
ClosePolyline("Part1Boundary");
//Part hole
CreateCircleCenterRadius("Part1Hole", part1XCoordinate + 50, part1YCoordinate + 50, 30);
//Part creation with boundary and hole
CreatePart("Part1", "Part1Boundary", "Part1Hole");
//Part label
SetNestingPartLabel(50, 50, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("D");
//Part creation ending
EndPartCreation();
//Part creation beginning
BeginPartCreation();
//Part 2 start point
double part2XCoordinate = 500;
double part2YCoordinate = 10;
//Part Boundary
CreatePolyline("Part2Boundary", part2XCoordinate, part2YCoordinate);
AddSegmentToPolyline(part2XCoordinate + 200, part2YCoordinate);
AddSegmentToPolyline(part2XCoordinate + 200, part2YCoordinate + 200);
AddSegmentToPolyline(part2XCoordinate, part2YCoordinate + 200);
ClosePolyline("Part2Boundary");
//Part creation beginning
CreatePart("Part2", "Part2Boundary");
//Part label
SetNestingPartLabel(100, 100, 0);
//Label parameters
AddNestingPartLabelParameter("A");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("B");
AddNestingPartLabelParameter("");
//Part creation ending
EndPartCreation();
//Remnant creation beginning
BeginRemnantCreation();
//Remnant Boundary
CreatePolyline("Remnant1Boundary", 800, 0);
AddSegmentToPolyline(3000, 0);
AddSegmentToPolyline(3000, 1200);
AddSegmentToPolyline(800, 1200);
ClosePolyline("Remnant1Boundary");
//Remnant creation with boundary
CreateRemnant("Remnant1", "Remnant1Boundary");
//Remnant label
SetNestingRemnantLabel(100, 100, 0);
//Label parameters
AddNestingPartLabelParameter("F");
AddNestingPartLabelParameter("");
AddNestingPartLabelParameter("G");
AddNestingPartLabelParameter("");
//Remnant creation ending
EndRemnantCreation();
//Scrap creation beginning
BeginScrapCreation();
//Scrap Boundary
CreatePolyline("Scrap1Boundary", 0, 500);
AddSegmentToPolyline(500, 500);
AddSegmentToPolyline(700, 700);
AddSegmentToPolyline(0, 1200);
ClosePolyline("Scrap1Boundary");
//Scrap creation with boundary
CreateScrap("Scrap1", "Scrap1Boundary");
//Scrap creation ending
EndScrapCreation();
//Part 1 selection
SelectPart("Part1");
//Part 1 machining
CreateSegment("Segment1", part1XCoordinate, part1YCoordinate, part1XCoordinate + 100,
part1YCoordinate + 100);
CreateRoughFinish("Segment1Machining", 5, "Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034", "-1", 2);

```

```
//Part 2 selection
SelectPart("Part2");
//Part 2 machining
CreateSegment("Segment2", 10,10,100,100);
CreateRoughFinish("Segment2Machining",5,"Part1BoundaryCut", TypeOfProcess.GeneralRouting
,"E034",-1", 2);
```

```
//Part 1 selection
SelectPart("Part1");
//Part 1 outer boundary incision
SelectGeometry("Part1Boundary");
CreateRoughFinish("Part1BoundaryIncisionCut",5,"Part1BoundaryCut",
```

```
TypeOfProcess.GeneralRouting ,"E034",-1", 2);
```

```
//Part 1 hole boundary incision
SelectGeometry("Part1Hole");
CreateRoughFinish("Part1HoleIncisionCut",5,"Part1HoleCut", TypeOfProcess.GeneralRouting
```

```
,"E034",-1", 1);
```

```
//Part 1 outer boundary cut
SelectGeometry("Part1Boundary");
CreateRoughFinish("Part1BoundaryCut",18,"Part1BoundaryCut", TypeOfProcess.GeneralRouting
```

```
,"E034",-1", 2);
```

```
//Part 1 hole boundary cut
SelectGeometry("Part1Hole");
CreateRoughFinish("Part1HoleCut",18,"Part1HoleCut", TypeOfProcess.GeneralRouting
```

```
,"E034",-1", 1);
```

```
//Part 2 selection
SelectPart("Part2");
//Part outer cut
SelectGeometry("Part2Boundary");
CreateRoughFinish("Part2BoundaryCut",18,"Part2BoundaryCut", TypeOfProcess.GeneralRouting
```

```
,"E034",-1", 2);
```

```
//Remnant selection
SelectRemnant("Remnant1");
//Remnant outer cut
SelectGeometry("Remnant1Boundary");
CreateRoughFinish("Remnant1BoundaryCut",18,"Remnant1BoundaryCut",
```

```
TypeOfProcess.GeneralRouting ,"E034",-1", 2);
```

```
//Scrap selection
SelectScrap("Scrap1");
//Scrap outer cut
SelectGeometry("Scrap1Boundary");
CreateRoughFinish("Scrap1BoundaryCut",18,"Scrap1BoundaryCut", TypeOfProcess.GeneralRouting
```

```
,"E034",-1", 2);
```

8 XConverter XConverter is an application contained inside Maestro suite used to execute operations by DOS command lines. A good way to invoke the XConverter is to create a batch file (.bat) with inside all DOS commands necessary to execute the operations.

XConverter operations are called “mode” and are identified by a number. Batch file must always have the instruction Call to execute the XConverter with the full path where it is located in the file system. Usually, for 64 bit case Program Files folder, for 32 bit case Program Files x86 folder. Calling instruction usually is :

32 bit -

```
Call "c:\Program files x86\Scm Group\MaestroXConverter.exe" ^
```

64 bit -

```
Call "c:\Program files\Scm Group\MaestroXConverter.exe" ^
```

speciale char ^ must be always put at the line ending and next line must be in a new line.

Nella tabella sottostante sono indicati i comandi che possono essere inseriti all’interno del file batch e possono essere utilizzati nei vari modi disponibili.

Comando Tipo Descrizione

```
-s Silent mode
-m string Working mode
-t string Tools file full path
-q string Tooling file full path (if not defined
```

option file is used)

```
-e string Edges file full path (if not defined option
```

file is used)

```
-i string Input file full path (from 1 to N)
-o string Output file full path (from 1 to N)
-env string Environment file full path
-cfgx string Configuration file full path (if not
```

defined option file is used)

```
-lbsx string Supports file full path (if not defined
```

option file is used)

```
-eplx string Percorso completo del file di layout del
```

piano di lavoro (se non definito viene usato quello delle opzioni)

```
-supp string Nome del supporto di default da montare
```

sulle piastre

```
-ca string Valore del carico agevolato  
-ow Indica se il file di uscita deve essere
```

sovrascritto

```
-vp Verifica del progetto  
-v string Percorso completo del file di variabili
```

d'ambiente di riferimento

XConverter parameters

8.1 Working modes

8.1.1 Mode 0 - Xcs Import

This mode imports xcs files and creates a pgmx file. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 0 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs\P1.xcs" ^  
-o "c:\Outputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 0 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs\P1.xcs" ^  
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 0 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs" ^  
-o "c:\Outputs" ^
```

8.1.2 Mode 1 - WorkPiece Import

This mode creates a new project where all workpieces contained in input pgmx files will be imported. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with 3 input pgmx files explicitly defined and the output file.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 1 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs\P1.pgm" "c:\Inputs\P2.pgm" "c:\Inputs\P3.pgm" ^  
-o "c:\Outputs\P0Out.pgm" ^
```

8.1.3 Mode 2 - Project optimization

This mode optimizes the input project. Optimization parameters used are the optimization parameters defined in optimization options. Input files can be specified with file system path and output file must be specified by file system path.

The example below shows a batch file with 1 input pgmx file and the output optimized file path.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 2 ^  
-i "c:\Inputs\P1.pgm" ^  
-o "c:\Outputs\P10Out.pgm" ^
```

8.1.4 Mode 3 - Xcs import, workpiece import and optimization

This mode creates a new project where all workpieces contained in input xcs files will be imported and optimized. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with 3 input xcs files explicitly defined and the output file.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 3 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs\P1.xcs" "c:\Inputs\P2.xcs" "c:\Inputs\P3.xcs" ^  
-o "c:\Outputs\P0Out.pgm" ^
```

8.1.5 Mode 4 - Xxl import

This mode imports xxl files and creates a pgmx file. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 4 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.xml" ^
-o "c:\Outputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 4 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.xml" ^
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 4 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs" ^
-o "c:\Outputs" ^
```

8.1.6 Mode 5 - Pgm import

This mode imports pgm files and creates a pgmx file. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 5 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 5 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 5 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs" ^
-o "c:\Outputs" ^
```

8.1.7 Mode 10 - Pgm process and Xxl generation

This mode processes a pgmx file, checks it and produces an xxl output file. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 10 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs\P1.xml" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 10 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 10 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs" ^
-o "c:\Outputs" ^
```

8.1.8 Mode 11 - Mixx project creation

This mode creates a new mixx project from a list of projects defined in a csv file. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with csv input files and the output mixx file.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 11 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\Distinta.csv" ^
-o "c:\Outputs\P0ut.mixx" ^
```

The .csv file must contains a set of lines describing pgmx files and associated informations to use to create the Mixx
Lines are composed by a set of elements defined by a bracketed key "[TAG]" separated by simbol "=" from associated value. Field must end with character ";".

Following allowed keys :

[PRG]=string; : Name of file pgmx.
[DX]=decimal value; : Panel length (single)
[DY]=decimal value; : Panel width (single)
[DZ]=decimal value; : Panel depth (single)
[BX]=decimal value; : X coordinate of panel positioning
[BY]=decimal value; : Y coordinate of panel positioning
[BZ]=decimal value; : Z coordinate of panel positioning
[R]=integer value; : Number of program repetitions
[FLD]=string; : Working field
[MES]=string; : Unit ("MM"=millimeter - "IN"=inches)
[OPM]=integer value; : Mechanical options (Xilog T field)
[BLK]=integer value; : Blocking options (Xilog V field)
[ORDER]=string; : Order

Mixx line representation is of this type : "[TAG]=Value":

[PRG]=string; [DX]=value; [DY]=value; [DZ]=value; [BX]=valore; [BY]=value; [BZ]=value; [R]=value; [FLD]=value;
[MES]=string; [OPM]=value; [BLK]=value;[ORDER]=string;

Key "[PRG]", name of PGMX, is mandatory, all other keys are optional. If not defined, default values will be applied.

Line keys sequence is not fixed, keys can assume each position on line.

To create a Mixx is necessary to have a folder with inside a set of informations. Folder must contains csv file describing pgmx files to import and the list of pgmx files.

Two cases are possible

1) Pgmx files at same csv level inside folder :

For example, with a folder called "Project :

Project\List.csv
Project\P1.pgmx
Project\P2.pgmx
Project\P3.pgmx

The Tag [PRG] inside each line of the csv must have pgmx file local to folder, example

[PRG]=P1.pgmx;
[PRG]=P2.pgmx;
[PRG]=P3.pgmx;

2) Pgmx files can be in sub folder:

For example, with a folder called "Project :

Project\List.csv
Project\PgmxFolder1\P1.pgmx
Project\PgmxFolder1\P2.pgmx
Project\PgmxFolder1\P3.pgmx
Project\PgmxFolder2\P1.pgmx
Project\PgmxFolder2\P2.pgmx
Project\PgmxFolder2\P3.pgmx

The Tag [PRG] inside each line of the csv must have pgmx file local to sub folder, example

[PRG]=PgmxFolder1\P1.pgmx
[PRG]=PgmxFolder1\P2.pgmx
[PRG]=PgmxFolder1\P3.pgmx
[PRG]=PgmxFolder2\P1.pgmx
[PRG]=PgmxFolder2\P2.pgmx
[PRG]=PgmxFolder2\P3.pgmx

8.1.9 Mode 12 - Automatic clamps setup

This mode creates a new project where all workpieces contained in input xcs files will be imported, optimized, automatically setup executed, generated initial e/o final setup and cycles generated. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with 2 input xcs files explicitly defined and the output file.

Easy loading values can be specified with ca command and it is possible to check project with v command.

The example below shows a batch file with 2 input xcs files and the generated output file

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 12 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.xcs" "c:\Inputs\P2.xcs" ^
-o "c:\Outputs\P0ut.pgm" ^
```

The example below shows a batch file with 2 input xcs files with easy loading values and the generated output file

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 12 ^
-ca 200 200 400 400 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.xcs" "c:\Inputs\P2.xcs" ^
-o "c:\Outputs\P0ut.pgm" ^
```

8.1.10 Mode 13 - Automatic suction cups setup

This mode executes the suction cups automatic set up on input files. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with pgmx input files to execute the automatic suction cup set up and the output mixx file.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 13 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs\P0ut.pgm" ^
```

8.1.11 Mode 14 - Import Pgm and automatic suction cups setup

This mode imports a pgm and executes the suction cups automatic set up. Tools file (.tlgx) must be explicitly indicated.

Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with pgm input files to execute the automatic suction cup set up and the output mixx file.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 14 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs\P0ut.pgm" ^
```

This mode imports a pgm and executes the suction cups automatic set up. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output file must be specified.

The example below shows a batch file with pgm input files to execute the automatic suction cup set up and the output mixx file.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 14 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Outputs\P0ut.pgm" ^
```

8.1.12 Mode 16 - Project time execution estimation

This mode executes the project time execution estimation on input files (pgmx, mixx) and produces for each of them a csv file containing results.

The example below shows a batch file with pgmx input file to execute the time execution estimation.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 16 ^
-i "c:\Inputs\P1.pgm" ^
-o "c:\Output\P1.csv" ^
```

8.1.13 Mode 18 - Advanced Dxf import

This mode imports advanced dxf files and creates a pgmx file. Tools file (.tlgx) must be explicitly indicated. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 18 ^
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^
```

```
-i "c:\Inputs\P1.dxf" ^  
-o "c:\Outputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 18 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs\P1.dxf" ^  
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 18 ^  
-t "c:\Program Files (x86)\Scm Group\Maestro\Tlgx\Tools.tlgx" ^  
-i "c:\Inputs" ^  
-o "c:\Outputs" ^
```

8.1.14 Mode 20 - Project check

This mode checks a pgmx file and produces the check results as output. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 20 ^  
-i "c:\Inputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 20 ^  
-i "c:\Inputs\P1.pgm" ^  
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 20 ^  
-i "c:\Inputs" ^  
-o "c:\Outputs" ^
```

8.1.15 Mode 21 - Build and check project to load on the machine

This mode processes a pgmx file, checks machine loading and produces the check results as output. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 21 ^  
-i "c:\Inputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 21 ^  
-i "c:\Inputs\P1.pgm" ^  
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 21 ^  
-i "c:\Inputs" ^  
-o "c:\Outputs" ^
```

8.1.16 Mode 22 - Build project to load on the machine

This mode processes a pgmx file, to load it on the machine. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 22 ^  
-i "c:\Inputs\P1.pgm" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^  
-s -m 22 ^  
-i "c:\Inputs\P1.pgm" ^  
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 22 ^
-i "c:\Inputs" ^
-o "c:\Outputs" ^
```

8.1.17 Mode 23 - Xcs import build project to load on machine

This mode imports a xcs file, processes it, checks machine loading and produces the check results as output. Input files can be specified with file system path and output files must be specified with file system path or with an output folder. Input files can be specified with an input folder and an output folder.

The example below shows a batch file with input and output explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 23 ^
-i "c:\Inputs\P1.xcs" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 23 ^
-i "c:\Inputs\P1.xcs" ^
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 23 ^
-i "c:\Inputs" ^
-o "c:\Outputs" ^
```

8.1.18 Mode 24 - Global parameters file update from csv file

This mode updates the global parameters file with parameters indicated in a csv file. Input files can be specified with file system path and the csv file path can be specified with v command.

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 24 ^
-v "c:\Inputs\Variable.csv" ^
-i "c:\Inputs\P1.evax" "c:\Inputs\P2.evax" ^
```

8.1.19 Mode 25 - Update project parametrics from csv file

This mode updates a pgmx file parametrically from parameters contained in a csv file. Input files can be specified explicitly and if the output folder is missing it will be directly updated, otherwise a copy will be created in the output folder. Csv file has to be in the same folder of pgmx file and must have the same name. For example, a P1.pgmx must have associated in the same folder a P1.csv file.

The example below shows a batch file with input explicitly defined

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 25 ^
-i "c:\Inputs\P1.pgmx" ^
```

The example below shows a batch file with input explicitly defined and output with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 25 ^
-i "c:\Inputs\P1.pgmx" ^
-o "c:\Outputs" ^
```

The example below shows a batch file with input and output defined with folder

```
Call "c:\Program Files (x86)\Scm Group\Maestro\XConverter.exe" ^
-s -m 25 ^
-i "c:\Inputs" ^
-o "c:\Outputs" ^
```

9 Appendix

9.1 Canonical working fields

Canonical working fields are all working fields where the workpiece is always placed in a quadrant defined by a system reference with positive X axis and positive Y axis.

Lower left working field

Upper left working field

Lower right working field

Upper right working field

9.2 Non canonical working fields

Non canonical working fields are all working fields where system reference is always the cartesian system reference and the workpiece is placed in a quadrant different from the quadrant relative to positive X axis and positive Y axis.

Lower left working field

Upper left working field where the workpiece is placed in the quadrant defined by positive X axis and negative Y axis (Q4).

Lower right working field where the workpiece is placed in the quadrant defined by negative X axis and positive Y axis (Q2).
Upper right working field where the workpiece is placed in the quadrant defined by negative X axis and negative Y axis (Q3).

XCS-Erzeugungsscheckliste

1. Projekt-/Maschinenparameter zuerst setzen, sofern für die Maschine erforderlich.
2. Finished WorkPiece definieren; Raw WorkPiece nur ergänzen, wenn benötigt.
3. Workpiece positionieren und richtigen WorkPlan/Arbeitsbereich sicherstellen.
4. Vor Geometrien den korrekten Plane auswählen/erzeugen.
5. Geometrien in der vorgesehenen Richtung und mit eindeutigem Namen erzeugen.
6. Strategien, Kompensation, Attribute, Maschinenfunktionen und sonstigen aktiven Zustand vor der zugehörigen Bearbeitung setzen bzw. resetteten.
7. Bearbeitung unmittelbar nach der zugehörigen aktiven Geometrie erzeugen; Werkzeug/Prozess/Parameter prüfen.
8. Bei Explicit Toolpaths keine inkompatiblen Lead-in/out-/Strategieanweisungen verwenden.
9. Maschinenabhängige Funktionen nur einsetzen, wenn die Zielmaschine sie unterstützt.
10. XCS anschließend mit Maestro bzw. XConverter importieren/prüfen; insbesondere Parameterreihenfolge und Werkzeugnamen gegen die installierte Maestro-Version validieren.